BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



KHÓA LUẬN CỬ NHÂN

NGÀNH CÔNG NGHỆ THÔNG TIN

MÃ ĐỀ TÀI CNTT_KLCN182

NGHIÊN CỨU PHƯƠNG PHÁP TÍCH HỢP VMAMBA VÀO MÔ HÌNH YOLO26-SEG TRONG PHÂN ĐOẠN POLYP TỪ ẢNH NỘI SOI ĐẠI TRỰC TRÀNG

Sinh viên thực hiện

LÊ ĐỨC LƯƠNG MSSV 2001230490 Lớp 14DHTH09
PHÙNG TUẤN HUY MSSV 2001230312 Lớp 14DHTH13
TRẦN MẠNH TOÀN MSSV 2001230830 Lớp 14DHTH09

Giảng viên hướng dẫn
TS. PHÙNG THẾ BẢO

TP. HỒ CHÍ MINH NĂM 2026

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



KHÓA LUẬN CỬ NHÂN

NGÀNH CÔNG NGHỆ THÔNG TIN

MÃ ĐỀ TÀI CNTT_KLCN182

NGHIÊN CỨU PHƯƠNG PHÁP TÍCH HỢP VMAMBA VÀO MÔ HÌNH YOLO26-SEG TRONG PHÂN ĐOẠN POLYP TỪ ẢNH NỘI SOI ĐẠI TRỰC TRÀNG

Sinh viên thực hiện

LÊ ĐỨC LƯƠNG MSSV 2001230490 Lớp 14DHTH09
PHÙNG TUẤN HUY MSSV 2001230312 Lớp 14DHTH13
TRẦN MẠNH TOÀN MSSV 2001230830 Lớp 14DHTH09

Giảng viên hướng dẫn
TS. PHÙNG THẾ BẢO

TP. HỒ CHÍ MINH NĂM 2026

LỜI CẢM ƠN

Nhóm sinh viên xin cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Công Thương Thành phố Hồ Chí Minh đã trang bị kiến thức và tạo điều kiện để nhóm thực hiện khóa luận. Nhóm xin cảm ơn TS. Phùng Thế Bảo đã hướng dẫn quá trình xác định vấn đề nghiên cứu, xây dựng mô hình và trình bày kết quả.

Nhóm cũng xin cảm ơn các tác giả bộ dữ liệu Kvasir, Kvasir-SEG và cộng đồng phát triển mã nguồn mở Ultralytics, PyTorch, Mamba cùng các thư viện được sử dụng. Những nguồn tài nguyên này giúp nhóm xây dựng thực nghiệm và kiểm tra lại các kết quả của đề tài.

Nhóm trân trọng các ý kiến góp ý nhằm hoàn thiện phương pháp đánh giá và sản phẩm ứng dụng. Những giới hạn của dữ liệu, thực nghiệm và triển khai được trình bày trong báo cáo để xác định phạm vi sử dụng của kết quả.

Nhóm sinh viên thực hiện

LỜI CAM ĐOAN

Nhóm sinh viên cam đoan các kết quả thực nghiệm trong khóa luận được trình bày theo số liệu và cấu hình lưu trữ của đề tài. Những nội dung kế thừa, kết quả công bố và phần mềm sử dụng được dẫn nguồn. Nhóm phân biệt các phép đo đã thực hiện với các nội dung dự kiến phát triển, và chịu trách nhiệm về nội dung báo cáo.

Các nhận xét không vượt quá phạm vi của tập dữ liệu và giao thức đánh giá. Báo cáo không xem sản phẩm minh họa như một thiết bị chẩn đoán, không gán các chỉ số chưa đo cho mô hình và không dùng các kết quả nghiên cứu cũ thay cho nhóm TSVM FP32 của thực nghiệm chính.

Nhóm sinh viên thực hiện

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Nội dung và phương pháp nghiên cứu

.....

.....

Kết quả thực nghiệm và sản phẩm ứng dụng

.....

.....

Hình thức trình bày và nội dung cần hoàn thiện

.....

.....

Đánh giá và kết luận

.....

.....

Giảng viên nhận xét

Ký và ghi rõ họ tên

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

Nội dung và phương pháp nghiên cứu

.....

.....

Kết quả thực nghiệm và sản phẩm ứng dụng

.....

.....

Hình thức trình bày và nội dung cần hoàn thiện

.....

.....

Đánh giá và kết luận

.....

.....

Giảng viên nhận xét

Ký và ghi rõ họ tên

TÓM TẮT KHÓA LUẬN

Khóa luận nghiên cứu phương pháp tích hợp Topology-Shape-aware VMamba vào YOLO26s-seg cho phân đoạn polyp từ ảnh nội soi đại trực tràng. Khối C2TSVMamba thay C2PSA tại tầng 10, kết hợp SS2D bốn hướng với nhánh hình dạng đa tỉ lệ và có hướng. Cổng guidance điều chỉnh đặc trưng trạng thái trước bước hợp nhất. Checkpoint mô hình cải tiến được lưu và suy luận bằng FP32 để bảo toàn biểu diễn số.

Thực nghiệm sử dụng BG20 gồm 1.040 ảnh huấn luyện và 160 ảnh kiểm định, với 20 cặp seed và 100 epoch cho mỗi lượt. Tại epoch có mask mAP50–95 cao nhất của mỗi lượt, baseline đạt 0,718868 ± 0,011768 và TSVM FP32 đạt 0,723171 ± 0,007197. Chênh lệch trung bình là 0,004303; khoảng tin cậy 95% [−0,000228; 0,008834] chứa 0, paired t-test p = 0,0614. Kết quả cho thấy xu hướng tăng trung bình và giảm độ phân tán giữa seed nhưng chưa chứng minh ưu thế trung bình ở ngưỡng 0,05.

Ứng dụng PolypVision cung cấp client Web Spring Boot và Flutter Mobile dùng chung FastAPI. Hai checkpoint thực nghiệm được tích hợp với định danh, mã băm và kiểm tra FP32. Kết luận giới hạn ở ảnh tĩnh và tập kiểm định đã dùng chọn mô hình; cần thêm kiểm thử ngoài, ablation và độ đo biên/topology.

Từ khóa: phân đoạn polyp; YOLO26s-seg; VMamba; SS2D; TSVM; FP32; thực nghiệm nhiều seed.

ABSTRACT

This thesis investigates the integration of a Topology-Shape-aware VMamba block into YOLO26s-seg for colorectal polyp instance segmentation. C2TSVMamba replaces the backbone C2PSA block at layer 10 and combines four-direction selective state-space scanning with multi-scale directional shape features. The improved model uses FP32 checkpoint storage and inference to preserve numerical fidelity.

The BG20 dataset contains 1,040 training and 160 validation images. Forty training runs cover 20 paired seeds for two models, with 100 epochs per run. At each run's maximum validation mask mAP50–95 epoch, baseline and TSVM FP32 achieve 0.718868 ± 0.011768 and 0.723171 ± 0.007197. The paired mean difference is 0.004303, with a 95% confidence interval of [−0.000228, 0.008834] and a paired t-test p-value of 0.0614. The observed mean improvement is therefore inconclusive at the 0.05 significance level.

PolypVision integrates the experiment checkpoints through a shared FastAPI inference service for Spring Boot Web and Flutter Mobile clients. The study separates validation-history summaries, saved-checkpoint inference and published literature results. External testing, controlled ablation and boundary or topology measurements remain necessary before broader claims can be made.

MỤC LỤC

MỞ ĐẦU....0
1 LÝ DO CHỌN ĐỀ TÀI....0
2 MỤC TIÊU VÀ CÂU HỎI NGHIÊN CỨU....0
3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU....0
4 PHƯƠNG PHÁP THỰC HIỆN VÀ ĐÓNG GÓP....0
5 BỐ CỤC BÁO CÁO....0
CHƯƠNG 1 TỔNG QUAN VÀ CƠ SỞ NGHIÊN CỨU....0
1.1 BÀI TOÁN PHÂN ĐOẠN POLYP....0
1.2 CÁC MÔ HÌNH NỀN TẢNG....0
1.3 MAMBA VÀ MÔ HÌNH TRẠNG THÁI THỊ GIÁC....0
1.4 NGHIÊN CỨU LIÊN QUAN VÀ VỊ TRÍ CỦA ĐỀ TÀI....0
1.5 TÓM TẮT CHƯƠNG....0
CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ MÔ HÌNH ĐỀ XUẤT....0
2.1 KIẾN TRÚC ĐỐI CHỨNG YOLO26S SEG....0
2.2 WRAPPER C2TSVMAMBA....0
2.3 CÁC NHÁNH CỦA TSVMAMBA....0
2.4 ĐỘ PHỨC TẠP VÀ ĐỘ CHÍNH XÁC SỐ....0
2.5 HUẤN LUYỆN VÀ KHỞI TẠO TRỌNG SỐ....0
2.6 CÁC CHẾ ĐỘ ABLATION VÀ GIẢ THUYẾT KIỂM CHỨNG....0
2.7 TÓM TẮT CHƯƠNG....0
CHƯƠNG 3 XÂY DỰNG BỘ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU....0
3.1. NGUỒN DỮ LIỆU....0
3.2. XÂY DỰNG VÀ PHÂN CHIA BỘ DỮ LIỆU BG20....0
3.3. CHUYỂN MẶT NẠ PHÂN ĐOẠN THÀNH NHÃN ĐA GIÁC....0
3.4. ĐỔI KÍCH THƯỚC ẢNH....0
3.5. TĂNG CƯỜNG DỮ LIỆU....0
3.6. ĐỊNH DẠNG VÀ CHUẨN HÓA DỮ LIỆU ĐẦU VÀO....0
3.7. KIỂM TRA TÍNH TOÀN VẸN CỦA DỮ LIỆU....0
3.8. TỔNG HỢP QUY TRÌNH TIỀN XỬ LÝ....0
CHƯƠNG 4 CÀI ĐẶT THUẬT TOÁN VÀ ỨNG DỤNG MINH HỌA....0
4.1 TỔ CHỨC CÀI ĐẶT VÀ MÔI TRƯỜNG....0
4.2 QUY TRÌNH NOTEBOOK VÀ LƯU KẾT QUẢ....0
4.3 TỔNG QUAN ỨNG DỤNG POLYPVISION....0
4.4 DỊCH VỤ SUY LUẬN VÀ HỢP ĐỒNG API....0

4.5 CHỨC NĂNG WEB VÀ MOBILE....0
4.6 KIỂM TRA TÍCH HỢP VỚI CHECKPOINT THỰC NGHIỆM....0
4.7 VẬN HÀNH VÀ GIỚI HẠN ỨNG DỤNG....0
4.8 TÓM TẮT CHƯƠNG....0
CHƯƠNG 5 THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ....0
5.1 MỤC TIÊU VÀ TỔ CHỨC THỰC NGHIỆM....0
5.2 QUY TẮC TRÍCH XUẤT VÀ PHƯƠNG PHÁP ĐÁNH GIÁ....0
5.3 CHẤT LƯỢNG PHÂN ĐOẠN VÀ PHÁT HIỆN ĐỐI TƯỢNG....0
5.4 CHÊNH LỆCH GHÉP CẶP VÀ ĐỘ BẤT ĐỊNH....0
5.5 PHÂN BỐ KẾT QUẢ VÀ ĐỘ ỔN ĐỊNH GIỮA SEED....0
5.6 DIỄN BIẾN HUẤN LUYỆN VÀ MẤT MÁT KIỂM ĐỊNH....0
5.7 MA TRẬN NHẦM LẪN VÀ CÁC SAI SỐ PHÁT HIỆN....0
5.8 CHI PHÍ MÔ HÌNH VÀ BENCHMARK SUY LUẬN FP32....0
5.9 ĐỐI CHIẾU VỚI KẾT QUẢ CÔNG BỐ....0
5.10 TỔNG HỢP BẰNG CHỨNG VÀ HẠN CHẾ....0
5.11 TÓM TẮT CHƯƠNG....0
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN....0
KẾT QUẢ ĐẠT ĐƯỢC....0
GIỚI HẠN CỦA KẾT LUẬN....0
HƯỚNG PHÁT TRIỂN....0
TÀI LIỆU THAM KHẢO....0
PHỤ LỤC A NGUỒN SỐ LIỆU VÀ QUY TẮC TÁI LẬP....0
PHỤ LỤC B ĐỐI CHIẾU EPOCH VÀ CHECKPOINT....0
PHỤ LỤC C CẤU HÌNH VÀ KIỂM TRA ỨNG DỤNG....0
PHỤ LỤC D KẾ HOẠCH THỰC NGHIỆM BỔ SUNG....0

DANH MỤC HÌNH

Hình 2.1 Kiến trúc YOLO26s seg và vị trí thay thế module ở tầng 10.....13
Hình 2.2 Hai nhánh SS2D và hình dạng cùng cơ chế guidance trong TSVMamba 17
Hình 3.1. Ví dụ ảnh có polyp, mặt nạ phân đoạn gốc và ảnh nền trong bộ dữ liệu BG20.....23
Hình 3.2. Trước và sau khi chuyển mặt nạ phân đoạn thành nhãn đa giác.....27
Hình 3.3. Trước và sau khi đổi kích thước ảnh bằng thu phóng giữ tỷ lệ và LetterBox.....29
Hình 3.4. Ảnh huấn luyện trước và sau các phép tăng cường dữ liệu.....32
Hình 3.5. Quy trình xây dựng bộ dữ liệu BG20 và tiền xử lý dữ liệu.....35
Hình 4.1 Kiến trúc Web và Mobile dùng chung dịch vụ suy luận FastAPI.....39
Hình 4.2 Ví dụ suy luận thực tế của hai checkpoint seed 0 và đường bao nhãn đối chiếu.....45
Hình 5.1 Trung bình và độ lệch chuẩn mask mAP50 95 của hai mô hình.....51
Hình 5.2 So sánh mask mAP50 trên 20 seed.....52
Hình 5.3 Mask Precision và Recall của hai nhóm thực nghiệm.....53
Hình 5.4 Các chỉ số hộp bao trên hai nhóm 20 seed.....54
Hình 5.5 Kết quả ghép cặp mask mAP50 95 và Precision của 20 seed.....55
Hình 5.6 Chênh lệch trung bình và khoảng tin cậy 95 phần trăm.....56
Hình 5.7 Chênh lệch mask mAP50 95 của từng cặp seed.....57
Hình 5.8 Diễn biến mask mAP50 95 theo chỉ số seed.....58
Hình 5.9 Phân bố mask mAP50 95 của hai mô hình.....59
Hình 5.10 Phân bố tần suất mask mAP50 95 giữa các seed.....60
Hình 5.11 Trung bình và độ lệch chuẩn các đường mAP theo epoch.....60
Hình 5.12 Validation segmentation loss tại epoch chọn theo mask.....61
Hình 5.13 Diễn biến các thành phần validation loss qua 100 epoch.....62
Hình 5.14 So sánh số đối tượng đúng sai và bỏ sót trung bình.....63
Hình 5.15 Ma trận số đếm trung bình của baseline.....64
Hình 5.16 Ma trận số đếm trung bình của TSVM FP32.....64
Hình 5.17 Độ trễ trung bình của forward và pipeline suy luận FP32.....68
Hình 5.18 Các phân vị độ trễ của pipeline predict.....69
Hình 5.19 Các thành phần thời gian được đo thật trong pipeline.....69
Hình 5.20 Dung lượng tệp và bộ nhớ tham số khi dùng float32.....70

DANH MỤC BẢNG

Bảng 1.1 Đối chiếu các công trình gần với đề tài.....11
Bảng 2.1 Các mức đặc trưng của cấu hình small.....13
Bảng 2.2 Các chế độ hỗ trợ trong mã nguồn.....20
Bảng 3.1. Phân bố ảnh và nhãn trong bộ dữ liệu BG20.....24
Bảng 3.2. Thông số tăng cường dữ liệu trong cấu hình huấn luyện.....31
Bảng 4.1 Cấu hình thống nhất của 40 lượt huấn luyện.....36
Bảng 4.2 Các endpoint chính của dịch vụ AI.....41
Bảng 4.3 Phạm vi kiểm tra ứng dụng sau chuyển mô hình.....44
Bảng 5.1 Kết quả tại epoch tối đa mask mAP50 95 của mỗi lượt.....50
Bảng 5.2 Khoảng tin cậy và kiểm định ghép cặp hai phía.....55
Bảng 5.3 Kết quả mask mAP50 95 và epoch theo từng seed.....57
Bảng 5.4 Phân bố mask mAP50 95 theo nhóm.....59
Bảng 5.5 Thống kê bổ sung từ ma trận phát hiện theo đối tượng.....62
Bảng 5.6 Quy mô hai checkpoint được tích hợp vào ứng dụng.....66
Bảng 5.7 Kết quả benchmark CPU FP32 với 100 lượt đo mỗi chế độ.....67
Bảng 5.8 Kết quả tham khảo do tác giả công bố trên Kvasir SEG.....71
Bảng B.1 Epoch chọn theo mask và theo fitness của 40 lượt.....79
Bảng D.1 Các phép kiểm cần bổ sung trước khi mở rộng kết luận.....82

DANH MỤC CHỮ VIẾT TẮT

Các thuật ngữ dùng trong báo cáo

Ký hiệu	Diễn giải
AI	Trí tuệ nhân tạo
API	Giao diện lập trình ứng dụng
BG20	Bổ sung 200 ảnh nền trên 1.000 ảnh polyp gốc
CNN	Mạng nơ-ron tích chập
SSM	Mô hình không gian trạng thái
SS2D	Quét chọn lọc hai chiều
TSVM	Topology-Shape-aware VMamba
FP16 và FP32	Số thực dấu phẩy động 16 bit và 32 bit
EMA	Trung bình trượt hàm mũ của trọng số
IoU	Phần giao trên phần hợp
Dice	Hệ số tương đồng Dice
AP và mAP	Diện tích đường precision–recall và trung bình AP
TP FP FN	Đúng dương tính, sai dương tính, sai âm tính
KTC	Khoảng tin cậy
SD	Độ lệch chuẩn mẫu
Seed	Hạt giống của bộ sinh số ngẫu nhiên
P3 P4 P5	Mức đặc trưng có stride 8, 16 và 32
FP32 checkpoint	Checkpoint giữ tensor dấu phẩy động ở float32

MỞ ĐẦU

1 LÝ DO CHỌN ĐỀ TÀI

Phân đoạn polyp từ ảnh nội soi là bài toán xác định vùng tổn thương ở mức điểm ảnh. So với việc chỉ trả về hộp bao, mặt nạ cung cấp mô tả cụ thể hơn về hình dạng và vị trí của đối tượng. Tuy nhiên, màu sắc polyp có thể gần với niêm mạc xung quanh; nếp gấp, phản quang và thay đổi kích thước quan sát cũng làm việc phân biệt đối tượng trở nên khó khăn. Các bộ dữ liệu công khai như Kvasir-SEG tạo điều kiện nghiên cứu những vấn đề này trong một quy trình có thể kiểm tra lại [1], [2].

YOLO26-seg cung cấp khung phát hiện và phân đoạn từng đối tượng trong một mô hình thống nhất. Trong khi đó, Visual State Space Model và cơ chế SS2D của VMamba tạo cách truyền thông tin giữa các vị trí xa nhau trên bản đồ đặc trưng [15], [16]. Đề tài khảo sát việc kết hợp ngữ cảnh toàn cục đó với đặc trưng hình dạng cục bộ ngay trong backbone YOLO26s-seg. Câu hỏi nghiên cứu là liệu khối kết hợp có cải thiện chất lượng phân đoạn và tính ổn định qua các lần khởi tạo, khi giữ cố định dữ liệu và siêu tham số huấn luyện hay không.

Đề tài triển khai C2TSVMamba tại tầng 10, đồng thời xử lý vấn đề độ chính xác số khi lưu và nạp checkpoint. Cặp đối chứng chính gồm YOLO26s-seg cơ bản và YOLO26s-seg tích hợp TSVM với checkpoint FP32. Kết quả được xem xét trên 20 cặp seed, thay vì lựa chọn một lượt chạy thuận lợi để đại diện cho toàn bộ phương pháp.

Trong bài toán xử lý ảnh nội soi, phát hiện và phân đoạn giải quyết hai mức yêu cầu khác nhau. Hộp bao giúp định vị nhanh một vùng nghi ngờ nhưng có thể bao gồm một phần niêm mạc ngoài đối tượng. Mặt nạ mô tả vùng đối tượng chi tiết hơn, thuận lợi cho việc quan sát đường bao và đối chiếu dự đoán với nhãn chuyên gia của bộ dữ liệu. Tuy vậy, đầu ra mặt nạ vẫn chỉ là kết quả của mô hình trên ảnh quan sát; nó không cung cấp kết luận mô bệnh học hay xác định mức độ ác tính. Việc chọn phân đoạn từng đối tượng trong đề tài xuất phát từ yêu cầu kỹ thuật mô tả vùng polyp cụ thể hơn, đồng thời duy trì đầu ra hộp bao và độ tin cậy trong cùng một luồng suy luận.

Một khó khăn của nghiên cứu là kết quả tốt trong một lượt huấn luyện chưa phản ánh đầy đủ hành vi của kiến trúc. Khởi tạo trọng số, thứ tự mẫu và các phép tăng cường dữ liệu có thể làm thay đổi quá trình tối ưu hóa ngay cả khi dùng

cùng tập chia. Nếu chỉ chọn lượt tốt nhất của mỗi mô hình, sự khác biệt quan sát được có thể trộn lẫn tác động của thiết kế với thuận lợi ngẫu nhiên của lượt chạy. Vì vậy, đề tài sử dụng 20 seed cho mỗi kiến trúc, trình bày cả mức trung bình, độ phân tán và chênh lệch ghép cặp. Cách tiếp cận này giúp đặt câu hỏi về độ ổn định của kết quả bên cạnh câu hỏi về mức chính xác cao nhất.

Đề tài cũng quan tâm đến khoảng cách giữa mã huấn luyện và phần mềm minh họa. Một checkpoint có thể tồn tại trên đĩa nhưng vẫn không dùng được trong ứng dụng nếu thiếu lớp tùy biến, sai quy mô mô hình hoặc sai kiểu dữ liệu khi nạp. Đối với SS2D tùy biến, chuyển kiểu số trong quá trình lưu còn có thể ảnh hưởng các tensor trạng thái. Vì vậy, tính tương thích của checkpoint FP32, tên định danh mô hình và đường truyền ảnh được xem là bộ phận của quy trình thực hiện, thay vì chỉ là công việc bổ sung sau khi hoàn thành bảng kết quả.

2 MỤC TIÊU VÀ CÂU HỎI NGHIÊN CỨU

Mục tiêu tổng quát là xây dựng, giải thích và đánh giá phương pháp tích hợp TSVM vào YOLO26s-seg cho phân đoạn polyp trên bộ dữ liệu BG20, sau đó tích hợp hai mô hình vào ứng dụng minh họa PolypVision. Mục tiêu cụ thể gồm mô tả đúng luồng tính toán của khối đề xuất; bảo đảm các lượt huấn luyện có thể đối chiếu; phân tích chênh lệch trung bình và biến thiên giữa seed; và kiểm tra việc nạp checkpoint thực nghiệm vào dịch vụ suy luận.

Ba câu hỏi được đặt ra. Thứ nhất, TSVM FP32 có đạt mask mAP50–95 trung bình cao hơn baseline trên tập kiểm định cố định không? Thứ hai, phân bố kết quả giữa 20 seed có thay đổi và bằng chứng thống kê hỗ trợ đến mức nào? Thứ ba, các checkpoint đã lưu có thể được nạp đúng kiến trúc và suy luận trong cùng luồng ứng dụng Web và Mobile hay không? Câu hỏi thứ ba là kiểm tra kỹ thuật triển khai, không phải kiểm chứng hiệu quả lâm sàng.

Để thực hiện mục tiêu trên, khóa luận triển khai các nhiệm vụ gắn với những sản phẩm có thể kiểm tra. Nhiệm vụ về dữ liệu tạo bộ ảnh và nhãn YOLO thống nhất, gồm cả ảnh không có đối tượng; nhiệm vụ về mô hình xác định vị trí thay thế C2PSA, mô tả C2TSVMamba và kiểm tra luồng kích thước tensor; nhiệm vụ về thực nghiệm tổ chức kết quả riêng theo seed và kiểm tra cấu hình đã lưu. Nhiệm vụ phân tích tổng hợp số liệu theo một quy tắc chọn epoch công khai, còn nhiệm vụ ứng dụng kết nối checkpoint thực nghiệm với dịch vụ AI và hai client Web, Mobile.

Các nhiệm vụ này tương ứng với bốn nhóm căn cứ trong báo cáo: mã nguồn và YAML để giải thích thiết kế; ảnh, nhãn và thống kê dữ liệu để mô tả đầu vào; args.yaml, results.csv và tệp tổng hợp để đánh giá thực nghiệm; mã ứng dụng, manifest checkpoint và kiểm tra tích hợp để xác nhận khả năng sử dụng trong phần mềm. Khi một kết luận không có căn cứ tương ứng, báo cáo chỉ nêu nó như giả thuyết hoặc hướng phát triển. Chẳng hạn, nhánh có hướng tạo động cơ khảo sát biên, nhưng chưa đủ để tuyên bố cải thiện HD95 khi thực nghiệm chưa đo chỉ số này.

Mức hoàn thành của nghiên cứu được xác định bằng việc thiết kế có thể được giải thích, kết quả có thể được truy nguyên và checkpoint có thể được nạp đúng trong ứng dụng. Mức tăng của một chỉ số không phải điều kiện để lựa chọn hoặc loại bỏ số liệu. Nếu kết quả ghép cặp chưa đủ bằng chứng thống kê, khóa luận vẫn cần báo cáo đầy đủ kết quả đó và xác định thí nghiệm tiếp theo cần thực hiện.

3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu là ảnh nội soi tĩnh có polyp và ảnh normal-cecum từ nguồn công khai. Bộ BG20 gồm 1.200 ảnh, chia thành 1.040 ảnh huấn luyện và 160 ảnh kiểm định. Chỉ có một lớp đối tượng là polyp; ảnh nền mang nhãn rỗng. Thực nghiệm chính gồm seed 0 đến 19 của hai mô hình, mỗi lượt 100 epoch và kích thước ảnh cấu hình 640 pixel.

Các kết quả TSVM không mang hậu tố FP32 thuộc giai đoạn nghiên cứu trước và không được gộp vào phép so sánh chính. Các bài báo quốc tế được dùng để định vị phương pháp và tham khảo kết quả đã công bố, không được xem là những mô hình mà nhóm đã chạy lại. Bộ dữ liệu hiện có không cung cấp tập kiểm thử độc lập hoặc định danh bệnh nhân; vì vậy, kết luận định lượng giới hạn ở tập kiểm định BG20 và biến thiên tối ưu hóa theo seed.

Trong phạm vi dữ liệu, BG20 là tên thiết lập bổ sung 200 ảnh nền vào 1.000 ảnh Kvasir-SEG, không có nghĩa 20% tổng 1.200 ảnh đều là ảnh nền. Tỷ lệ nền trên toàn bộ dữ liệu là 16,67%; các tập train và validation có tỷ lệ khác nhau theo danh sách đã được tạo. Các seed của thực nghiệm thay đổi quá trình huấn luyện trên cùng danh sách này, không tạo lại 20 phép chia dữ liệu. Vì vậy, nghiên cứu đánh giá biến thiên do tối ưu hóa trong một thiết lập cố định và chưa lượng hóa toàn bộ bất định do lấy mẫu dữ liệu.

Trong phạm vi hệ thống, Web và Mobile sử dụng ảnh tĩnh tải lên. Báo cáo phân biệt thời gian tính toán mô hình với thời gian của một yêu cầu ứng dụng, vì yêu cầu còn có truyền mạng, giải mã ảnh và tạo kết quả hiển thị. Thực nghiệm hiệu năng chỉ được sử dụng trong điều kiện phần cứng, số lần lặp và phạm vi đo đã công bố. Việc chạy được trên máy cá nhân chưa đủ để suy rộng sang thiết bị nội soi chuyên dụng hoặc khẳng định vận hành video thời gian thực.

4 PHƯƠNG PHÁP THỰC HIỆN VÀ ĐÓNG GÓP

Đề tài sử dụng phương pháp thực nghiệm có đối chứng. Hai kiến trúc được huấn luyện với cùng tập ảnh, nhãn và cấu hình tối ưu hóa. Mỗi lượt chạy được lưu riêng results.csv và args.yaml; số liệu được ghép theo seed để tính chênh lệch, khoảng tin cậy và kiểm định. Khi có khác biệt giữa mô tả và cấu hình đã lưu, cấu hình cùng số liệu gốc là căn cứ cho phần thực nghiệm.

Đóng góp của đề tài nằm ở thiết kế khối TSVM kết hợp SS2D với nhánh hình dạng đa tỉ lệ và có hướng trong wrapper C2; phân tích khả năng tích hợp tại tầng sâu của YOLO26s-seg; duy trì checkpoint FP32 cho nhánh trạng thái; và nối kết kết quả thực nghiệm với ứng dụng minh họa. Những đóng góp này cần được phân biệt với các ý tưởng có trước về Mamba cho polyp, topology guidance và convolutional gating.

Quy trình nghiên cứu bắt đầu bằng khảo sát các hướng phân đoạn dựa trên CNN, mô hình trạng thái và cơ chế hướng dẫn cấu trúc, sau đó đối chiếu với mã nguồn đang sử dụng. Bước mô hình hóa xác định giả thuyết: ngữ cảnh từ SS2D có thể bổ sung thông tin cho nhánh hình dạng, còn nhánh hình dạng có thể điều chỉnh đáp ứng toàn cục trước hợp nhất. Bước thực nghiệm giữ cố định những điều kiện có thể đối chiếu và lưu kết quả của cả hai nhóm. Bước phân tích xem xét giả thuyết qua độ đo đã xác định, không suy ngược cơ chế từ một biểu đồ thuận lợi.

Giá trị khoa học của khóa luận nằm ở một trường hợp tích hợp cụ thể được mô tả đến mức có thể tái lập, cùng cách phân biệt kiến trúc, độ chính xác số và quy tắc tổng hợp kết quả. Giá trị thực hành nằm ở quy trình chuyển đổi nhãn, tổ chức các lượt chạy và tích hợp checkpoint vào PolypVision. Các sản phẩm này có thể phục vụ học tập, trình diễn và kiểm thử phương pháp trên ảnh công khai. Chúng chưa thay thế một nghiên cứu đánh giá lâm sàng trên bệnh nhân hoặc một hệ thống được chứng nhận sử dụng trong khám chữa bệnh.

5 BỐ CỤC BÁO CÁO

Chương 1 trình bày cơ sở và nghiên cứu liên quan. Chương 2 mô tả kiến trúc, luồng dữ liệu và độ chính xác số. Chương 3 trình bày dữ liệu và tiền xử lý. Chương 4 mô tả notebook huấn luyện cùng ứng dụng PolypVision. Chương 5 trình bày quy tắc so sánh, kết quả 20 seed, các giới hạn và đối chiếu tài liệu. Phần kết luận tổng hợp kết quả và xác định các công việc cần kiểm chứng tiếp theo.

CHƯƠNG 1 TỔNG QUAN VÀ CƠ SỞ NGHIÊN CỨU

1.1 BÀI TOÁN PHÂN ĐOẠN POLYP

1.1.1 Phân đoạn ngữ nghĩa và phân đoạn từng đối tượng

Phân đoạn ngữ nghĩa gán một lớp cho mỗi điểm ảnh, trong khi phân đoạn từng đối tượng còn tách các đối tượng cùng lớp thành những mặt nạ riêng. Trong BG20, một ảnh có thể có nhiều vùng polyp tách rời; vì vậy số đối tượng lớn hơn số ảnh có polyp. YOLO26-seg xử lý bài toán theo hướng từng đối tượng, đồng thời xuất hộp bao, độ tin cậy và mặt nạ. Đặc điểm này quyết định cách sử dụng Precision, Recall và AP trong đánh giá.

Đối với ảnh nền, hệ thống cần có khả năng không trả về đối tượng. Một lớp background trong ma trận phát hiện thường biểu thị các trường hợp không ghép được giữa dự đoán và nhãn; nó không đồng nghĩa với việc bộ dữ liệu có một lớp bệnh học thứ hai. Tương tự, số dương tính giả tính theo đối tượng không tự chuyển được thành số ảnh nền có báo động giả.

Có thể biểu diễn một mẫu có nhãn bằng cặp ảnh I và tập đối tượng $G = \{g_1, g_2, \ldots, g_m\}$. Mỗi đối tượng đi kèm lớp, hộp bao suy ra từ vùng nhãn và mặt nạ hoặc đa giác. Đầu ra mô hình là một tập dự đoán có số phần tử thay đổi, mỗi phần tử gồm hộp bao, điểm tin cậy, lớp và mặt nạ. Với ảnh nền, m bằng 0 và đầu ra mong muốn là tập rỗng. Biểu diễn này giải thích vì sao số đối tượng, số ảnh và số pixel phải được thống kê riêng, cũng như vì sao tệp nhãn rỗng là một mẫu hợp lệ của bài toán.

Từ góc độ đánh giá, mô hình có thể định vị một đối tượng tương đối đúng nhưng dự đoán đường bao còn thô; ngược lại, mặt nạ có hình dạng hợp lý nhưng không được ghép thành true positive nếu không đạt ngưỡng IoU đang xét. Vì vậy, một kết quả hộp bao tốt chưa thay thế được đánh giá mặt nạ. Báo cáo giữ riêng các chỉ số Box và Mask, đồng thời không đồng nhất F1 tính trên đối tượng với Dice tính trên pixel. Hai đại lượng chỉ có cùng dạng công thức khi dùng cùng định nghĩa TP, FP và FN trên cùng không gian đánh giá.

1.1.2 Các khó khăn về hình ảnh và dữ liệu

Biên mờ có thể khiến mặt nạ thu hẹp hoặc lan sang niêm mạc; phản quang có thể tạo khoảng trống trong vùng dự đoán. Polyp nhỏ cần giữ được chi tiết không gian, còn polyp lớn cần thông tin ngữ cảnh rộng. Tuy nhiên, những mô tả này là

động cơ thiết kế. Muốn khẳng định một module cải thiện biên hoặc liên thông, cần độ đo và tập ca kiểm chứng tương ứng, ngoài các chỉ số phát hiện tổng hợp.

Kvasir-SEG và ảnh normal-cecum có khác biệt về bối cảnh giải phẫu, thiết bị và các ký hiệu xuất hiện trong khung hình. Việc thêm mẫu âm có ích cho nghiên cứu phát hiện sai, nhưng nguồn âm chỉ từ một mốc giải phẫu chưa đại diện cho mọi niêm mạc lành. Do đó, phân bố BG20 là một thiết lập nghiên cứu cụ thể, chưa phản ánh đầy đủ phân bố khung hình của một phiên nội soi.

Các khó khăn về hình ảnh còn tác động khác nhau ở từng giai đoạn xử lý. Khi co ảnh về kích thước đầu vào, đối tượng chiếm ít diện tích có thể chỉ còn một số ít ô đặc trưng ở tầng sâu. Khi raster hóa đa giác hoặc tăng lại mặt nạ về kích thước ảnh gốc, đường bao có thể thay đổi do nội suy và ngưỡng hóa. Do đó, sai khác trên mặt nạ cuối cùng không nhất thiết chỉ bắt nguồn từ khối trích đặc trưng. Quy trình đánh giá cần giữ thống nhất kích thước đầu vào, cách hậu xử lý và dạng nhãn được dùng làm chuẩn.

Ảnh nội soi cũng có những dấu hiệu ngoài đối tượng như viền đen, ký hiệu và vùng thông tin chèn vào khung hình. Nếu các dấu hiệu đó liên quan đến nguồn dữ liệu, mô hình có thể tận dụng đặc điểm nguồn thay vì chỉ học cấu trúc polyp. Kiểm tra trùng ảnh bằng mã băm là một bước kiểm soát hữu ích nhưng chỉ loại trừ trùng khớp tệp chính xác; nó chưa xác định được ảnh gần trùng, các khung hình liền nhau hoặc nhiều ảnh của cùng bệnh nhân. Đây là lý do cần diễn giải mức tổng quát hóa phù hợp với metadata hiện có.

1.2 CÁC MÔ HÌNH NỀN TẢNG

1.2.1 Mạng tích chập và kiến trúc encoder decoder

Mạng tích chập học các mẫu cục bộ bằng kernel dùng chung giữa các vị trí. Depthwise convolution thực hiện lọc riêng từng kênh, còn convolution 1×1 trộn thông tin giữa các kênh. Những thao tác này được dùng trong nhánh hình dạng của TSVM để tạo nhiều trường tiếp nhận mà không cần attention theo mọi cặp vị trí.

U-Net tổ chức encoder và decoder với các kết nối bỏ qua để kết hợp thông tin ngữ nghĩa sâu và chi tiết ở độ phân giải cao [12]. PraNet khai thác reverse attention cho phân đoạn polyp [13]. Các hướng này cho thấy thông tin toàn cục và chi tiết biên cần được kết hợp ở nhiều mức, nhưng cách kết hợp và nhiệm vụ đầu ra khác nhau giữa từng mô hình.

Trong các tầng nông, một kernel nhỏ quan sát vùng lân cận và học những biến đổi màu sắc, cạnh hoặc kết cấu. Khi xếp chồng nhiều tầng và giảm độ phân giải, trường tiếp nhận lý thuyết mở rộng, đồng thời đặc trưng mang nhiều thông tin ngữ nghĩa hơn. Vì vậy, không chính xác khi mô tả mọi mạng tích chập là hoàn toàn không có ngữ cảnh toàn cục. Vấn đề thiết kế là cơ chế trao đổi thông tin giữa các vị trí, chi phí tính toán và mức bảo toàn chi tiết không gian. Đề tài đưa thêm nhánh trạng thái để khảo sát một cách trao đổi thông tin bổ sung cho nền tích chập đã có.

Kết nối bỏ qua và hợp nhất đa mức giải quyết yêu cầu kết hợp ngữ nghĩa với định vị ở những cách khác nhau. Trong mạng encoder decoder, các tầng nông có thể được nối trực tiếp với tầng giải mã tương ứng; trong YOLO26-seg, các mức đặc trưng đi qua neck trước khi đầu dự đoán sử dụng chúng. Sự khác biệt về đường truyền đặc trưng và đầu ra khiến một module đạt kết quả tốt trong U-Net chưa đương nhiên đạt cùng tác dụng khi chuyển sang YOLO. Việc tích hợp cần được đánh giá như một cấu hình mới, dù nguyên lý thành phần có thể đã được công bố.

1.2.2 YOLO26 và đầu phân đoạn

Trong mã được sử dụng, YOLO26s-seg gồm backbone, neck hợp nhất đa mức và đầu Segment26. Backbone cơ bản kết thúc bằng SPPF và C2PSA. Đầu phân đoạn nhận đặc trưng P3, P4 và P5; mặt nạ được tạo từ các prototype và hệ số của từng đối tượng. Cấu hình end2end được bật và reg_max bằng 1. Vì vậy không nên diễn giải YOLO26 như một bản sao nguyên trạng của các phiên bản YOLO dùng DFL với nhiều bin [16].

Chữ s xác định quy mô small thông qua hệ số depth bằng 0,50 và width bằng 0,50. Mã nguồn và notebook của đề tài tạo YAML có scale s rõ ràng. Việc chỉ mở tệp YAML chung mà không chỉ định scale có thể dẫn đến sử dụng biến thể mặc định khác; đây là một yếu tố cần kiểm tra khi tái lập hoặc khi nạp mô hình từ kiến trúc.

Cơ chế mặt nạ nguyên mẫu giúp phân chia công việc thành hai phần. Nhánh prototype tạo một cơ sở mặt nạ dùng chung cho ảnh; nhánh hệ số xác định cách kết hợp cơ sở đó cho từng dự đoán. Số prototype là chiều biểu diễn của tổ hợp này, không phải số lượng polyp tối đa trong ảnh. Những mặt nạ prototype trung gian cũng không được xem riêng như nhãn giải phẫu vì chúng được tối ưu để phối hợp với hệ số đầu ra.

Ở giai đoạn suy luận, đầu ra cần được chuyển từ không gian đặc trưng về ảnh quan sát và áp dụng quy tắc chọn dự đoán của phiên bản đang dùng. Vì vậy, khi tái lập cần cố định không chỉ tên YOLO mà cả phiên bản Ultralytics, chế độ end2end, ngưỡng tin cậy và các tham số hậu xử lý liên quan. Việc đổi tên tệp trọng số hoặc dùng một lớp head khác có thể làm thay đổi hành vi ngay cả khi giao diện hiển thị vẫn ghi cùng tên mô hình.

1.3 MAMBA VÀ MÔ HÌNH TRẠNG THÁI THỊ GIÁC

1.3.1 Biểu diễn trạng thái chọn lọc

Một mô hình trạng thái rời rạc truyền thông tin qua trạng thái ẩn h. Với Mamba, một số tham số phụ thuộc đầu vào, cho phép đáp ứng thay đổi theo nội dung thay vì áp dụng cùng một bộ lọc cố định cho toàn chuỗi [14]. Trong triển khai của đề tài, bước thời gian Δ được tạo qua softplus, ma trận chuyển trạng thái A có giá trị âm và mỗi vị trí tạo các hệ số B, C.

$$h_t = \exp(\Delta_t A) \odot h_{t-1} + \Delta_t B_t u_t \; ; \; y_t = \Sigma_n \, C_{t,n} h_{t,n}$$

Biểu thức trên mô tả phép lặp thực tế trong module, với thành phần đầu vào được tính bằng ΔBu. Đây là cách cài đặt trạng thái chọn lọc của đề tài; không đồng nhất toàn bộ các thành phần hoặc tối ưu phần cứng của thư viện Mamba gốc.

Trạng thái ẩn là một biểu diễn nén được cập nhật lần lượt theo vị trí chuỗi. Với cùng một đầu vào hiện tại, đầu ra có thể khác nhau nếu trạng thái tích lũy từ các vị trí trước khác nhau. Trong selective SSM, các đại lượng phụ thuộc nội dung làm cho mức cập nhật và cách đọc trạng thái thay đổi theo dữ liệu. Tuy nhiên, không nên gán sẵn ý nghĩa y khoa cho mỗi tham số, chẳng hạn khẳng định Δ luôn nhỏ ở vùng phản quang và luôn lớn tại biên polyp. Ý nghĩa đó chỉ có thể được khảo sát qua phân tích đáp ứng thực tế, chưa được bảo đảm bởi cấu trúc phương trình.

Ma trận chuyển trạng thái được tham số hóa để có các hệ số suy giảm; bước thời gian dương giúp định nghĩa mức chuyển tiếp tại từng vị trí. Tính chất này hỗ trợ cách mô hình hóa bộ nhớ trên chuỗi nhưng không tự bảo đảm mọi phép tính luôn ổn định ở tất cả kiểu số. Giá trị đầu vào, phép chiếu, phép tích lũy và các thống kê chuẩn hóa vẫn có thể có miền giá trị rộng. Vì vậy, phân tích cấu trúc trạng thái và kiểm tra kiểu số của checkpoint là hai công việc cần đi cùng nhau trong triển khai của khóa luận.

1.3.2 Cơ chế SS2D bốn hướng

VMamba mở rộng cách xử lý chuỗi sang dữ liệu ảnh thông qua SS2D [15]. Một bản đồ H×W được sắp thành chuỗi theo hàng và theo cột; hai chuỗi đảo chiều bổ sung hai hướng ngược lại. Kết quả được đưa về đúng vị trí không gian rồi hợp nhất. Bốn hướng này là hàng xuôi, hàng ngược, cột xuôi và cột ngược, không phải bốn đường quét chéo.

Ở tầng P5 với ảnh 640×640, chiều dài chuỗi là L = 20×20 = 400. Đặt module ở độ phân giải thấp giúp giới hạn chi phí quét nhưng có thể làm mất chi tiết biên nhỏ. Vì thế lợi ích dự kiến của nhánh hình dạng tại P5 cần được đánh giá thực nghiệm; không thể suy ra độ chính xác biên trực tiếp từ vị trí module.

Nếu chỉ trải bản đồ theo hàng, hai vị trí kề nhau theo cột có thể cách nhau W bước trên chuỗi, trong khi các vị trí ở cuối hàng và đầu hàng tiếp theo lại liên tiếp trong thứ tự xử lý. Bổ sung thứ tự theo cột giúp mạng có thêm một cách tiếp cận quan hệ lân cận. Các hướng đảo chiều cung cấp phụ thuộc từ chiều ngược lại khi kết quả được hợp nhất. Đây là cách xử lý ảnh thông qua nhiều thứ tự chuỗi, chưa phải một chứng minh rằng đầu ra bất biến với phép quay hoặc phản chiếu ảnh.

Sau quét, các phép đảo và chuyển vị phải được thực hiện ngược đúng với quá trình tạo chuỗi. Nếu ghép một chuỗi chưa phục hồi về cùng tọa độ, phép cộng sẽ kết hợp thông tin của những vị trí không tương ứng. Việc giải thích SS2D vì thế cần mô tả cả thứ tự quét lẫn bước đưa kết quả trở lại lưới H×W, thay vì chỉ liệt kê bốn hướng. Mã nguồn của đề tài thực hiện cả hai thao tác trước khi đưa tensor đến phép điều biến và chiếu đầu ra.

1.4 NGHIÊN CỨU LIÊN QUAN VÀ VỊ TRÍ CỦA ĐỀ TÀI

1.4.1 Mamba cho phân đoạn polyp

Polyp-Mamba kết hợp biểu diễn đa tỉ lệ và cơ chế đưa ngữ nghĩa toàn cục vào đặc trưng cục bộ [17]. Công trình này cho thấy việc dùng Visual Mamba trong phân đoạn polyp đã có trước đề tài. Điểm cần đối chiếu là cách xây dựng nhánh hình dạng và vị trí tích hợp trong YOLO26s-seg, thay vì chỉ sự xuất hiện của Mamba.

Topo-VM-UNetV2 sử dụng persistent homology từ probability map để tạo topology attention map và đưa vào Topo-SDI [18]. TSVM hiện tại tạo guidance trực tiếp từ các convolution học được. Do đó, hai mô hình có động cơ gần nhau

nhưng nguồn tín hiệu topology khác nhau; TSVM chưa có phép đo hoặc ràng buộc topology tường minh như persistent homology.

CSG-Mamba đặt convolutional scoring gate tại bottleneck của mạng Vision Mamba để điều chỉnh đặc trưng bằng thông tin cục bộ [19]. Đây là đối chiếu gần về cơ chế gating. TSVM sử dụng nhiều kích thước và hướng kernel, phép điều chỉnh 1+G và hợp nhất thêm nhánh hình dạng trong wrapper C2.

1.4.2 Mamba trong YOLO và các hướng về biên

MambaPSA thay C2PSA cuối backbone YOLO26 bằng lõi Mamba, giữ cơ chế chia nhánh và ghép kênh, đồng thời khảo sát vị trí module trong neck [20]. Công trình đánh giá phát hiện đối tượng trên PASCAL VOC. Vì vậy, đóng góp của đề tài không được định nghĩa đơn thuần là thay C2PSA bằng Mamba trong YOLO26; nó gắn với thiết kế TSVM và thực nghiệm phân đoạn polyp.

SAM-MaGuP kết hợp boundary distillation và Mamba adapter với SAM cho polyp [21]. Hướng nghiên cứu này nhấn mạnh nhu cầu kiểm chứng biên riêng. Trong đề tài, kết quả mAP chưa đủ để suy ra lợi ích trên HD95, Boundary F1 hoặc tính liên thông; những phép đo đó được xác định là phần mở rộng.

Bảng 1.1 Đối chiếu các công trình gần với đề tài

Công trình	Thành phần trọng tâm	Khác biệt của đề tài
Polyp-Mamba [17]	Ngữ nghĩa đa tỉ lệ và global injection	Tích hợp trong YOLO26s-seg
Topo-VM-UNetV2 [18]	Persistent homology và Topo-SDI	Guidance học từ convolution
CSG-Mamba [19]	Convolutional gate tại bottleneck	Nhánh đa hướng và cổng 1+G
MambaPSA [20]	Thay C2PSA trong YOLO26	Shape guidance cho polyp instance segmentation
SAM-MaGuP [21]	SAM và boundary distillation	Không sử dụng SAM hoặc boundary distillation

1.4.3 Phạm vi tính mới

Đề tài xem C2TSVMamba là một thiết kế kết hợp có thể kiểm chứng, không tuyên bố phát minh ra SS2D, convolution đa tỉ lệ hoặc sigmoid gating. Điểm

phân biệt gồm nhánh hình dạng có hướng, guidance một chiều từ nhánh đó đến SS2D, cách hợp nhất hai nhánh và tích hợp vào YOLO26s-seg tại tầng 10. Tuy nhiên, các đóng góp riêng của từng thành phần chỉ có thể tách bạch bằng ablation có đối chứng.

Từ topology trong tên module biểu thị định hướng học cấu trúc. Các kernel học được không bảo đảm số thành phần liên thông hoặc số lỗ của mặt nạ. Khóa luận vì vậy phân biệt cơ chế kiến trúc, giả thuyết tác dụng và kết quả đã đo; không dùng tên module thay cho bằng chứng về bảo toàn topology.

Tính mới được xem xét trên ba mức. Ở mức nguyên lý, selective state space, quét nhiều hướng, convolution đa tỉ lệ và gating đều có các công trình trước. Ở mức tổ chức kiến trúc, đề tài xác định một đường truyền guidance cụ thể từ biểu diễn hình dạng sang nhánh SS2D rồi hợp nhất trong wrapper C2 tại P5. Ở mức thực nghiệm và triển khai, đề tài tổ chức 20 cặp seed trên BG20 và xử lý tính tương thích của checkpoint FP32 với ứng dụng. Việc tách ba mức giúp xác định đóng góp đúng phạm vi và tránh biến một tổ hợp kỹ thuật thành tuyên bố ưu tiên tuyệt đối.

Khảo sát tài liệu là căn cứ định vị chứ chưa thể chứng minh không có bất kỳ nhóm nào trên thế giới nghiên cứu cấu hình gần giống. Đặc biệt, các công trình gần về Mamba trong YOLO26 hoặc guidance cấu trúc cần được đối chiếu theo thời điểm công bố, nhiệm vụ, vị trí chèn module và loại tín hiệu điều khiển. Khóa luận vì vậy sử dụng cách diễn đạt “đề xuất và đánh giá một cấu hình tích hợp” thay cho khẳng định “lần đầu tiên” khi chưa có bằng chứng đầy đủ về quyền ưu tiên.

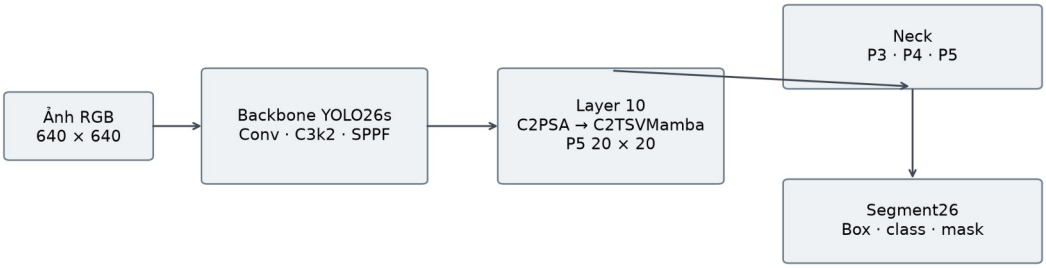
1.5 TÓM TẮT CHƯƠNG

Các nghiên cứu liên quan cho thấy bài toán kết hợp ngữ cảnh dài hạn và chi tiết hình dạng đã được quan tâm trong phân đoạn polyp. Đề tài chọn một vị trí tích hợp cụ thể trong YOLO26s-seg và kiểm tra bằng thực nghiệm ghép cặp nhiều seed. Cơ sở này dẫn đến yêu cầu mô tả chính xác khối TSVM và phân biệt tác động kiến trúc với độ chính xác số của checkpoint ở chương tiếp theo.

CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ MÔ HÌNH ĐỀ XUẤT

2.1 KIẾN TRÚC ĐỐI CHỨNG YOLO26S SEG

Mô hình cơ bản sử dụng cấu hình yolo26-seg với scale s, số lớp được thay bằng một lớp polyp khi huấn luyện. Các khối Conv giảm độ phân giải; C3k2 học và hợp nhất đặc trưng; SPPF bổ sung trường tiếp nhận; C2PSA thực hiện xử lý đặc trưng ở cuối backbone. Neck kết hợp các mức sâu và nông để đầu Segment26 dự đoán trên ba thang không gian.



Hình 2.1 Kiến trúc YOLO26s seg và vị trí thay thế module ở tầng 10

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Khối thay thế có cùng kích thước đầu vào và đầu ra với C2PSA. Những kết nối lên neck và các chỉ số tầng ở đầu phân đoạn được giữ tương thích. Cách can thiệp này giúp xác định rõ phần kiến trúc thay đổi, nhưng không có nghĩa tổng số tham số hoặc động lực tối ưu hóa của hai mô hình giống nhau.

Bảng 2.1 Các mức đặc trưng của cấu hình small

Vị trí	Stride	Kích thước với 640	Số kênh
Backbone P3	8	80 × 80	256
Backbone P4	16	40 × 40	512
Backbone P5 và layer 10	32	20 × 20	512
Head P3 layer 16	8	80 × 80	128

Vị trí	Stride	Kích thước với 640	Số kênh
Head P4 layer 19	16	40 × 40	256
Head P5 layer 22	32	20 × 20	512

Với tensor đầu vào B×3×640×640, các mức P3, P4 và P5 lần lượt có kích thước không gian 80×80, 40×40 và 20×20. P3 giữ nhiều vị trí không gian hơn nhưng mức ngữ nghĩa thấp hơn; P5 có ít vị trí hơn và được xây dựng từ nhiều tầng trích đặc trưng. Neck trao đổi thông tin giữa các mức bằng upsample, concatenate và các khối xử lý tiếp theo. Khối thay thế ở cuối backbone có thể ảnh hưởng những đặc trưng được truyền xuống các mức khác qua neck, dù nó không trực tiếp nằm trên bản đồ phân giải cao nhất.

Trong cấu hình đầu Segment26, 32 hệ số mặt nạ của mỗi đối tượng được kết hợp với 32 prototype dùng chung. Có thể viết logit mặt nạ của đối tượng i tại vị trí (u,v) dưới dạng tổng các tích $a_{ij}P_j(u,v)$, sau đó chuyển sang đáp ứng mặt nạ theo bước hậu xử lý tương ứng. Công thức mô tả cơ chế tổ hợp tuyến tính; các thao tác crop, nội suy và ngưỡng hóa cần được xem là những bước riêng khi tạo mặt nạ trên ảnh gốc. Nhờ đó, có thể phân biệt tác động lên đặc trưng với tác động của quy trình dựng mặt nạ đầu ra.

$$M_i(u,v) = \text{sigmoid}(\Sigma_{j=1}^{32} a_{ij}P_j(u,v))$$

Phương án thay C2PSA được chọn nhằm giữ giao diện giữa backbone và neck tương thích. Nếu chèn thêm một tầng mà không điều chỉnh các chỉ số from trong YAML, những kết nối ghép đặc trưng và đầu Segment26 có thể trỏ sai tầng. Ở thiết kế hiện tại, vị trí tầng 10 được thay bằng C2TSVMamba nên các tham chiếu từ neck được giữ theo cùng cấu trúc chỉ số. Khả năng tương thích này cần được kiểm tra sau khi parser áp dụng scale, thay vì chỉ so sánh tên các dòng trong YAML.

2.2 WRAPPER C2TSVMAMBA

C2TSVMamba trước hết chiếu đầu vào bằng convolution 1×1 rồi tách thành hai nhánh a và b có cùng số kênh. Nhánh a đi tắt, nhánh b đi qua TSVMamba. Hai nhánh được ghép và chiếu lại bằng convolution 1×1. Với đầu vào 512 kênh, hệ số e bằng 0,5 tạo mỗi nhánh 256 kênh; đây là số kênh được SS2D bên trong xử lý.

$$[a, b] = \text{split}(\text{Conv}_{1\times1}(x)) \; ; \; y = \text{Conv}_{1\times1}(\text{concat}(a, \text{TSVMamba}(b)))$$

YAML ghi repeats bằng 2, nhưng parser áp dụng depth multiplier bằng 0,5 cho biến thể s nên số block nội bộ là 1. Điều kiện c1 = c2 được kiểm tra trong constructor. Vì module có đường đi tắt và phép chiếu đầu ra, residual trong TSVMamba chỉ là một phần của toàn wrapper, không phải đường đi duy nhất của đặc trưng.

Hai đường truyền trong wrapper tạo ra một phần đặc trưng được chuyển qua lõi tùy biến và một phần được giữ qua nhánh ngắn. Phép ghép kênh ở cuối không phải lấy trung bình hai nhánh; convolution 1×1 tiếp theo học cách kết hợp chúng. Vì vậy, ngay cả khi hệ số residual trong TSVMamba được khởi tạo nhỏ, toàn wrapper vẫn có các phép chiếu được huấn luyện. Cách mô tả này tránh coi module mới như một phép cộng gần bằng 0 hoàn toàn không thay đổi backbone lúc khởi tạo.

Số kênh của lõi 256 cũng là cơ sở để xác định đúng chi phí. Nếu lấy 512 kênh đầu vào wrapper làm số kênh của SS2D sẽ đánh giá sai kích thước các tensor trạng thái và các phép chiếu. Tương tự, repeats bằng 2 trên YAML là giá trị trước co giãn chiều sâu; số block thực sự của biến thể s là 1. Báo cáo sử dụng kích thước sau parser để giải thích mô hình đã chạy, còn giá trị trước parser được giữ để người đọc hiểu cách dùng cùng một YAML cho nhiều quy mô.

2.3 CÁC NHÁNH CỦA TSVMAMBA

2.3.1 Nhánh SS2D toàn cục

Đầu vào được chiếu thành hai tensor u và z. Tensor u qua convolution depthwise 3×3 và SiLU, sau đó được trải thành bốn chuỗi có cùng chiều dài. Ở mỗi hướng, phép chiếu tuyến tính sinh Δ, B và C; A được tham số hóa bằng âm của hàm mũ để tạo đáp ứng suy giảm. Kết quả quét được đưa lại về lưới không gian rồi cộng giữa bốn hướng. Tensor z tạo điều biến SiLU trước phép chiếu đầu ra.

State dimension mặc định bằng 16; expand bằng 1,0 và dt_rank bằng ceil(c/16). Với c bằng 256, dt_rank bằng 16. Mã hiện tại không có LayerNorm đầu ra SS2D hoặc nhánh skip D độc lập giống một số cấu hình VMamba tham chiếu. Vì vậy, thuật ngữ SS2D trong báo cáo chỉ cách quét chọn lọc đang được cài đặt, không ngụ ý đồng nhất từng chi tiết với mọi phiên bản VMamba gốc.

Tại mỗi hướng, tín hiệu đầu vào được chiếu để sinh các hệ số phụ thuộc vị trí; trạng thái sau đó kết hợp thông tin hiện tại với trạng thái trước. Những hệ số này được chia sẻ hoặc bố trí theo các chiều tensor của triển khai, không phải một ma trận attention H×W giữa mọi cặp vị trí. Điều này tạo cơ chế truyền thông tin khác với self-attention. Mặc dù độ dài chuỗi ở P5 chỉ bằng 400, lõi vẫn phải xử lý chiều kênh và chiều trạng thái 16; chỉ nêu độ dài chuỗi sẽ chưa phản ánh đủ chi phí thực tế.

Nhánh z được tách từ phép chiếu đầu vào và điều biến đáp ứng sau quét. Nhánh này khác với guidance hình dạng: z thuộc cấu trúc nội bộ SS2D, còn guidance được tạo từ tensor S của nhánh ShapeAwareBranch. Hai tín hiệu tác động ở các vị trí khác nhau trong luồng tính toán. Khi thực hiện ablation no_guidance, cần hiểu phần bị bỏ là guidance giữa hai nhánh; không đồng nghĩa mọi điều biến bên trong SS2D đều bị loại bỏ.

2.3.2 Nhánh hình dạng đa tỉ lệ và có hướng

ShapeAwareBranch áp dụng hai convolution depthwise 3×3 và 5×5 song song rồi ghép bằng convolution 1×1. DirectionalShapeExtractor tiếp tục xử lý theo kernel ngang 1×5, kernel dọc 5×1 và kernel cục bộ 3×3. Các kernel này được học cùng mô hình và tạo một thiên hướng biểu diễn theo kích thước, hướng, thay vì một phép trích biên cố định.

$$S_grad = \sqrt{(S_h^2 + S_v^2 + 10^{-6})}$$

Đáp ứng S_grad được ghép với S_h, S_v và S_local để tạo biểu diễn cấu trúc, sau đó kết hợp lại với đặc trưng shape ban đầu. Vì S_h và S_v là đầu ra convolution có BatchNorm và activation, biểu thức trên không phải gradient đạo hàm không gian chuẩn như Sobel. Tương tự, convolution 3×3 không tự tạo ra phép đo độ cong hình học tường minh.

Depthwise convolution giới hạn việc lọc không gian trong từng kênh, sau đó convolution 1×1 trộn các đáp ứng giữa kênh. Kernel 3×3 và 5×5 tiếp cận những lân cận có độ rộng khác nhau; các kernel 1×5 và 5×1 tạo thiên hướng theo hai trục chính. Việc phối hợp này là một giả thuyết thiết kế về biểu diễn cấu trúc, không phải gán nhãn thủ công điểm nào thuộc biên. Tất cả các trọng số của nhánh vẫn được học qua mục tiêu phân đoạn và phát hiện chung.

Đại lượng căn bậc hai từ hai đáp ứng ngang, dọc cung cấp một tín hiệu kết hợp độ lớn. Hằng số 10^{-6} giúp phép căn có đối số dương khi hai đáp ứng gần 0. Tín hiệu này được đưa vào cùng các đáp ứng có dấu và nhánh cục bộ, vì chỉ giữ

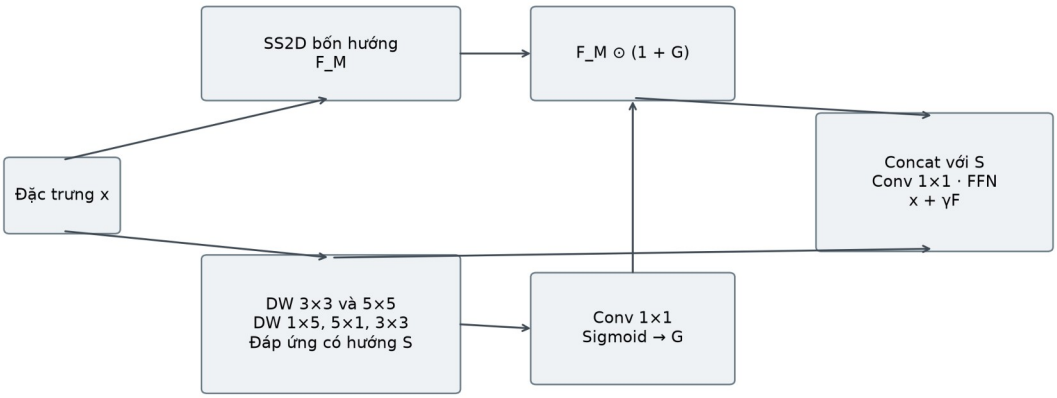
độ lớn có thể làm mất thông tin về hướng và dấu của các bộ lọc. Dù vậy, việc mạng học được cấu trúc hữu ích nào vẫn phải được khảo sát bằng đầu ra và ablation; không thể suy ra một đường bao hoàn chỉnh chỉ từ biểu thức của S_grad.

2.3.3 Guidance và hợp nhất

$$G = \mathrm{sigmoid}(\mathrm{Conv}_{1\times1}(S)) \; ; \; F'_M = F_M \odot (1 + G)$$

$$y = x + \gamma \cdot \mathrm{FFN}(\mathrm{Conv}_{1\times1}(\mathrm{concat}(F'_M, S)))$$

Cổng sigmoid có giá trị trong đoạn từ 0 đến 1 nên hệ số trực tiếp nhân với F_M nằm từ 1 đến 2. Nó điều chỉnh mức khuếch đại theo vị trí và kênh; không trực tiếp làm nhỏ biên độ F_M dưới giá trị ban đầu. Khả năng hạn chế dự đoán nền, nếu có, là kết quả của toàn bộ mạng sau hợp nhất và phải được đo ở đầu ra.



Hình 2.2 Hai nhánh SS2D và hình dạng cùng cơ chế guidance trong TSVMamba

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

FFN dùng hai convolution 1×1 với số kênh mở rộng gấp đôi rồi thu lại. Hệ số residual γ là một tham số học được, khởi tạo 0,001, nhằm giới hạn tác động ban đầu của nhánh mới. Giá trị khởi tạo nhỏ không chứng minh quá trình huấn luyện luôn ổn định; thống kê kích hoạt và checkpoint vẫn cần được kiểm tra.

Guidance một chiều có nghĩa tensor hình dạng được dùng để tạo trọng số điều chỉnh nhánh trạng thái; mã không xây dựng một bước điều chỉnh ngược độc lập từ SS2D về ShapeAwareBranch trước hợp nhất. Tuy nhiên, trong lan truyền ngược, gradient của mục tiêu vẫn đi qua cả hai đường truyền và phép hợp nhất.

Do đó, “một chiều” mô tả hướng tính toán của tín hiệu guidance trong forward, không phải nhánh shape được tách khỏi quá trình học của toàn mạng.

Hệ số γ được học và khởi tạo bằng 0,001 để bắt đầu với đóng góp residual có biên độ nhỏ. Mục đích thiết kế là hạn chế thay đổi đột ngột của đường residual khi đưa module mới vào mạng đã sử dụng trọng số pretrained. Đây là cơ chế khởi tạo, chưa phải bằng chứng bảo đảm hội tụ hoặc loại bỏ hoàn toàn gradient bất ổn. Việc kết hợp hai nhánh qua convolution và FFN vẫn có thể thay đổi phân bố đặc trưng, cần được xem cùng diễn biến loss và các kiểm tra tensor hữu hạn.

2.4 ĐỘ PHỨC TẠP VÀ ĐỘ CHÍNH XÁC SỐ

2.4.1 Chi phí của phép quét đang triển khai

Hàm parallel_associative_scan ghép cặp phép chuyển trạng thái theo các bước có độ lệch 1, 2, 4 và tiếp tục nhân đôi. Chuỗi được đệm đến lũy thừa hai gần nhất. Với L bằng 400, chuỗi đệm có 512 vị trí và có 9 bước ghép. Mỗi bước vẫn tạo và xử lý các tensor trải trên phần lớn chiều dài chuỗi.

Khi cố định batch, kênh và state dimension, số bước phụ thuộc là O(log L), còn tổng công việc tensor của triển khai này là O(L log L). Không thể suy ra thời gian suy luận tuyến tính hoặc tốc độ cao chỉ từ tính chất của Mamba trong tài liệu gốc. Các tensor trạng thái có chiều B×D×N×L cũng ảnh hưởng đáng kể đến bộ nhớ và chi phí đọc ghi.

Đặt module ở P3 với đầu vào 640×640 sẽ làm độ dài không gian tăng từ 400 lên 6.400 vị trí, tức gấp 16 lần P5, trước khi xét các thay đổi về kênh. Tại P4, độ dài là 1.600 vị trí, gấp 4 lần P5. Những tỷ lệ này giải thích động cơ chọn tầng sâu để giới hạn quy mô phép quét; chúng chưa xác định tỷ lệ tăng latency của cả mạng vì mỗi tầng còn khác về kênh, kernel và khả năng song song. Muốn khẳng định P5 là vị trí tốt nhất cần thí nghiệm vị trí có cùng quy trình đánh giá.

Các đại lượng Parameters, kích thước tệp checkpoint, FLOPs và latency đo những mặt khác nhau. Số tham số phản ánh lượng trọng số học được; kích thước checkpoint còn phụ thuộc kiểu số và thành phần được lưu; số phép toán là đại lượng ước lượng theo mô hình đếm; latency phụ thuộc cả triển khai và phần cứng. Đối với phép quét tùy biến, một công cụ đếm chỉ nhận diện convolution có thể bỏ qua phần trạng thái và cho ra tổng chưa đầy đủ. Kết quả benchmark cần nêu rõ những toán tử được đếm và những phần còn thiếu, đồng thời ưu tiên số đo thời gian thực tế cho cấu hình đang sử dụng.

2.4.2 Vì sao cần checkpoint FP32

Trong fork cơ bản, bước lưu checkpoint chuyển bản sao EMA sang half. Đối với TSVM, nhánh trạng thái có thể tạo các giá trị lớn trước BatchNorm; các running statistics lớn có nguy cơ vượt miền số hữu hạn FP16. Việc tắt AMP lúc train không thay đổi phép chuyển kiểu trong hàm lưu checkpoint. Vì vậy, amp=False và lưu FP32 là hai yêu cầu khác nhau.

Bản FP32 đổi snapshot EMA và checkpoint sau strip_optimizer sang float32, đồng thời dừng với FloatingPointError khi phát hiện tensor NaN hoặc Inf. Thay đổi này bảo toàn biểu diễn số của checkpoint; nó không thêm một nhánh kiến trúc mới. Ứng dụng cũng ép mô hình về float32 và không yêu cầu half khi suy luận. Kiểm tra nạp thành công chưa thay thế cho việc đánh giá lại toàn bộ tập kiểm định từ checkpoint.

FP16 và FP32 khác nhau không chỉ về số bit lưu trữ mà cả miền giá trị hữu hạn và độ chính xác biểu diễn. Khi một tensor hữu hạn ở float32 được ép sang float16, giá trị có thể vượt miền biểu diễn hoặc bị làm tròn mạnh. Việc chuyển lại sang float32 sau đó không phục hồi được thông tin đã mất. Do đó, nạp một checkpoint bị tràn ở thời điểm lưu rồi gọi .float() không tương đương lưu và giữ tensor hữu hạn bằng FP32 từ đầu.

Quy trình FP32 của đề tài cần được xem theo vòng đời: tính toán trong huấn luyện, sao chép EMA để lưu, loại bỏ thành phần không cần thiết của checkpoint, nạp ở dịch vụ và thực hiện predict. Chỉ đặt amp=False kiểm soát một phần của vòng đời này. Khi kiểm tra checkpoint, cần xác nhận tensor dấu phẩy động có kiểu phù hợp và không chứa NaN hoặc Inf. Việc thay giá trị không hữu hạn bằng số hữu hạn để cho mô hình tiếp tục chạy sẽ che đi lỗi số học; bản fork chọn phát hiện và từ chối trường hợp đó để giữ khả năng kiểm tra nguyên nhân.

2.5 HUẤN LUYỆN VÀ KHỞI TẠO TRỌNG SỐ

Baseline được tạo từ yolo26s-seg.pt. Mô hình đề xuất được tạo từ YAML scale s rồi nạp các trọng số tương thích từ cùng tệp pretrained. Các tham số của module thay thế không có đối ứng trong C2PSA nên được khởi tạo riêng. Đây là học chuyển giao từ checkpoint YOLO phân đoạn; không mô tả trực tiếp như nạp trọng số ImageNet cho toàn bộ TSVM.

Hàm mất mát thuộc pipeline Segment26 của fork Ultralytics, bao gồm thành phần phát hiện và phân đoạn theo cấu hình. Đề tài không bổ sung topology loss, persistent homology loss hoặc một boundary supervision riêng trong khối

TSVM. Các hệ số có tên trong args.yaml phải được diễn giải cùng nhánh thực thi; riêng reg_max bằng 1 làm mất mát DFL dạng nhiều bin không hoạt động như ở cấu hình reg_max lớn hơn 1.

Cùng sử dụng nguồn pretrained giúp hai mô hình bắt đầu từ một chính sách khởi tạo có thể đối chiếu, nhưng không làm mọi trọng số giống nhau. Các lớp còn tương thích có thể nhận trọng số từ mô hình gốc; lớp C2TSVMamba thay thế C2PSA có cấu trúc tham số riêng. Vì vậy, mức khác biệt cuối cùng bao gồm cả cách biểu diễn và cách kiến trúc mới được tối ưu từ trạng thái khởi tạo đó. Một thí nghiệm bổ sung huấn luyện từ đầu hoặc khởi tạo theo chính sách khác có thể trả lời câu hỏi khác, nhưng không được trộn vào nhóm kết quả chính nếu điều kiện đã thay đổi.

Đầu phân đoạn và phát hiện được tối ưu chung, trong khi các thành phần loss đã lưu có vai trò giám sát những phần đầu ra khác nhau. Loss thấp hơn không luôn dẫn đến AP cao hơn vì AP còn phụ thuộc thứ hạng điểm tin cậy và quy tắc ghép ở nhiều ngưỡng IoU. Với reg_max bằng 1, nhánh loss hộp bao dùng chế độ tương ứng trong mã, nên không sao chép nguyên công thức DFL nhiều bin từ tài liệu của cấu hình khác. Khóa luận giữ cách diễn giải theo nhánh thực thi và dùng tên cột loss nguyên gốc khi trình bày các đường học.

2.6 CÁC CHẾ ĐỘ ABLATION VÀ GIẢ THUYẾT KIỂM CHỨNG

Mã nguồn có bốn mode: topology_shape_vmamba, vmamba_only, shape_only và shape_vmamba_no_guidance. Chúng cho phép phân biệt vai trò ngữ cảnh, hình dạng và guidance. Thực nghiệm chính được trình bày trong báo cáo hiện gồm baseline và full TSVM FP32; chưa có bộ 20 seed tương ứng cho từng mode để định lượng đóng góp riêng.

Bảng 2.2 Các chế độ hỗ trợ trong mã nguồn

Chế độ	SS2D	Shape	Guidance
topology_shape_vmamba	Có	Có	Có
vmamba_only	Có	Không	Không
shape_only	Không	Có	Không
shape_vmamba_no_guidance	Có	Có	Không

Thử nghiệm ablation trong tương lai cần dùng cùng seed, dữ liệu, pretrained policy, epoch và tiêu chí chọn checkpoint. Nếu thêm một module làm tăng tham số, có thể bổ sung đối chứng convolution với mức tham số tương đương. Điều này giúp tránh quy kết mọi mức tăng cho guidance khi nguyên nhân có thể gồm dung lượng mạng hoặc thay đổi tối ưu hóa.

2.7 TÓM TẮT CHƯƠNG

Mô hình đề xuất thay một block sâu bằng C2TSVMamba, kết hợp trạng thái toàn cục với tín hiệu hình dạng học được. Bản FP32 xử lý vấn đề lưu và suy luận chính xác số, không thay đổi sơ đồ kiến trúc. Những mô tả này xác định các giả thuyết cần đánh giá và giới hạn những khẳng định có thể rút ra từ thực nghiệm.

CHƯƠNG 3 XÂY DỰNG BỘ DỮ LIỆU VÀ TIỀN XỬ LÝ DỮ LIỆU

Chất lượng của dữ liệu đầu vào ảnh hưởng trực tiếp đến khả năng học của mô hình phân đoạn. Chương này trình bày cách đề tài xây dựng bộ dữ liệu BG20 từ hai nguồn ảnh nội soi công khai và các bước xử lý giúp đưa ảnh cùng nhãn về dạng mà mô hình YOLO26-seg tích hợp khối TSVM có thể sử dụng được.

Quá trình tiền xử lý được chia thành hai giai đoạn. Giai đoạn thứ nhất chỉ thực hiện một lần trước khi huấn luyện, gồm chọn ảnh, chia tập dữ liệu và chuyển mặt nạ phân đoạn thành nhãn đa giác. Giai đoạn thứ hai diễn ra mỗi khi ảnh được nạp vào mô hình, gồm đổi kích thước, tăng cường dữ liệu và chuẩn hóa giá trị đầu vào. Các thao tác và thông số trong chương đều được đối chiếu với mã nguồn, tệp cấu hình huấn luyện và dữ liệu thực tế của đề tài.

3.1. NGUỒN DỮ LIỆU

3.1.1. Bộ dữ liệu Kvasir-SEG

Kvasir-SEG là bộ dữ liệu công khai gồm 1.000 ảnh nội soi đường tiêu hóa có polyp, được Jha và cộng sự công bố năm 2020 [1]. Các ảnh này được lấy từ lớp polyp của bộ dữ liệu Kvasir [2]. Với mỗi ảnh, một nhóm gồm kỹ sư và bác sĩ đã khoanh vùng polyp bằng công cụ gán nhãn Labelbox, sau đó một bác sĩ chuyên khoa tiêu hóa giàu kinh nghiệm kiểm tra lại kết quả [1]. Vùng được khoanh được lưu thành một ảnh mặt nạ cùng tên với ảnh gốc, trong đó các pixel thuộc polyp có màu trắng và các pixel còn lại có màu đen.

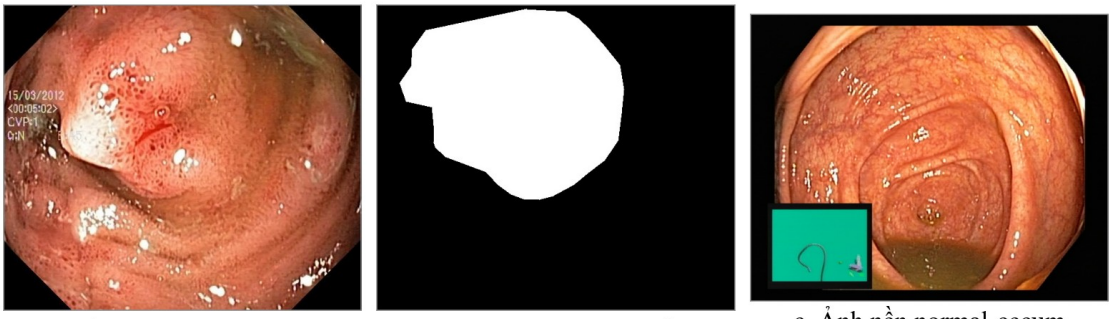
Khi kiểm tra trực tiếp 1.000 ảnh, đề tài nhận thấy kích thước ảnh không đồng nhất. Bộ dữ liệu có 333 kích thước khác nhau, chiều rộng thay đổi từ 332 đến 1.920 pixel và chiều cao từ 352 đến 1.072 pixel, phổ biến nhất là các ảnh có kích thước xấp xỉ 622 × 530 pixel. Kích thước polyp cũng chênh lệch rất lớn: vùng polyp nhỏ nhất chỉ chiếm khoảng 0,15% diện tích ảnh, vùng lớn nhất chiếm khoảng 81% và một nửa số vùng polyp chiếm không quá 10% diện tích ảnh. Một số ảnh còn có chữ hiển thị thông tin ca nội soi hoặc ô hình nhỏ của hệ thống ScopeGuide thể hiện vị trí ống soi [1]. Những đặc điểm này cho thấy dữ liệu cần được đưa về một kích thước chung và mô hình cần học được polyp ở nhiều kích cỡ khác nhau.

Ngoài ra, cả ảnh và mặt nạ đều được lưu ở định dạng nén JPEG [1]. Phép nén này làm xuất hiện một số pixel có mức xám trung gian dọc theo ranh giới giữa vùng trắng và vùng đen, nên mặt nạ không còn hoàn toàn chỉ gồm hai giá trị. Đây là lý do mặt nạ cần được nhị phân hóa lại trước khi chuyển thành nhãn, như trình bày ở mục 3.3.

3.1.2. Ảnh nền normal-cecum

Bộ dữ liệu Kvasir gồm 8 lớp ảnh, mỗi lớp 1.000 ảnh, bao gồm các mốc giải phẫu, các phát hiện bệnh lý và các thủ thuật nội soi [1], [2]. Đề tài sử dụng lớp normal-cecum, tức ảnh chụp manh tràng bình thường, làm nguồn ảnh nền. Manh tràng là đoạn đầu của đại tràng và là mốc giải phẫu mà bác sĩ thường phải quan sát trong quá trình nội soi. Các ảnh thuộc lớp này không chứa polyp và đều có kích thước 720 × 576 pixel.

Ảnh nền đóng vai trò là mẫu âm tính. Nếu mô hình chỉ được huấn luyện trên ảnh có polyp, nó có xu hướng cho rằng ảnh nào cũng chứa polyp và dễ khoanh nhầm các nếp gấp hay vùng phản quang trên niêm mạc bình thường, gây ra báo động giả. Việc bổ sung ảnh không có polyp giúp mô hình học được trường hợp không có đối tượng nào cần phân đoạn. Hình 3.1 minh họa một ảnh polyp, mặt nạ tương ứng và một ảnh nền được sử dụng trong đề tài.



a. Ảnh nội soi có polyp b. Mặt nạ phân đoạn gốc c. Ảnh nền normal-cecum

Hình 3.1. Ví dụ ảnh có polyp, mặt nạ phân đoạn gốc và ảnh nền trong bộ dữ liệu BG20

Nguồn: ảnh a và b từ bộ dữ liệu Kvasir-SEG [1], ảnh c từ lớp normal-cecum của bộ dữ liệu Kvasir [2]; đề tài ghép và đặt chú thích.

Trong Hình 3.1, ảnh a và mặt nạ b có cùng kích thước, vùng trắng của mặt nạ trùng khớp với vị trí polyp trên ảnh. Ảnh c là ảnh manh tràng bình thường, góc dưới bên trái có ô hình ScopeGuide. Ảnh nền không có mặt nạ đi kèm vì không có vùng nào cần phân đoạn.

3.2. XÂY DỰNG VÀ PHÂN CHIA BỘ DỮ LIỆU BG20

Ảnh polyp được chia theo danh sách có sẵn gồm 880 ảnh huấn luyện và 120 ảnh kiểm định do nhóm tác giả Kvasir-SEG cung cấp trong hai tệp train.txt và val.txt [3]. Việc dùng danh sách có sẵn thay vì tự chia ngẫu nhiên giúp kết quả của đề tài có thể so sánh với các nghiên cứu khác dùng cùng cách chia, đồng thời bảo đảm không có ảnh nào xuất hiện ở cả hai tập.

Đối với ảnh nền, chương trình sắp xếp 1.000 tên tệp normal-cecum theo thứ tự, cố định hạt giống ngẫu nhiên seed bằng 42 rồi chọn ngẫu nhiên 200 ảnh. Trong đó, 160 ảnh đầu tiên được đưa vào tập huấn luyện và 40 ảnh còn lại được đưa vào tập kiểm định. Nhờ sắp xếp danh sách trước khi chọn và cố định seed, mỗi lần chạy lại chương trình đều cho ra đúng 200 ảnh như nhau. Danh sách ảnh được chọn cũng được lưu lại thành hai tệp văn bản để đối chiếu về sau.

Tên gọi BG20 thể hiện số ảnh nền bằng 20% của 1.000 ảnh polyp gốc. Cần lưu ý rằng khi tính trên toàn bộ 1.200 ảnh, ảnh nền chỉ chiếm 16,67%; tỷ lệ này là 15,38% trong tập huấn luyện và 25% trong tập kiểm định. Phân bố cuối cùng của bộ dữ liệu được tổng hợp trong Bảng 3.1.

Bảng 3.1. Phân bố ảnh và nhãn trong bộ dữ liệu BG20

Thành phần	Huấn luyện	Kiểm định	Tổng
Ảnh có polyp	880	120	1.000
Ảnh nền normal-cecum	160	40	200
Tổng số ảnh	1.040	160	1.200
Số vùng polyp được gán nhãn	936	127	1.063

Nguồn: đề tài thống kê từ bộ dữ liệu BG20.

Số vùng polyp lớn hơn số ảnh polyp vì một ảnh có thể chứa nhiều polyp tách rời nhau, mỗi polyp được gán một nhãn riêng. Chẳng hạn, 120 ảnh polyp của tập kiểm định chứa tổng cộng 127 vùng polyp.

Bộ dữ liệu được tổ chức theo định dạng của thư viện Ultralytics. Thư mục images chứa ảnh và thư mục labels chứa nhãn, mỗi thư mục chia tiếp thành train và val. Mỗi ảnh có một tệp nhãn dạng văn bản cùng tên. Tệp cấu hình YAML khai báo đường dẫn dữ liệu và một lớp duy nhất là polyp với chỉ số 0. Ảnh nền được thêm tiền tố bg_ vào tên tệp để tránh trùng tên và đi kèm một tệp nhãn rỗng có dung lượng 0 byte. Tệp nhãn rỗng cho mô hình biết ảnh không chứa polyp nào, vì vậy ảnh nền không phải là một lớp đối tượng thứ hai.

3.3. CHUYỂN MẶT NẠ PHÂN ĐOẠN THÀNH NHÃN ĐA GIÁC

Mặt nạ của Kvasir-SEG là một ảnh, mỗi pixel cho biết vị trí đó có thuộc polyp hay không. Trong khi đó, YOLO26-seg yêu cầu nhãn ở dạng văn bản: mỗi dòng mô tả một đối tượng, gồm chỉ số lớp và dãy tọa độ các đỉnh của đa giác bao quanh đối tượng, các tọa độ được chuẩn hóa về đoạn từ 0 đến 1 [4]. Vì vậy, đề tài chuyển mỗi mặt nạ thành nhãn đa giác qua năm bước dưới đây, sử dụng các hàm xử lý ảnh của thư viện OpenCV [5]. Công đoạn này chỉ tác động lên mặt nạ, còn ảnh nội soi được sao chép nguyên vẹn sang bộ dữ liệu mới.

3.3.1. Nhị phân hóa mặt nạ bằng phương pháp Otsu

Đầu tiên, mặt nạ được đọc và chuyển thành ảnh xám, mỗi pixel có giá trị từ 0 đến 255. Do ảnh hưởng của nén JPEG đã nêu ở mục 3.1.1, mặt nạ không chỉ có hai giá trị 0 và 255; ví dụ, mặt nạ dùng trong Hình 3.2 có 15 mức xám khác nhau. Nhị phân hóa là việc chọn một ngưỡng t: pixel có giá trị lớn hơn t được gán thành 255 và thuộc polyp, các pixel còn lại được gán thành 0.

Phương pháp Otsu [6] chọn ngưỡng này một cách tự động. Thuật toán thử lần lượt mọi ngưỡng có thể, mỗi ngưỡng chia các pixel thành nhóm tối và nhóm sáng, rồi giữ lại ngưỡng làm hai nhóm tách biệt nhau rõ nhất. Mức độ tách biệt được đo bằng phương sai giữa hai nhóm:

$$\sigma_B^2(t) = \omega_0(t) \cdot \omega_1(t) \cdot [\mu_0(t) - \mu_1(t)]^2 \tag{3.1}$$

$$t^* = \arg \max_{0 \le t \le 255} \sigma_B^2(t) \tag{3.2}$$

Trong đó, ω_0 và ω_1 là tỷ lệ số pixel thuộc nhóm tối và nhóm sáng; μ_0 và μ_1 là mức xám trung bình của hai nhóm. Ngưỡng tối ưu t^* là ngưỡng làm cho phương sai giữa hai nhóm đạt giá trị lớn nhất. Trên 1.000 mặt nạ của đề tài, ngưỡng Otsu chỉ nằm trong khoảng từ 5 đến 8. Điều này cho thấy mặt nạ gốc gần như đã nhị phân, và bước Otsu chủ yếu loại bỏ các pixel nhiễu do nén ảnh tạo ra.

3.3.2. Làm liền vùng polyp bằng phép đóng hình thái học

Sau khi nhị phân hóa, trên mặt nạ có thể còn những lỗ đen rất nhỏ bên trong vùng polyp hoặc những khe hở mảnh trên đường biên. Đề tài sử dụng phép đóng, còn gọi là Closing, thuộc nhóm phép toán hình thái học [7] để làm liền các chỗ này. Phép đóng gồm hai bước liên tiếp:

$$A \bullet B = (A \oplus B) \ominus B \tag{3.3}$$

Trong đó, A là tập các pixel trắng của mặt nạ và B là phần tử cấu trúc, ở đây là một hình vuông kích thước 3 × 3 pixel. Phép giãn nở, ký hiệu $\oplus$, làm vùng trắng phình ra thêm một pixel về mọi phía nên các lỗ và khe nhỏ bị lấp kín. Phép co, ký hiệu $\ominus$, sau đó thu vùng trắng lại một pixel để đường biên trở về gần vị trí ban đầu. Kết quả là vùng polyp liền mạch hơn nhưng kích thước gần như không thay đổi. Phép đóng chỉ được áp dụng một lần với phần tử cấu trúc nhỏ để tránh làm sai lệch hình dạng polyp.

3.3.3. Tìm đường bao và loại bỏ vùng nhiễu nhỏ

Đường bao là chuỗi các điểm nằm trên ranh giới của một vùng trắng. OpenCV tìm đường bao dựa trên thuật toán dò biên của Suzuki và Abe [8]. Đề tài chỉ lấy đường bao ngoài cùng của mỗi vùng, vì nhãn YOLO mô tả mỗi đối tượng bằng đúng một đa giác khép kín. Khi lưu đường bao, các đoạn thẳng nằm ngang, dọc hoặc chéo chỉ được giữ lại điểm đầu và điểm cuối nhằm giảm bớt số điểm không cần thiết.

Tiếp theo, mọi đường bao có diện tích nhỏ hơn 20 pixel vuông đều bị loại bỏ. Những vùng nhỏ như vậy thường là các đốm nhiễu, nếu giữ lại sẽ khiến mô hình học những đối tượng không có thật. Trên toàn bộ 1.000 mặt nạ, chương trình tìm được 1.206 đường bao, trong đó 143 đường bao bị loại do quá nhỏ, còn lại đúng 1.063 vùng polyp như trong Bảng 3.1.

3.3.4. Đơn giản hóa đường bao

Đường bao thu được bám theo từng pixel trên biên nên có rất nhiều điểm, trung bình khoảng 333 điểm cho mỗi polyp. Để nhãn gọn hơn, đề tài dùng thuật toán Douglas–Peucker [9]. Thuật toán bắt đầu bằng đoạn thẳng nối hai đầu của đường cong, sau đó tìm điểm nằm xa đoạn thẳng này nhất. Nếu khoảng cách lớn hơn ngưỡng sai số ε, điểm đó được giữ lại và quá trình lặp lại cho hai nửa đường cong; ngược lại, mọi điểm ở giữa đều bị bỏ đi. Ngưỡng sai số được tính theo chu vi của đường bao:

$$\varepsilon = 0{,}002 \cdot L \tag{3.4}$$

Trong đó, L là chu vi đường bao tính bằng pixel. Ví dụ, một polyp có chu vi 1.000 pixel sẽ có ε bằng 2 pixel, nghĩa là đa giác sau khi rút gọn lệch khỏi đường bao ban đầu không quá 2 pixel. Cách tính theo chu vi giúp polyp lớn và polyp nhỏ được rút gọn với mức độ tương xứng. Nếu đa giác thu được có ít hơn 3 đỉnh, chương trình giữ lại đường bao ban đầu. Kết quả, số điểm trung bình của mỗi đa

giác giảm từ 333,2 xuống còn 24,6 điểm, tức giảm 92,6%, và mỗi đa giác có từ 10 đến 80 đỉnh.

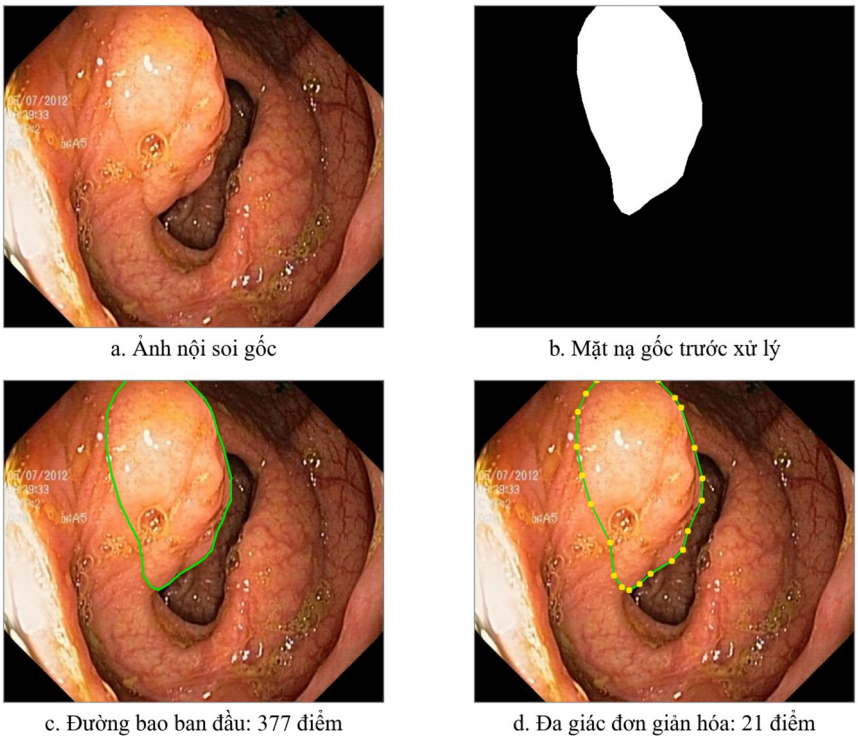
3.3.5. Chuẩn hóa tọa độ và ghi tệp nhãn

Cuối cùng, tọa độ mỗi đỉnh được chia cho chiều rộng W và chiều cao H của mặt nạ để đưa về đoạn từ 0 đến 1:

$$\hat{x} = \frac{x}{W}, \ \hat{y} = \frac{y}{H} \tag{3.5}$$

Giá trị sau chuẩn hóa được giới hạn trong đoạn từ 0 đến 1 và làm tròn đến 6 chữ số thập phân. Mỗi polyp được ghi thành một dòng trong tệp nhãn theo cấu trúc: chỉ số lớp, sau đó là lần lượt các cặp tọa độ x̂ và ŷ của từng đỉnh. Ví dụ, dòng nhãn của ảnh trong Hình 3.2 bắt đầu bằng 0 0.321027 0.000000 0.292135 0.041667 và tiếp tục cho đến hết 21 đỉnh. Nhờ chuẩn hóa, nhãn không phụ thuộc vào kích thước ảnh, nên khi ảnh được phóng to hay thu nhỏ ở các bước sau, đa giác vẫn khớp đúng vị trí polyp.

Hình 3.2 minh họa toàn bộ công đoạn chuyển đổi trên một ảnh thuộc tập huấn luyện, từ mặt nạ gốc đến đa giác cuối cùng.



a. Ảnh nội soi gốc

b. Mặt nạ gốc trước xử lý

c. Đường bao ban đầu: 377 điểm

d. Đa giác đơn giản hóa: 21 điểm

Hình 3.2. Trước và sau khi chuyển mặt nạ phân đoạn thành nhãn đa giác

Nguồn: ảnh a và b từ bộ dữ liệu Kvasir-SEG [1]; ảnh c và d do đề tài tạo bằng chương trình chuyển đổi mặt nạ của đề tài.

Ảnh b là mặt nạ gốc trước khi xử lý. Sau các bước Otsu, phép đóng và tìm đường bao, ảnh c cho thấy đường bao màu xanh gồm 377 điểm bám sát biên polyp. Ở mặt nạ này còn có một đốm nhiễu nhỏ hơn 20 pixel vuông và đã bị loại bỏ. Sau khi đơn giản hóa, đa giác trong ảnh d chỉ còn 21 đỉnh, được đánh dấu bằng các chấm màu vàng, nhưng vẫn giữ được hình dạng của polyp. Như vậy, dung lượng nhãn giảm đáng kể trong khi thông tin về vị trí và hình dạng polyp gần như được bảo toàn.

3.4. ĐỔI KÍCH THƯỚC ẢNH

Từ mục này trở đi, các bước xử lý diễn ra mỗi khi ảnh được nạp vào mô hình. Mô hình được cấu hình với kích thước đầu vào 640 pixel, trong khi ảnh gốc có nhiều kích thước khác nhau. Nếu kéo giãn trực tiếp mọi ảnh thành hình vuông 640 × 640, polyp sẽ bị méo và mô hình học sai hình dạng thật của tổn thương. Vì vậy, ảnh được thu phóng nhưng vẫn giữ nguyên tỷ lệ giữa chiều rộng và chiều cao, sao cho cạnh dài nhất bằng 640 pixel:

$$r = \frac{640}{max(W, H)} \tag{3.6}$$

$$W' = \lfloor r \cdot W \rfloor, \; H' = \lfloor r \cdot H \rfloor \tag{3.7}$$

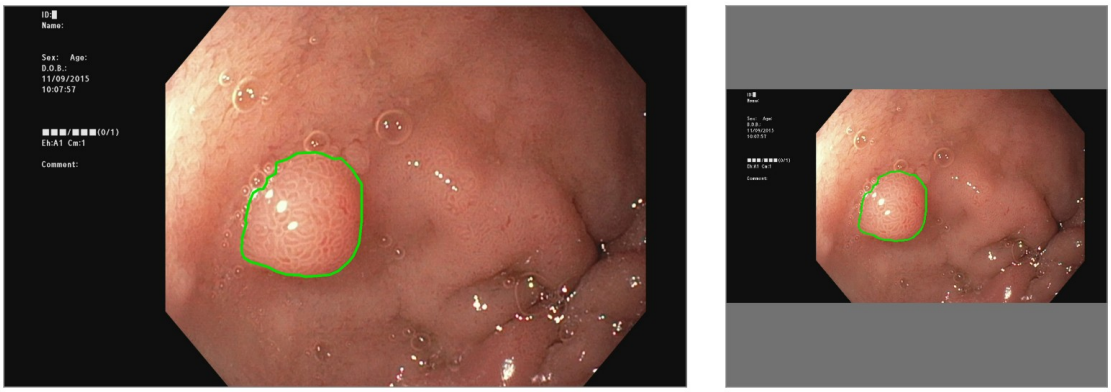
Trong đó, W và H là kích thước ảnh gốc, r là hệ số thu phóng, W' và H' là kích thước sau khi thu phóng, được làm tròn lên thành số nguyên. Ảnh lúc này có cạnh dài bằng 640 nhưng cạnh ngắn vẫn nhỏ hơn 640.

Ở nhánh kiểm định, đề tài dùng kỹ thuật LetterBox để đưa ảnh về khung kích thước cố định. LetterBox thêm các dải viền màu xám có giá trị 114 vào hai phía của cạnh ngắn, giống như các dải đen trên màn hình khi xem phim có tỷ lệ khung hình khác. Phần viền được chia đều cho hai phía:

$$p_x = \frac{640 - W'}{2}, \; p_y = \frac{640 - H'}{2} \tag{3.8}$$

Trong đó, p_x là độ rộng viền thêm vào mỗi bên trái và phải, p_y là độ rộng viền thêm vào mỗi phía trên và dưới. Tọa độ đa giác cũng được nhân với r và cộng thêm p_x, p_y, nhờ đó nhãn luôn trùng với vị trí polyp trên ảnh mới. Trong quá trình kiểm định thực tế, các ảnh có tỷ lệ khung hình gần nhau được gom vào cùng một lô, và khung đích được tính theo từng lô nên có thể là hình chữ nhật với cạnh dài 640 pixel; cách làm này giúp giảm phần viền thừa. Ở nhánh huấn luyện, việc đưa ảnh vào khung 640 × 640 được thực hiện cùng với các phép tăng cường dữ liệu ở mục 3.5, phần trống cũng được tô màu xám 114.

Hình 3.3 minh họa kết quả trước và sau công đoạn đổi kích thước trên một ảnh của tập kiểm định có kích thước lớn nhất trong bộ dữ liệu, với khung đích 640 × 640 pixel.



a. Ảnh gốc 1920 × 1072 pixel b. Sau LetterBox 640 × 640 pixel

Hình 3.3. Trước và sau khi đổi kích thước ảnh bằng thu phóng giữ tỷ lệ và LetterBox

Nguồn: ảnh a từ bộ dữ liệu Kvasir-SEG [1]; ảnh b do đề tài tạo theo cách xử lý LetterBox trong mã nguồn Ultralytics.

Ảnh gốc có kích thước 1.920 × 1.072 pixel nên hệ số thu phóng là 1/3, ảnh sau thu nhỏ có kích thước 640 × 358 pixel. LetterBox bổ sung hai dải xám, mỗi dải cao 141 pixel, ở phía trên và phía dưới để tạo thành khung 640 × 640 pixel. Có thể thấy polyp không bị méo và đường bao màu xanh vẫn nằm đúng vị trí sau khi đổi kích thước.

3.5. TĂNG CƯỜNG DỮ LIỆU

Tập huấn luyện chỉ có 1.040 ảnh, khá ít đối với một mô hình học sâu. Khi dữ liệu ít, mô hình dễ ghi nhớ từng ảnh huấn luyện thay vì học các đặc điểm chung của polyp, dẫn đến kết quả kém trên ảnh mới. Tăng cường dữ liệu khắc phục vấn đề này bằng cách tạo ra một biến thể ngẫu nhiên khác nhau của ảnh mỗi lần ảnh được nạp vào mô hình. Nhờ vậy, qua 100 epoch, mô hình gần như không gặp lại đúng một ảnh hai lần. Tăng cường dữ liệu chỉ áp dụng cho tập huấn luyện; tập kiểm định được giữ nguyên để việc đánh giá phản ánh đúng khả năng của mô hình trên ảnh thật.

Các phép biến đổi được chia thành hai nhóm. Phép biến đổi hình học như ghép ảnh, dịch chuyển, thu phóng và lật làm thay đổi vị trí các pixel, nên đa giác nhãn được biến đổi theo đúng cách tương ứng. Phép biến đổi màu chỉ thay đổi giá trị màu của pixel, nên nhãn được giữ nguyên. Trong mã nguồn, các phép

được thực hiện theo thứ tự: Mosaic, dịch chuyển và thu phóng, biến đổi màu HSV, cuối cùng là lật ngang. Thông số của từng phép được lấy từ tệp cấu hình của lần huấn luyện và tổng hợp trong Bảng 3.2.

3.5.1. Ghép bốn ảnh bằng Mosaic

Mosaic là kỹ thuật ghép bốn ảnh huấn luyện thành một ảnh, được giới thiệu trong YOLOv4 [10]. Trong mã nguồn, chương trình tạo một khung trống 1.280 × 1.280 pixel màu xám, chọn ngẫu nhiên một điểm làm tâm ghép, rồi đặt bốn ảnh vào bốn góc xung quanh điểm này. Sau đó, khung ghép được dịch chuyển, thu phóng và cắt về kích thước 640 × 640 pixel. Kết quả là một ảnh huấn luyện chứa nhiều bối cảnh khác nhau, polyp xuất hiện ở nhiều vị trí và kích thước, đôi khi chỉ hiện ra một phần. Ảnh nền cũng có thể được ghép chung với ảnh polyp, giúp mô hình phân biệt polyp với niêm mạc bình thường ngay trong cùng một ảnh.

Với xác suất bằng 1,0, Mosaic được áp dụng cho mọi ảnh trong phần lớn quá trình huấn luyện. Tuy nhiên, ảnh ghép không giống ảnh nội soi thật, nên Mosaic được tắt trong 10 epoch cuối của tổng số 100 epoch [11]. Ở giai đoạn này, mô hình được tinh chỉnh trên ảnh nguyên vẹn, gần với điều kiện sử dụng thực tế hơn.

3.5.2. Dịch chuyển và thu phóng ngẫu nhiên

Phép dịch chuyển và thu phóng được thực hiện bằng một phép biến đổi affine. Mỗi điểm có tọa độ x, y trên ảnh được chuyển đến vị trí mới theo công thức:

$$x' = s \cdot (x - c_x) + t_x, \; y' = s \cdot (y - c_y) + t_y \tag{3.9}$$

Trong đó, c_x và c_y là tọa độ tâm ảnh đầu vào. Hệ số thu phóng s được chọn ngẫu nhiên trong đoạn từ 0,5 đến 1,5, tương ứng với tham số scale bằng 0,5, nghĩa là ảnh có thể bị thu nhỏ còn một nửa hoặc phóng to gấp rưỡi. Vị trí tâm mới t_x, t_y được chọn ngẫu nhiên lệch khỏi tâm khung 640 × 640 tối đa 10% kích thước khung, tương ứng với tham số translate bằng 0,1. Phép biến đổi này giúp mô hình nhận ra polyp ở các khoảng cách quan sát và vị trí khác nhau trong khung hình. Phần ảnh nằm ngoài khung bị cắt bỏ, phần trống được tô màu xám 114, và đa giác nhãn được biến đổi theo cùng công thức. Các phép xoay, cắt xiên và biến đổi phối cảnh đều có tham số bằng 0, nên không được sử dụng.

3.5.3. Biến đổi màu trong không gian HSV

Màu sắc ảnh nội soi thay đổi theo nguồn sáng, máy nội soi và vị trí chụp. Để mô hình không phụ thuộc vào một điều kiện màu cụ thể, ảnh được chuyển sang

không gian màu HSV, trong đó H là sắc độ, tức màu cơ bản như đỏ hay cam; S là độ bão hòa, tức độ đậm nhạt của màu; V là độ sáng. Với mỗi ảnh, ba hệ số ngẫu nhiên g_h, g_s, g_v được chọn đều trong các đoạn từ −0,015 đến 0,015, từ −0,7 đến 0,7 và từ −0,4 đến 0,4. Giá trị của từng pixel được biến đổi như sau:

$$H' = (H + 180 \cdot g_h) \bmod 180 \tag{3.10}$$

$$S' = min(255, S \cdot (1 + g_s)),\ V' = min(255, V \cdot (1 + g_v)) \tag{3.11}$$

Trong OpenCV, sắc độ nhận giá trị từ 0 đến 179, nên phép chia lấy dư cho 180 giúp sắc độ quay vòng trên bánh xe màu. Sắc độ chỉ được dịch rất ít, tối đa khoảng 2,7 đơn vị trên thang 180, để màu niêm mạc không bị biến thành những màu phi thực tế như xanh lá hay xanh dương. Ngược lại, độ bão hòa và độ sáng được thay đổi nhiều hơn để mô phỏng các điều kiện chiếu sáng khác nhau. Vì đây là phép biến đổi màu, vị trí polyp và nhãn không thay đổi.

3.5.4. Lật ngang và các phép bổ sung

Mỗi ảnh có xác suất 0,5 bị lật ngang, tức đối xứng qua trục dọc ở giữa ảnh. Khi đó, tọa độ ngang đã chuẩn hóa của mọi đỉnh đa giác được tính lại theo công thức:

$$\hat{x}' = 1 - \hat{x} \tag{3.12}$$

Polyp có thể nằm ở bất kỳ phía nào của khung hình, nên ảnh lật ngang vẫn là một ảnh nội soi hợp lý và giúp tăng gấp đôi số trường hợp vị trí mà mô hình được học. Phép lật dọc không được sử dụng. Các phép trộn ảnh như MixUp, CutMix và Copy-Paste cũng đều có tham số bằng 0. Ngoài ra, mã nguồn có một nhánh tăng cường bổ sung từ thư viện Albumentations gồm làm mờ, lọc trung vị, chuyển ảnh xám và cân bằng độ tương phản cục bộ CLAHE, mỗi phép chỉ có xác suất 1%. Nhánh này chỉ hoạt động khi môi trường có cài thư viện Albumentations; nhật ký huấn luyện mô hình TSVM với seed 0 trên Kaggle ghi nhận nhánh này đã được kích hoạt.

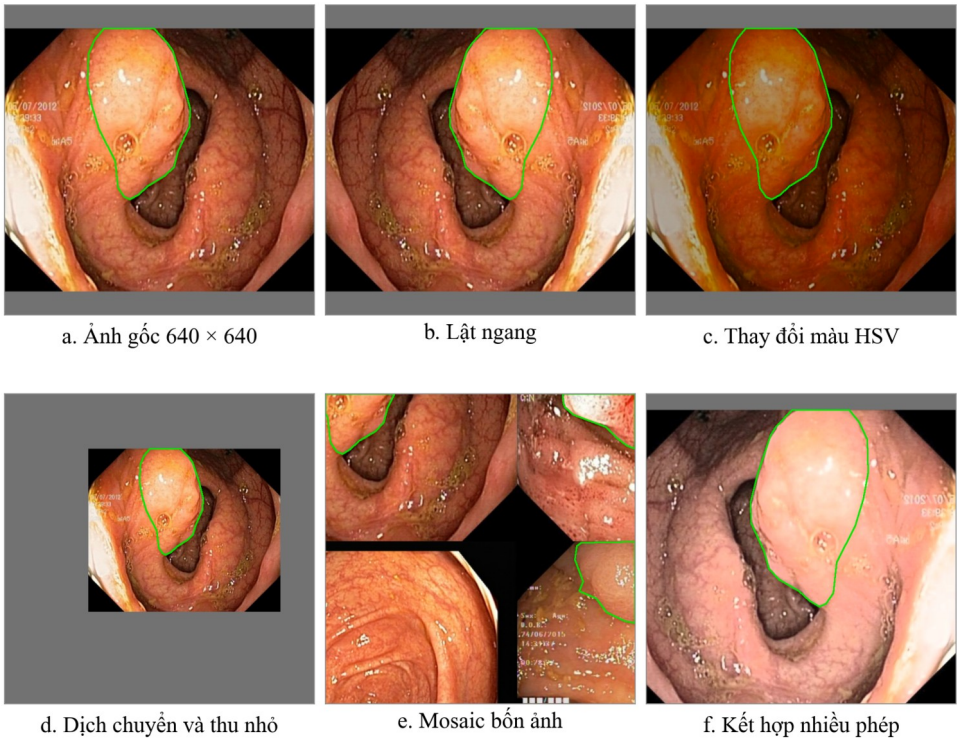
Bảng 3.2. Thông số tăng cường dữ liệu trong cấu hình huấn luyện

Phép biến đổi	Tham số	Giá trị	Ý nghĩa
Mosaic	mosaic	1,0	Luôn ghép bốn ảnh
Tắt Mosaic	close_mosaic	10	Tắt trong 10 epoch cuối
Dịch chuyển	translate	0,1	Lệch tối đa 10% khung
Thu phóng	scale	0,5	Hệ số từ 0,5 đến 1,5

Phép biến đổi	Tham số	Giá trị	Ý nghĩa
Sắc độ	hsv_h	0,015	Dịch sắc độ rất nhẹ
Độ bão hòa	hsv_s	0,7	Thay đổi tối đa 70%
Độ sáng	hsv_v	0,4	Thay đổi tối đa 40%
Lật ngang	fliplr	0,5	Xác suất 50%
Lật dọc, xoay, cắt xiên, phối cảnh	flipud, degrees, shear, perspective	0	Không sử dụng
Trộn ảnh	mixup, cutmix, copy_paste	0	Không sử dụng

Nguồn: tệp cấu hình huấn luyện của đề tài.

Hình 3.4 minh họa tác động của từng phép tăng cường lên cùng một ảnh huấn luyện. Để dễ quan sát, các hệ số trong hình được chọn cố định trong phạm vi cho phép của Bảng 3.2; khi huấn luyện, các hệ số này được chọn ngẫu nhiên cho từng ảnh.



Hình 3.4. Ảnh huấn luyện trước và sau các phép tăng cường dữ liệu

Nguồn: ảnh gốc từ bộ dữ liệu Kvasir-SEG [1] và Kvasir [2]; các phép biến đổi do đề tài tạo minh họa theo thông số trong Bảng 3.2.

So với ảnh gốc a, ảnh b được lật ngang nên polyp chuyển sang phía bên phải. Ảnh c có màu cam đậm hơn do độ bão hòa tăng và độ sáng giảm, nhưng đường

bao màu xanh không đổi. Ảnh d được thu nhỏ và dịch lệch khỏi tâm, phần trống được tô xám. Ảnh e là kết quả Mosaic từ bốn ảnh, trong đó có một ảnh nền, và các polyp chỉ hiện ra một phần ở góc ảnh. Ảnh f kết hợp lật ngang, biến đổi màu và phóng to. Trong mọi trường hợp, đường bao luôn khớp với polyp, cho thấy nhãn được biến đổi đồng bộ với ảnh.

3.6. ĐỊNH DẠNG VÀ CHUẨN HÓA DỮ LIỆU ĐẦU VÀO

Sau khi tăng cường, ảnh và nhãn được chuyển thành dạng số mà mạng nơ-ron có thể tính toán. Công đoạn này gồm bốn thao tác.

Đổi thứ tự kênh màu. Thư viện OpenCV đọc ảnh theo thứ tự kênh xanh dương, xanh lá, đỏ, gọi là BGR. Ảnh được đảo lại thành thứ tự đỏ, xanh lá, xanh dương, gọi là RGB, là thứ tự màu thông dụng của các mô hình học sâu.

Sắp xếp lại chiều dữ liệu. Ảnh ban đầu được lưu theo thứ tự chiều cao, chiều rộng, số kênh màu. Ảnh được sắp xếp lại thành số kênh màu, chiều cao, chiều rộng, rồi nhiều ảnh được ghép thành một lô. Trong cấu hình huấn luyện, mỗi lô gồm 8 ảnh.

Chuẩn hóa giá trị pixel. Mỗi pixel ban đầu là số nguyên từ 0 đến 255. Giá trị này được chuyển sang số thực và chia cho 255:

$$I_{norm} = \frac{I}{255} \tag{3.13}$$

Trong đó, I là giá trị pixel ban đầu và I_{norm} là giá trị sau chuẩn hóa, nằm trong đoạn từ 0 đến 1. Đưa dữ liệu về một thang nhỏ giúp quá trình tối ưu ổn định hơn và tránh việc các giá trị lớn chi phối phép tính. Đề tài chỉ chia cho 255, không trừ giá trị trung bình hay chia cho độ lệch chuẩn của bộ dữ liệu ImageNet.

Tạo mặt nạ huấn luyện từ đa giác. Hàm mất mát phân đoạn cần so sánh dự đoán của mô hình với một mặt nạ, nên đa giác trong tệp nhãn được tô đầy để tạo lại mặt nạ. Để tiết kiệm bộ nhớ, mặt nạ có độ phân giải nhỏ hơn ảnh 4 lần theo tham số mask_ratio bằng 4; với ảnh 640 × 640 pixel, mặt nạ có kích thước 160 × 160. Khi tham số overlap_mask được bật, các polyp trong cùng một ảnh được gộp vào một mặt nạ duy nhất, mỗi polyp được đánh một số thứ tự riêng. Với ảnh nền, mặt nạ chỉ gồm toàn giá trị 0.

3.7. KIỂM TRA TÍNH TOÀN VẸN CỦA DỮ LIỆU

Ngay sau khi tạo bộ dữ liệu, chương trình chuyển đổi tự kiểm tra số lượng ảnh và nhãn của từng tập, gồm 1.040 ảnh và 1.040 tệp nhãn ở tập huấn luyện,

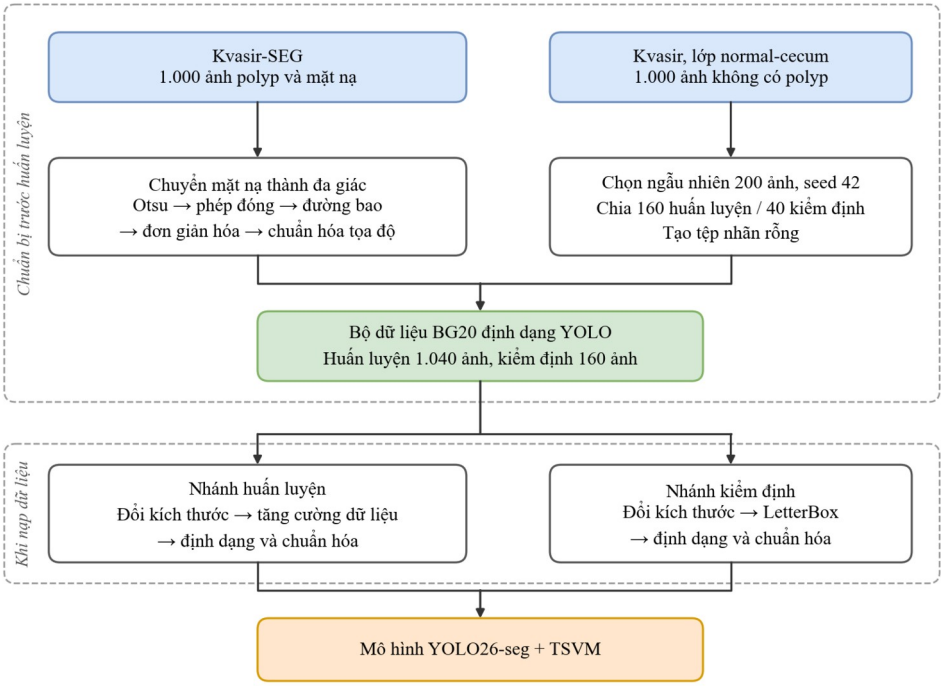
160 ảnh và 160 tệp nhãn ở tập kiểm định, cùng với số tệp nhãn rỗng lần lượt là 160 và 40. Nếu một con số không khớp, chương trình dừng lại và báo lỗi.

Bên cạnh đó, đề tài thực hiện thêm một lượt kiểm tra độc lập trên bộ dữ liệu đã dùng để huấn luyện. Kết quả cho thấy mỗi ảnh có đúng một tệp nhãn và không có tệp nhãn nào thiếu ảnh. Toàn bộ 1.063 dòng nhãn đều có chỉ số lớp 0, có ít nhất ba đỉnh và mọi tọa độ đều nằm trong đoạn từ 0 đến 1. Không có ảnh nào trùng tên hoặc trùng hoàn toàn nội dung giữa tập huấn luyện và tập kiểm định, được xác định bằng cách so sánh mã băm SHA-256 của từng tệp ảnh. Khi chạy lại chương trình chuyển đổi trên 1.000 mặt nạ gốc, cả 1.000 tệp nhãn thu được đều trùng khớp từng ký tự với nhãn đang sử dụng. Bước chọn ngẫu nhiên ảnh nền với seed bằng 42 cũng cho ra đúng danh sách 200 ảnh đã lưu. Như vậy, bộ dữ liệu có thể được tái tạo đầy đủ từ dữ liệu gốc.

Lượt kiểm tra này vẫn có một số giới hạn. Các tệp dữ liệu mà đề tài sử dụng không có thông tin định danh bệnh nhân, nên chưa thể khẳng định hai tập không chứa ảnh của cùng một bệnh nhân. Phép so sánh mã băm chỉ phát hiện được các ảnh giống nhau hoàn toàn, chưa phát hiện được các ảnh gần giống nhau như hai khung hình liên tiếp. Ngoài ra, bộ dữ liệu chỉ gồm tập huấn luyện và tập kiểm định, không có tập kiểm thử độc lập, nên kết quả thực nghiệm ở các chương sau được đánh giá trên tập kiểm định.

3.8. TỔNG HỢP QUY TRÌNH TIỀN XỬ LÝ

Hình 3.5 tóm tắt toàn bộ quy trình xây dựng bộ dữ liệu và tiền xử lý đã trình bày trong chương.



Hình 3.5. Quy trình xây dựng bộ dữ liệu BG20 và tiền xử lý dữ liệu

Nguồn: đề tài tự xây dựng.

Phần phía trên của sơ đồ là giai đoạn chuẩn bị dữ liệu, chỉ thực hiện một lần trước khi huấn luyện. Ảnh polyp từ Kvasir-SEG được giữ nguyên, còn mặt nạ được chuyển thành nhãn đa giác; ảnh normal-cecum được chọn ngẫu nhiên và gắn nhãn rỗng. Hai nguồn này hợp lại thành bộ dữ liệu BG20 gồm 1.200 ảnh. Phần phía dưới là giai đoạn xử lý khi nạp dữ liệu, được chia thành hai nhánh. Nhánh huấn luyện có thêm bước tăng cường dữ liệu để tạo sự đa dạng, còn nhánh kiểm định chỉ đổi kích thước bằng LetterBox để giữ ảnh gần với thực tế nhất. Cả hai nhánh đều kết thúc bằng bước định dạng và chuẩn hóa trước khi đưa vào mô hình.

Khối TSVM xử lý đặc trưng bên trong mô hình sau khi ảnh đã được chuẩn hóa. Hai kiến trúc sử dụng cùng quy trình tiền xử lý để giảm nhiễu do dữ liệu; đánh giá tác động kiến trúc vẫn cần xem cùng khởi tạo, tiêu chí chọn mô hình và cách bảo toàn checkpoint FP32.

CHƯƠNG 4 CÀI ĐẶT THUẬT TOÁN VÀ ỨNG DỤNG MINH HỌA

4.1 TỔ CHỨC CÀI ĐẶT VÀ MÔI TRƯỜNG

Mã mô hình cơ bản nằm trong thư mục ultralytics; bản đề xuất dùng ultralytics_Topology-Shape-aware_VMamba_FP32. Hai notebook tại Cell Kaggle mô tả cách khởi tạo dữ liệu, nạp mô hình và huấn luyện một seed. Các tệp args.yaml trong từng thư mục chạy là căn cứ cho siêu tham số đã sử dụng. Việc tổng hợp 20 seed dựa trên 40 lượt lưu độc lập, không dựa vào giả định notebook đã chứa sẵn một vòng lặp tự động 20 seed.

Bảng 4.1 Cấu hình thống nhất của 40 lượt huấn luyện

Tham số	Giá trị	Ý nghĩa
Seed	0–19	20 seed cho mỗi kiến trúc
epochs / patience	100 / 100	100 epoch được lưu mỗi lượt
imgsz / batch	640 / 8	Cấu hình kích thước và lô
optimizer / lr0	AdamW / 0,001	Cùng thuật toán và tốc độ đầu
weight_decay	0,0005	Suy giảm trọng số
warmup_epochs	5	Giai đoạn khởi động
lrf / cos_lr	0,01 / False	Không bật cosine schedule
workers	2	Nạp dữ liệu
amp / deterministic	False / True	Tắt AMP và bật chế độ tất định
close_mosaic	10	Tắt Mosaic ở 10 epoch cuối
mask_ratio / overlap_mask	4 / True	Biểu diễn mặt nạ
Pretrained	yolo26s-seg.pt	Nạp trực tiếp hoặc nạp phần tương thích

Cấu hình device bằng 0 xác định thiết bị CUDA được chọn trong phiên Kaggle, nhưng không xác nhận tên GPU, dung lượng VRAM hoặc phiên bản CUDA. Báo cáo không gán một cấu hình Tesla T4 cụ thể cho mọi lượt khi chưa có log tương ứng. AdamW là thuật toán tối ưu có suy giảm trọng số tách biệt

[22]. Các phiên bản Python, PyTorch và phần cứng dùng trong kiểm tra ứng dụng cục bộ được tách khỏi môi trường huấn luyện trên Kaggle.

Các thành phần được tổ chức thành ba nhóm. Nhóm nghiên cứu chứa mã Ultralytics cơ bản, fork FP32 và cấu hình mô hình; nhóm thực nghiệm chứa notebook, dữ liệu đã chuyển đổi và các thư mục kết quả theo seed; nhóm ứng dụng chứa dịch vụ AI cùng client Web, Mobile. Cách phân tách này giúp tránh nhầm thư mục dùng để huấn luyện với thư mục cung cấp mô hình cho ứng dụng. Đặc biệt, ứng dụng phải nạp checkpoint đã xác định nguồn và đăng ký đúng mã lớp, chứ không chỉ trỏ đến một thư mục có tên tương tự mô hình đề xuất.

Để tái lập, trước hết cần xác định phiên bản thư viện và module được import thực sự. Python có thể ưu tiên một bản Ultralytics đã cài sẵn hoặc một đường dẫn editable thay vì fork vừa sao chép. Vì vậy, in đường dẫn ultralytics.__file__, kiểm tra sự hiện diện của C2TSVMamba và kiểm tra scale s là những bước có ý nghĩa trước huấn luyện. Cấu hình GPU trong notebook chỉ biểu thị thiết bị yêu cầu; tên GPU thực tế và tài nguyên cần lấy từ phiên đang chạy khi ghi nhận môi trường.

4.2 QUY TRÌNH NOTEBOOK VÀ LƯU KẾT QUẢ

4.2.1 Notebook baseline

Notebook baseline cài Ultralytics 8.4.127, tạo data_bg20.yaml trỏ tới images/train và images/val, khai báo một lớp polyp. Mô hình được nạp bằng YOLO(yolo26s-seg.pt). Cell huấn luyện đặt seed, batch, epoch và các thiết lập tất định; cuối cùng nén thư mục runs. Danh sách seed phải được thay hoặc điều phối giữa các lượt; bản notebook đang lưu chứa một giá trị SEED cụ thể, không tự chứng minh đã chạy tất cả seed.

Quy trình notebook baseline gồm chuẩn bị môi trường, xác nhận cấu trúc ảnh và nhãn, tạo cấu hình dữ liệu, nạp checkpoint pretrained và gọi train với cấu hình thống nhất. Bước kiểm tra dữ liệu cần phân biệt một nhãn rỗng hợp lệ với một nhãn bị thiếu do lỗi chuyển đổi. Trước khi chạy, có thể mở một số ảnh kèm đa giác để xác nhận tọa độ chuẩn hóa và lớp polyp được đọc đúng. Việc chỉ kiểm tra đủ số ảnh mà không kiểm tra nhãn có thể bỏ sót sai lệch đầu vào ảnh hưởng toàn bộ phép so sánh.

Mỗi lượt phải có tên duy nhất gắn với seed để tránh ghi đè và phải giữ args.yaml cùng results.csv. Khi notebook kết thúc hoặc môi trường bị ngắt, chỉ những tệp đã lưu mới là căn cứ xác định mức hoàn thành. Trong bộ số liệu đang

dùng, 40 results.csv đều có 100 dòng epoch; đây là bằng chứng lịch sử đã ghi nhận đủ số epoch, khác với việc suy đoán từ một biến EPOCHS trong notebook. Sau khi tải về máy, cần kiểm tra tệp kết quả và trọng số còn tương ứng cùng lượt trước khi tổng hợp.

4.2.2 Notebook mô hình đề xuất và yêu cầu FP32

Notebook đề xuất sao chép YAML chung thành tên yolo26s-seg-TopologyShapeVMamba.yaml và gán scale s, sau đó nạp trọng số pretrained. Tuy nhiên, đường dẫn custom repo trong bản notebook được lưu vẫn nhắc thư mục TSVM cũ. Khi tái lập thực nghiệm FP32 cần trỏ đến fork có PATCH_FP32, kiểm tra đường dẫn module thực sự được import, đặt tên lượt chạy có TSVM_FP32 và xác nhận checkpoint được lưu bằng float32.

Các tệp kết quả được tổ chức theo seed để giữ tương ứng giữa args.yaml, results.csv, weights/best.pt và hình đánh giá. Bảng tổng hợp không thay thế các tệp gốc. Khi cần so sánh mô hình đã triển khai, phải ghi thêm mã băm checkpoint, epoch theo fitness và cấu hình suy luận; các chỉ số tại epoch tối đa mask mAP trong lịch sử train chưa phải mặc nhiên là chỉ số của best.pt.

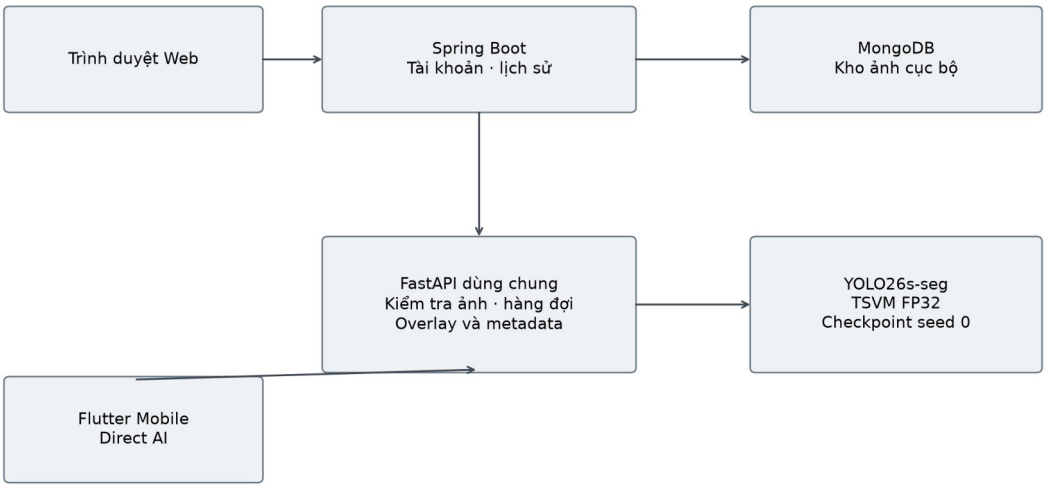
Với mô hình tùy biến, trình tự cài đặt còn gồm đăng ký lớp trong module và parser, tạo YAML có scale s, rồi xây dựng mô hình trước khi nạp các trọng số có thể dùng lại. Kiểm tra tensor đầu vào giả có kích thước đã quy định giúp phát hiện sớm lỗi concatenate hoặc số kênh. Tuy nhiên, một forward pass thành công chưa thay thế kiểm tra loss, backward và checkpoint sau lưu. Đối với bản FP32, cần xác nhận cả đường lưu cuối cùng vì lỗi ép half có thể xuất hiện sau một lượt train vẫn hoạt động bình thường.

Việc đóng gói kết quả cần giữ mối liên hệ giữa mã nguồn, cấu hình và trọng số. Một bộ tối thiểu gồm phiên bản hoặc bản chụp mã, YAML thực dùng, data.yaml, args.yaml, results.csv, weights và thông tin seed. Những biểu đồ được sinh từ các tệp này là kết quả dẫn xuất, có thể tái tạo khi thuật toán tổng hợp được lưu. Nếu chỉ giữ ảnh biểu đồ mà thiếu số liệu, việc tính lại chênh lệch hoặc đổi quy tắc chọn epoch sẽ khó kiểm tra. Đây là cơ sở cho cách tổ chức bộ kết quả mở rộng ở Chương 5.

4.3 TỔNG QUAN ỨNG DỤNG POLYPVISION

PolypVision có hai client độc lập dùng chung dịch vụ AI. Trình duyệt giao tiếp với Spring Boot; tầng Web quản lý tài khoản, phân quyền, ảnh và lịch sử trong MongoDB cùng kho tệp cục bộ. Flutter Mobile Direct AI gửi yêu cầu trực

tiếp tới FastAPI, không bắt buộc đi qua Web và không dùng MongoDB như một lịch sử cloud mặc định.



Hình 4.1 Kiến trúc Web và Mobile dùng chung dịch vụ suy luận FastAPI

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Mã Web khai báo Java 17 và Spring Boot 4.1.0, sử dụng Thymeleaf, Spring Security và Spring Data MongoDB. Mobile sử dụng Flutter và các thư viện Dio, Riverpod, go_router, image_picker. Dịch vụ AI dùng FastAPI, PyTorch, Ultralytics, Pillow và OpenCV. Những thông tin này mô tả mã nguồn hiện tại, không thay cho một phép đo tải đồng thời hoặc khả năng vận hành dài hạn.

Việc tách dịch vụ suy luận khỏi giao diện giúp hai client dùng chung một nguồn mô hình và quy tắc tiền xử lý. Phía giao diện tiếp nhận thao tác người dùng và trình bày kết quả; phía AI chịu trách nhiệm kiểm tra ảnh, lựa chọn runtime, suy luận và tạo dữ liệu phản hồi. Với Web, tầng Spring Boot bổ sung quản lý người dùng và lịch sử; với Mobile Direct AI, yêu cầu được gửi thẳng đến FastAPI. Do đó, một lỗi kết nối dịch vụ có thể ảnh hưởng cả hai client, trong khi lỗi lưu lịch sử trên Web không đồng nghĩa mô hình AI không thể chạy trên Mobile.

Luồng sử dụng chính bắt đầu bằng chọn ảnh và mô hình. Người dùng có thể xem ảnh gốc cùng overlay, số đối tượng, độ tin cậy và thời gian phản hồi được cung cấp. Trong chế độ so sánh, cùng ảnh được gửi đến hai mô hình và hai kết quả được đặt cạnh nhau để quan sát. Thiết kế này phục vụ giải thích sự khác nhau trên một trường hợp; nó không tự tạo nhãn chuẩn và không thay thế bộ

đánh giá định lượng. Vì vậy, giao diện cần giữ thông tin tên mô hình và ảnh gốc để người đọc truy nguyên kết quả đang xem.

4.4 DỊCH VỤ SUY LUẬN VÀ HỢP ĐỒNG API

4.4.1 Nạp và định danh mô hình

Registry được chuyển sang hai khóa YOLO26s-seg và YOLO26s-TSVM-FP32-seg. Tệp mặc định là yolo26s-seg-best.pt và yolo26s-tsvm-fp32-best.pt, lấy từ seed 0 của hai nhóm thực nghiệm. Chọn cùng một seed cố định để minh họa tránh việc mỗi mô hình được chọn riêng theo lượt thuận lợi nhất. Đây không phải khẳng định seed 0 đại diện cho phân bố 20 seed.

Trước khi nạp checkpoint TSVM, dịch vụ đăng ký các lớp trong ultralytics.nn.modules.topology_shape_vmamba, tương ứng đúng tên module được lưu trong checkpoint. Sau khi tải, dịch vụ kiểm tra task segment, nhãn polyp và sự hiện diện của C2TSVMamba trong mô hình cải tiến. Tệp experiment_manifest.json ghi nguồn, seed, kích thước và SHA-256 của hai checkpoint để đối chiếu.

Checkpoint tùy biến lưu tham chiếu tới các lớp Python đã tạo nên kiến trúc. Nếu tên module hoặc tên lớp không có trong môi trường dịch vụ, lỗi có thể xảy ra ngay khi giải tuần tự, trước bước predict. Vì vậy, bản sao mã TSVM dùng trong ứng dụng phải tương thích với checkpoint và được đăng ký trước khi gọi YOLO để nạp. Kiểm tra số lượng lớp và task sau tải giúp phát hiện trường hợp nạp nhầm một mô hình detection hoặc checkpoint nhiều lớp vào luồng segmentation một lớp.

Manifest đóng vai trò nhận dạng hiện vật thực nghiệm. Khi đổi checkpoint, cần cập nhật đường dẫn nguồn và mã băm, không chỉ đổi nhãn hiển thị. Seed, quy tắc chọn checkpoint và kiểu số cũng cần được giữ cùng thông tin này. Nhờ đó, ảnh minh họa của ứng dụng có thể được đối chiếu lại với đúng tệp trọng số; những kết quả cũ vẫn được nhận biết là của phiên bản trước và không bị gán sang mô hình FP32 bằng thao tác đổi tên.

4.4.2 Tiếp nhận ảnh và suy luận

Ảnh tải lên được kiểm tra MIME và nội dung giải mã, giới hạn 10 MB và 40 triệu pixel. Các lỗi tệp rỗng, tệp giả ảnh, định dạng không hỗ trợ và ảnh hỏng được trả về rõ ràng. Sau xử lý hướng ảnh, dữ liệu RGB được giữ dưới dạng PIL

khi chuyển cho Ultralytics. Cách này tránh việc một mảng RGB bị thư viện hiểu là BGR và đảo sai kênh màu.

Cả hai mô hình được chuyển về float32; nhánh predict sử dụng half=False. Dịch vụ trả thông tin phát hiện, nhãn, độ tin cậy, số mặt nạ, thời gian xử lý và ảnh overlay JPEG mã hóa Base64. Độ tin cậy của dự đoán không phải xác suất bệnh ác tính; diện tích mặt nạ ở mức pixel cũng không tự chuyển được thành kích thước tổn thương theo milimét nếu không có hiệu chuẩn hình học.

Bảng 4.2 Các endpoint chính của dịch vụ AI

Endpoint	Client	Nội dung
GET /health	Web	Trạng thái thiết bị và registry
POST /predict	Web	Ảnh, modelType, overlay và metadata
GET /api/v1/health	Mobile	Trạng thái và requestId
GET /api/v1/models	Mobile	Hai model key và ready
POST /api/v1/inference	Mobile	Ảnh và modelKey
POST /api/v1/comparison	Mobile	Cùng ảnh qua hai mô hình

Giới hạn đầu vào có hai lớp: dung lượng tệp hạn chế lượng dữ liệu nhận, còn số pixel hạn chế ảnh giải mã có kích thước quá lớn. Tên đuôi tệp và MIME chỉ là thông tin ban đầu; khả năng giải mã nội dung mới xác nhận dữ liệu có thể được xử lý như ảnh. Sau khi mở ảnh, hệ thống xử lý hướng hiển thị và chuẩn hóa về RGB trước khi chuyển đến mô hình. Các thao tác này giúp ảnh người dùng nhìn thấy và ảnh mô hình nhận vào có cùng cách diễn giải về hướng và màu.

Overlay được tạo từ ảnh đầu vào và kết quả dự đoán để hỗ trợ quan sát. Mã hóa Base64 giúp truyền ảnh này trong phản hồi JSON nhưng làm tăng lượng dữ liệu so với tệp nhị phân tương ứng, nên không nên coi thời gian predict là toàn bộ thời gian phản hồi. Một quy trình đo ứng dụng hoàn chỉnh cần xác định rõ điểm bắt đầu, điểm kết thúc và các bước bao gồm trong phép đo. Phần benchmark mô hình vì vậy được trình bày tách với các kiểm tra chức năng giao thức.

4.4.3 Điều phối yêu cầu và xử lý lỗi

Các endpoint Mobile sử dụng requestId để theo dõi yêu cầu và trả lỗi JSON thống nhất. Dịch vụ giới hạn số lượt suy luận đồng thời bằng semaphore, áp dụng timeout chờ hàng đợi và timeout xử lý. Mỗi runtime còn có khóa bảo vệ mô hình. Khi một mô hình không sẵn sàng, API cần phản ánh trạng thái degraded hoặc unavailable thay vì trả một kết quả suy luận giả.

Thời gian trong phản hồi được đo quanh lệnh predict. Nó có thể bao gồm khởi tạo predictor ở lần đầu và không nhất thiết bao gồm tải ảnh qua mạng, thời gian chờ, mã hóa ảnh hoặc lưu lịch sử ở tầng Web. Do đó, không dùng trực tiếp thời gian của một yêu cầu minh họa để tuyên bố FPS ổn định cho cả ứng dụng.

Khóa runtime bảo vệ một mô hình khi nhiều yêu cầu cùng truy cập, còn semaphore giới hạn số tác vụ được phép thực hiện theo cấu hình dịch vụ. Những cơ chế này nhằm kiểm soát tài nguyên và hành vi khi tải tăng, không phải bằng chứng rằng hệ thống đã được kiểm thử tải lớn. Một yêu cầu vượt thời gian chờ cần có thông báo phù hợp để client không nhầm thành kết quả "không có polyp". Tương tự, kết quả rỗng hợp lệ phải được phân biệt với lỗi nạp mô hình, lỗi giải mã hoặc lỗi xử lý.

requestId cung cấp một định danh chung để ghép phản hồi với yêu cầu đang hiển thị. Điều này có ý nghĩa khi người dùng đổi ảnh hoặc gửi lại thao tác trong lúc yêu cầu trước chưa hoàn tất. Thông tin lỗi cần đủ để xác định bước hỏng trong luồng phần mềm nhưng không đưa nội dung nhạy cảm hoặc đường dẫn nội bộ không cần thiết vào thông báo công khai. Ở phía báo cáo, chỉ mô tả các nhóm lỗi và hợp đồng đã triển khai; không coi một vài kiểm tra đơn vị là kiểm chứng an toàn vận hành trong mọi điều kiện.

4.5 CHỨC NĂNG WEB VÀ MOBILE

4.5.1 Ứng dụng Web

Web cung cấp trang tải ảnh phân tích đơn, trang so sánh hai mô hình trên cùng ảnh, trang kết quả và lịch sử. Nhánh so sánh sử dụng cùng ảnh đầu vào và lưu hai kết quả theo một comparison group. Tầng dịch vụ kiểm tra quyền sở hữu khi truy xuất, xuất hoặc xóa bản ghi. Trang quản trị phục vụ quản lý người dùng và thống kê của ứng dụng.

Khi chuyển phiên bản nghiên cứu, tên mô hình trong lựa chọn, khóa gửi API và tên trả về phải đồng nhất. Chỉ đổi nhãn hiển thị mà không đổi checkpoint hoặc

lớp tùy biến sẽ không tạo ra một phép so sánh đúng. Lịch sử của nghiên cứu cũ cần được nhận biết theo modelType đã lưu, không được đổi dữ liệu cũ thành kết quả TSVM FP32 chỉ bằng cách đổi tên giao diện.

Luồng phân tích đơn và luồng so sánh cần được quản lý khác nhau ở tầng lịch sử. Một kết quả phân tích đơn gắn với một mô hình, còn một nhóm so sánh phải giữ được hai kết quả của cùng ảnh. Khi xem lại, tên mô hình, thời điểm và thông tin dự đoán cho phép nhận biết đúng nguồn kết quả. Quyền sở hữu được kiểm tra ở phía dịch vụ để việc biết một định danh bản ghi chưa đủ cho phép người dùng khác truy xuất hoặc xóa dữ liệu tương ứng.

Giao diện kết quả cần trình bày mặt nạ như thông tin tham khảo cùng ảnh gốc và độ tin cậy, tránh chuyển điểm confidence thành tỷ lệ xác suất mắc bệnh. Với ảnh không có phát hiện, nội dung phù hợp là mô hình không trả về đối tượng tại ngưỡng đang dùng. Cách diễn đạt này phản ánh đúng đầu ra kỹ thuật và không biến dự đoán rỗng thành kết luận ảnh hoàn toàn bình thường. Các màn hình trong ứng dụng là nơi thể hiện hành vi mô hình để trình diễn đề tài, trong khi nhận định y khoa nằm ngoài chức năng đã kiểm chứng.

4.5.2 Ứng dụng Mobile Direct AI

Mobile có các màn hình trạng thái dịch vụ, chọn ảnh, phân tích đơn và so sánh trực tiếp. Danh sách mô hình lấy từ API phải khớp hai khóa được hỗ trợ. Ảnh overlay được giải mã từ Base64 và hiển thị cùng thông tin model, confidence, số mặt nạ và thời gian. Địa chỉ dịch vụ được truyền bằng AI_BASE_URL; khi chạy trên điện thoại cần dùng địa chỉ mạng có thể truy cập từ thiết bị.

Nhánh Mobile Direct AI không được mô tả như đã có lưu hồ sơ bệnh án hoặc đồng bộ lịch sử với Web nếu luồng đó không được triển khai. Phạm vi ứng dụng hiện tại là minh họa nghiên cứu trên ảnh tĩnh. Các chức năng video thời gian thực, định lượng kích thước vật lý và hỗ trợ quyết định lâm sàng cần một quy trình phát triển và kiểm chứng riêng.

Khác biệt quan trọng của Mobile là địa chỉ localhost được hiểu theo chính thiết bị chạy ứng dụng. Khi thử trên điện thoại, dịch vụ AI phải có địa chỉ mạng có thể truy cập và cấu hình cổng phù hợp; khi thử trên môi trường giả lập, cách truy cập máy chủ cũng phụ thuộc môi trường. Vì vậy, cấu hình AI_BASE_URL được tách khỏi logic phân tích để tránh phải sửa mã chức năng cho mỗi thiết bị.

Kiểm tra kết nối và trạng thái mô hình trước khi gửi ảnh giúp phân biệt lỗi mạng với lỗi suy luận.

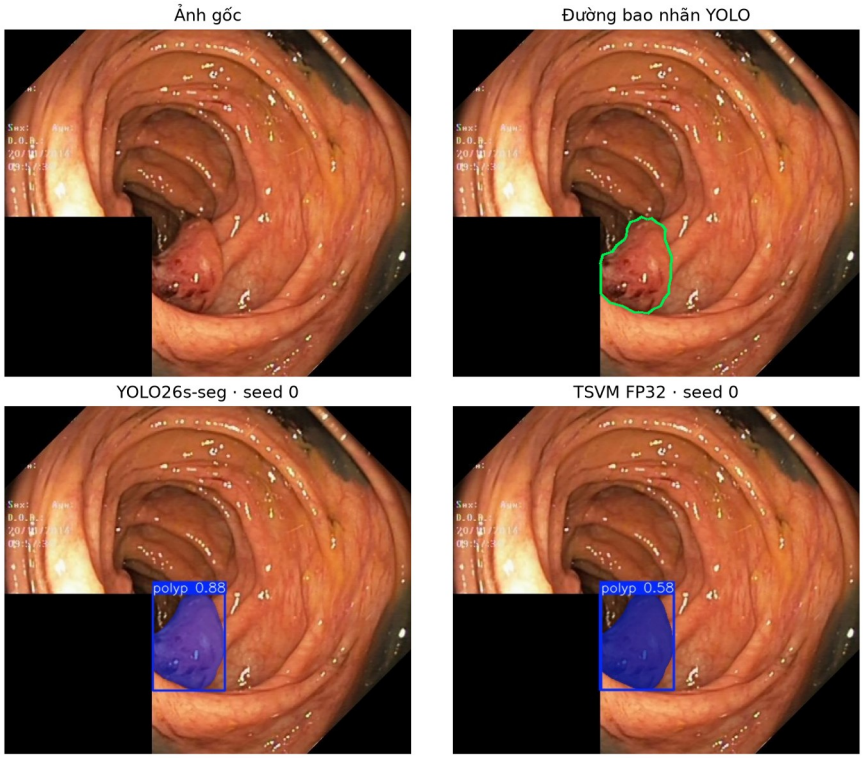
Ảnh chọn từ bộ nhớ hoặc camera cần được đưa vào cùng quy trình kiểm tra của API. Nguồn ảnh camera có thể tạo góc nhìn và chất lượng khác với dữ liệu nội soi dùng trong thực nghiệm; chức năng chọn ảnh không đồng nghĩa mọi ảnh chụp bất kỳ đều thuộc miền dữ liệu phù hợp. Trong phạm vi trình diễn, Mobile đóng vai trò client gửi ảnh nội soi đã có, nhận overlay và so sánh hai mô hình. Không suy ra khả năng chẩn đoán từ việc giao diện có nút mở camera.

4.6 KIỂM TRA TÍCH HỢP VỚI CHECKPOINT THỰC NGHIỆM

Bảng 4.3 Phạm vi kiểm tra ứng dụng sau chuyển mô hình

Nhóm kiểm tra	Phạm vi	Kết quả
FastAPI	Giao thức, lỗi, so sánh, đồng thời, RGB	51 kiểm thử đạt
Spring Boot	Dịch vụ, controller, phân quyền, hợp đồng API	261 kiểm thử đạt
Flutter	Mô hình dữ liệu, điều hướng, màn hình, hợp đồng	498 kiểm thử đạt
Checkpoint thật	Hai model seed 0, hai ảnh mỗi model	Nạp và dự đoán thành công
Kiểu số	Tensor mô hình sau nạp	Float32 và hữu hạn

Hai checkpoint seed 0 đã được nạp với đúng task segment và lớp polyp; các tensor dấu phẩy động trong mô hình sau tải là float32 và hữu hạn. Kiểm tra thực hiện dự đoán trên cùng một ảnh có polyp và một ảnh normal-cecum cho cả hai mô hình, với ngưỡng confidence 0,25. Cả hai mô hình trả một mặt nạ trên ảnh polyp và không trả mặt nạ trên ảnh nền trong hai ví dụ này.



Hình 4.2 Ví dụ suy luận thực tế của hai checkpoint seed 0 và đường bao nhãn đối chiếu

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Hình 4.2 dùng cùng ảnh và cùng checkpoint đã nạp vào dịch vụ. Confidence của hai mô hình khác nhau, nhưng không được dùng một ảnh để suy luận mô hình nào tốt hơn trên toàn bộ dữ liệu. Đường bao nhãn trong hình được dựng từ nhãn YOLO hiện có, không phải một lần gán nhãn y khoa mới. Các kiểm tra giao thức với mô hình giả lập được tách khỏi kiểm tra tải và suy luận với checkpoint thật.

Các kiểm tra được phân thành hai nhóm để tránh nhầm phạm vi chứng minh. Kiểm thử phần mềm dùng đầu vào và đối tượng giả lập khi cần để xác nhận định dạng phản hồi, xử lý lỗi, chọn mô hình và tương tác giữa các lớp. Kiểm tra với checkpoint thật xác nhận mô hình được giải tuần tự, lớp tùy biến đã đăng ký, tensor hữu hạn và lệnh predict tạo đầu ra. Cả hai nhóm đều cần cho triển khai, nhưng nhóm thứ nhất không đo độ chính xác phân đoạn, còn nhóm thứ hai với số ảnh ít cũng chưa tạo thành một đánh giá hiệu năng mới.

Hai ví dụ seed 0 trong Hình 4.2 là kiểm tra tính khả dụng của luồng suy luận. Để đánh giá chất lượng checkpoint seed 0 một cách độc lập với biểu đồ huấn luyện, cần chạy lại toàn bộ tập đánh giá với quy tắc cố định và lưu dự đoán, thời gian, cấu hình. Muốn đánh giá ưu thế kiến trúc còn phải xét nhiều seed và bất

định như Chương 5. Việc tách các mức kiểm tra giúp báo cáo ghi nhận đúng phần mềm đã chạy mà không gán ý nghĩa vượt quá dữ liệu quan sát.

4.7 VẬN HÀNH VÀ GIỚI HẠN ỨNG DỤNG

Để vận hành, khởi động dịch vụ AI trước, kiểm tra /health và /api/v1/models, sau đó chạy Spring Boot hoặc Flutter. Web cần cấu hình MongoDB và kho lưu ảnh; Mobile Direct AI chỉ cần kết nối được dịch vụ AI. Tham số đường dẫn checkpoint có thể thay bằng biến môi trường AI_BASELINE_MODEL_PATH và AI_VMAMBA_MODEL_PATH. Các thông tin đăng nhập cá nhân và mật khẩu không được đưa vào báo cáo hoặc bản mã chia sẻ.

Phần mềm là sản phẩm minh họa của đề tài. Kết quả cần được người dùng xem cùng ảnh gốc và bối cảnh nghiên cứu; mô hình chưa được đánh giá tiền cứu trên một quy trình khám bệnh. Việc kiểm tra API, Web hoặc Mobile thành công xác nhận hành vi phần mềm trong các ca kiểm thử, không chứng nhận khả năng sử dụng như thiết bị y tế.

Khi thay mô hình, trình tự kiểm tra gồm xác nhận manifest, nạp dịch vụ AI, kiểm tra trạng thái sẵn sàng và danh sách mô hình, rồi thử một ảnh trước khi sử dụng các client. Web cần kiểm tra thêm kết nối cơ sở dữ liệu và đường dẫn lưu ảnh; Mobile cần kiểm tra địa chỉ máy chủ từ thiết bị. Sau thay đổi mã hoặc tệp cấu hình, tiến trình đang chạy có thể vẫn giữ runtime cũ trong bộ nhớ, nên cần khởi động lại dịch vụ để xác nhận phiên bản đang phục vụ.

Các giới hạn về dữ liệu và đánh giá của mô hình cũng đi theo ứng dụng. Phần mềm không làm tăng bằng chứng về tổng quát hóa chỉ bằng việc đặt một giao diện quanh checkpoint. Nếu triển khai cho nguồn ảnh khác, cần đánh giá lại miền dữ liệu, cách kiểm soát đầu vào, sai số và yêu cầu bảo vệ thông tin. Trong khóa luận, ứng dụng thể hiện khả năng kết nối một kết quả nghiên cứu với quy trình sử dụng có thể quan sát; những bước đánh giá vận hành dài hạn được xác định là công việc tiếp theo.

4.8 TÓM TẮT CHƯƠNG

Chương này nối kết cấu hình thực nghiệm với dịch vụ suy luận và hai client của PolypVision. Việc chuyển sang checkpoint S và TSVM FP32 bao gồm cả tên định danh, mã kiến trúc, độ chính xác số và đường truyền ảnh. Các kết quả kiểm tra phần mềm được trình bày riêng với đánh giá chất lượng mô hình ở chương tiếp theo.

CHƯƠNG 5 THỬ NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

5.1 MỤC TIÊU VÀ TỔ CHỨC THỰC NGHIỆM

5.1.1 Các câu hỏi cần trả lời

Thực nghiệm đánh giá việc thay C2PSA bằng C2TSVMamba trong YOLO26s-seg trên bộ BG20. Phân tích tập trung vào ba khía cạnh: chất lượng phân đoạn theo đối tượng, độ biến thiên giữa các lần khởi tạo và chi phí suy luận khi sử dụng checkpoint đã lưu. Cách tổ chức này giúp làm rõ lợi ích và chi phí của mô hình đề xuất, đồng thời phân biệt chất lượng dự đoán với khả năng vận hành phần mềm.

Chỉ số chính là mask mAP50–95. Mask mAP50, Precision, Recall, các chỉ số hộp bao và validation loss được dùng để giải thích thêm hành vi mô hình. Việc tăng một chỉ số riêng lẻ không đủ để kết luận toàn bộ hệ thống tốt hơn: Precision có thể tăng trong khi Recall giảm; mặt nạ có AP cao hơn cũng chưa chắc có khoảng cách biên nhỏ hơn theo một phép đo hình học.

5.1.2 Hai nhóm đối chứng và đơn vị phân tích

Nhóm cơ bản sử dụng YOLO26s-seg; nhóm đề xuất sử dụng YOLO26s-seg tích hợp TSVM với quy trình lưu checkpoint FP32. Mỗi nhóm có 20 seed từ 0 đến 19, tạo thành 40 lượt huấn luyện, mỗi lượt lưu 100 epoch. Các kết quả TSVM thuộc giai đoạn không có hậu tố FP32 không được đưa vào phép so sánh này. Tập dữ liệu, quy tắc chia, kích thước đầu vào và các siêu tham số chung được mô tả tại Chương 3 và Chương 4.

Đơn vị dùng để tính trung bình và kiểm định là một lượt huấn luyện. Một trăm epoch của một lượt có quan hệ phụ thuộc theo thời gian, nên không được xem như 100 phép thử độc lập. Tương tự, 20 seed cùng sử dụng một tập kiểm định cố định; chúng phản ánh biến thiên do khởi tạo và tối ưu hóa, không tương đương 20 tập bệnh nhân độc lập.

Các lượt được ghép theo chỉ số seed để khảo sát chênh lệch giữa hai kiến trúc. Việc đặt cùng seed giúp tổ chức đối chứng nhưng không bảo đảm mọi phép lấy số ngẫu nhiên diễn ra giống nhau, vì hai mạng có số tham số và luồng tính toán khác nhau. Do đó, báo cáo sử dụng cả mô tả từng cặp và phân bố của từng mô hình để tránh diễn giải quá mức ý nghĩa của phép ghép.

5.1.3 Nguồn số liệu và khả năng truy vết

Nguồn chính là 40 tệp results.csv cùng args.yaml tương ứng trong hai nhóm thư mục thực nghiệm. Bộ KQ_Nen_DX_20seed_FP32 được tổ chức lại thành các mục dữ liệu trích xuất, thống kê, nhóm độ đo, ma trận nhầm lẫn, biểu đồ, báo cáo và kiểm tra. Các thư mục này chứa những cách tổng hợp khác nhau của cùng nguồn dữ liệu, không phải các đợt huấn luyện mới.

Mục 01_raw_analysis lưu dữ liệu từng lượt và lịch sử epoch; 02_statistics chứa trung bình, độ lệch chuẩn, khoảng giá trị và chênh lệch theo seed; 03_metrics tách kết quả mặt nạ, hộp bao và loss; 04_confusion_matrix xử lý số đếm đối tượng; 05_charts cung cấp hình minh họa; 06_reports trình bày diễn giải; 07_audit lưu đối chiếu và thông tin nguồn. efficiency_benchmark là phép đo suy luận riêng trên hai checkpoint seed 0, không được trộn với thống kê chất lượng của 20 seed.

5.2 QUY TẮC TRÍCH XUẤT VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

5.2.1 Phân biệt epoch tối đa mask với checkpoint best

Để thống nhất với bộ tổng hợp FP32 đã lưu, mỗi lượt được trích tại epoch có metrics/mAP50-95(M) lớn nhất. Những chỉ số khác của lượt đó được lấy trên cùng dòng, không chọn riêng cực đại Precision, cực đại Recall và cực tiểu loss rồi ghép thành một mô hình giả định. Nếu nhiều epoch có cùng giá trị cực đại, lấy lần xuất hiện đầu tiên trong lịch sử.

Ultralytics lưu best.pt dựa trên fitness tổng hợp. Trong fork đang sử dụng, fitness phân đoạn là tổng thành phần mAP50–95 của hộp bao và mặt nạ. Đối chiếu lịch sử cho thấy 16 trong 40 lượt có epoch tối đa mask khác epoch tối đa fitness. Bởi vậy, bảng tổng hợp theo max-mask mô tả lịch sử huấn luyện tại mốc chọn theo độ đo chính; nó không mặc nhiên là kết quả đánh giá lại mọi tệp best.pt sau lưu. Phụ lục B liệt kê hai mốc để người đọc kiểm tra.

Quy tắc chọn epoch sử dụng chính tập kiểm định nên có thể tạo mức đánh giá lạc quan so với một tập kiểm thử chưa từng được dùng để lựa chọn mô hình. Giữ cùng quy tắc ở cả hai nhóm giúp phép đối chiếu nhất quán, nhưng không loại bỏ giới hạn này. Các checkpoint của ứng dụng và benchmark được nhận diện bằng seed, đường dẫn và SHA-256 riêng.

5.2.2 Ý nghĩa các độ đo

IoU đo tỷ lệ phần giao trên phần hợp giữa vùng dự đoán và vùng nhãn. Trong đánh giá từng đối tượng, dự đoán còn phải được ghép với đối tượng thật theo điều kiện IoU và lớp; sau đó mới xác định dự đoán đúng, dự đoán sai và đối tượng bị bỏ sót. Precision biểu thị tỷ lệ dự đoán đúng trong các dự đoán được chấp nhận, còn Recall biểu thị tỷ lệ đối tượng thật được tìm thấy.

$$IoU = |P \cap G| / |P \cup G| \; ; \; Precision = TP / (TP + FP) \; ; \; Recall = TP / (TP + FN)$$

AP tổng hợp đường precision–recall khi thay đổi ngưỡng confidence. mAP50 sử dụng ngưỡng IoU 0,50; mAP50–95 lấy trung bình các ngưỡng từ 0,50 đến 0,95 với bước 0,05. Trong bài toán một lớp, phép lấy trung bình theo lớp chỉ còn lớp polyp. Hậu tố M chỉ đánh giá mặt nạ; hậu tố B chỉ đánh giá hộp bao. Các ngưỡng IoU chặt hơn làm mAP50–95 nhạy hơn với sai lệch định vị so với mAP50, nhưng vẫn không thay thế trực tiếp một độ đo khoảng cách đường biên.

Dice và IoU theo điểm ảnh thường xuất hiện trong các bài báo phân đoạn ngữ nghĩa. Chúng khác với AP theo đối tượng về cách tổng hợp, ngưỡng dự đoán và cách xử lý nhiều đối tượng trong cùng ảnh. Vì vậy, báo cáo không suy Dice từ mAP hoặc từ Precision và Recall theo đối tượng. Bộ số liệu chính chưa có Dice theo ảnh, Boundary F1 hoặc HD95 được tính lại cho cùng checkpoint.

5.2.3 Thống kê mô tả và kiểm định ghép cặp

Với mỗi độ đo, báo cáo tính trung bình và độ lệch chuẩn mẫu trên 20 lượt. Độ lệch chuẩn sử dụng mẫu số $n-1$. Trung bình thể hiện mức đạt được của nhóm trong thiết lập này, còn SD mô tả mức phân tán giữa seed. Min, max, median và khoảng biến thiên bổ sung thông tin về các trường hợp thuận lợi và bất lợi mà một cột trung bình có thể che lấp.

Với seed i, đặt d_i bằng chỉ số TSVM FP32 trừ chỉ số baseline. Khoảng tin cậy 95% của trung bình chênh lệch được tính theo phân bố t với 19 bậc tự do [23]. Kiểm định t ghép cặp hai phía kiểm tra giả thuyết trung bình chênh lệch bằng 0; Wilcoxon signed-rank là phân tích bổ sung dựa trên thứ hạng độ lớn chênh lệch [24]. Kiểm định dấu bổ sung câu hỏi về số cặp tăng và giảm, không xét mức tăng của từng cặp.

$$d_i = TSVM_i - Baseline_i \; ; \; KTC95\% = \text{trung bình}(d) \pm t_{0,975;19} \times SD(d) / \sqrt{20}$$

Mask mAP50–95 là chỉ số chính; những phép kiểm định khác có tính bổ sung hoặc thăm dò. Khi xem nhiều p-value cùng lúc, nguy cơ gặp một giá trị nhỏ do ngẫu nhiên tăng lên. Báo cáo không chọn riêng một phép kiểm định thuận lợi để tuyên bố ưu thế. Một p-value lớn hơn 0,05 cũng không chứng minh hai mô hình tương đương; nó cho biết bằng chứng hiện có chưa đủ để bác bỏ giả thuyết theo phép kiểm định tương ứng.

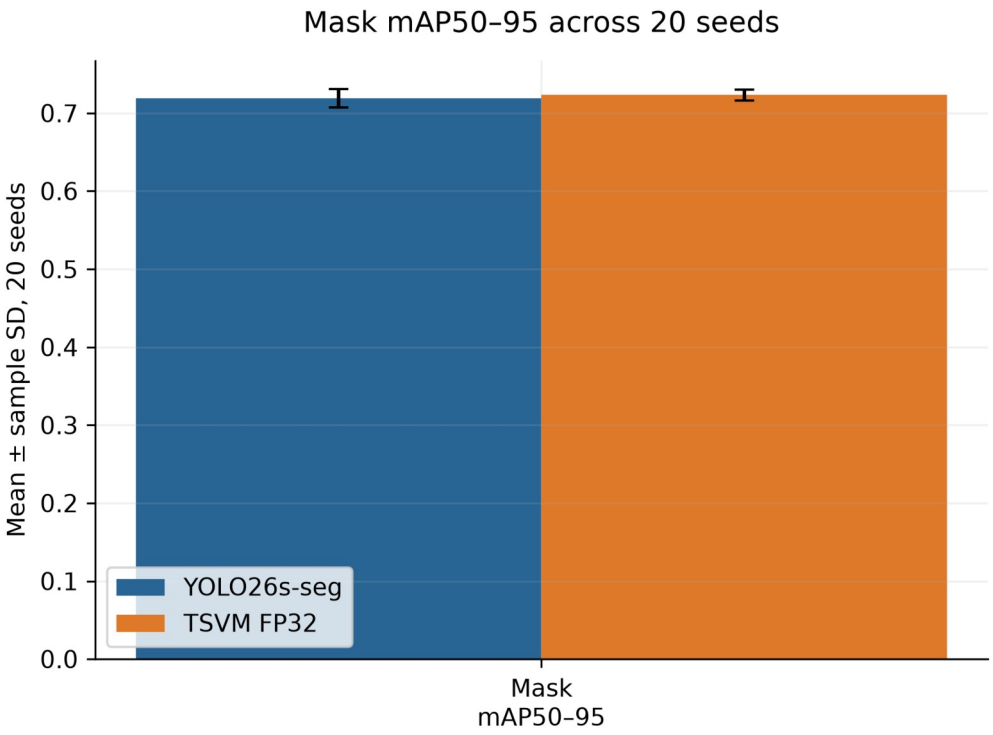
5.3 CHẤT LƯỢNG PHÂN ĐOẠN VÀ PHÁT HIỆN ĐỐI TƯỢNG

5.3.1 Bảng kết quả tổng hợp

Bảng 5.1 Kết quả tại epoch tối đa mask mAP50 95 của mỗi lượt

Chỉ số	Baseline trung bình ± SD	TSVM FP32 trung bình ± SD	Δ
Mask mAP50–95	0,7189 ± 0,0118	0,7232 ± 0,0072	0,0043
Mask mAP50	0,9069 ± 0,0125	0,9083 ± 0,0086	0,0014
Mask Precision	0,9041 ± 0,0287	0,9145 ± 0,0233	0,0104
Mask Recall	0,8594 ± 0,0241	0,8590 ± 0,0179	-0,0004
Box mAP50–95	0,7228 ± 0,0159	0,7273 ± 0,0134	0,0045
Box mAP50	0,9006 ± 0,0116	0,9014 ± 0,0083	0,0008
Box Precision	0,9006 ± 0,0263	0,9067 ± 0,0238	0,0061
Box Recall	0,8501 ± 0,0293	0,8522 ± 0,0145	0,0021
Val segmentation loss	1,2663 ± 0,0843	1,2329 ± 0,0410	-0,0334

Tại mốc chọn theo mask của mỗi lượt, mask mAP50–95 trung bình tăng từ 0,718868 lên 0,723171. Chênh lệch là 0,004303 trên thang 0–1, tương đương 0,4303 điểm phần trăm; đây không phải mức tăng 0,4303 lần. SD giảm từ 0,011768 xuống 0,007197, tương ứng giảm khoảng 38,85%. Hai con số này cần được đọc riêng: mức trung bình nhích lên, trong khi các lượt TSVM phân tán ít hơn trong tập seed khảo sát.

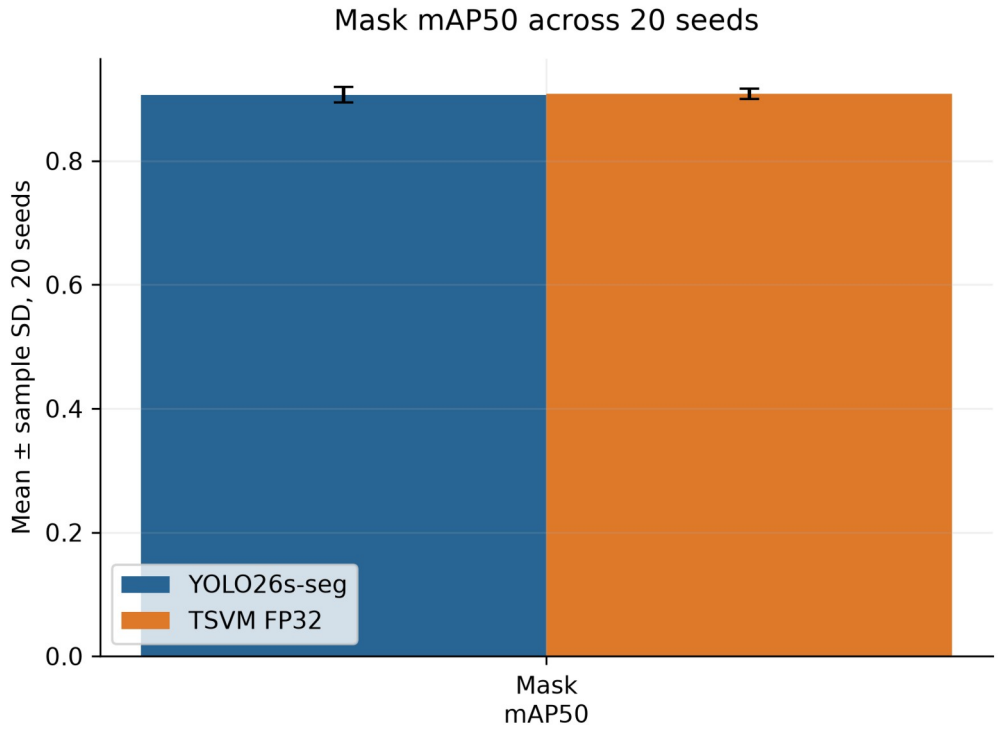


Hình 5.1 Trung bình và độ lệch chuẩn mask mAP50 95 của hai mô hình

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Hình 5.1 thể hiện đồng thời mức trung bình và biến thiên giữa seed. Độ cao hai cột gần nhau cho thấy chênh lệch tuyệt đối nhỏ so với toàn thang đo; các thanh sai số thể hiện SD của từng nhóm, không phải khoảng tin cậy của chênh lệch ghép cặp. Việc hai thanh sai số giao nhau không phải quy tắc quyết định p-value. Phần kiểm định ở mục 5.4 sử dụng trực tiếp 20 chênh lệch theo seed.

5.3.2 Đối chiếu mAP50 và mAP50 95



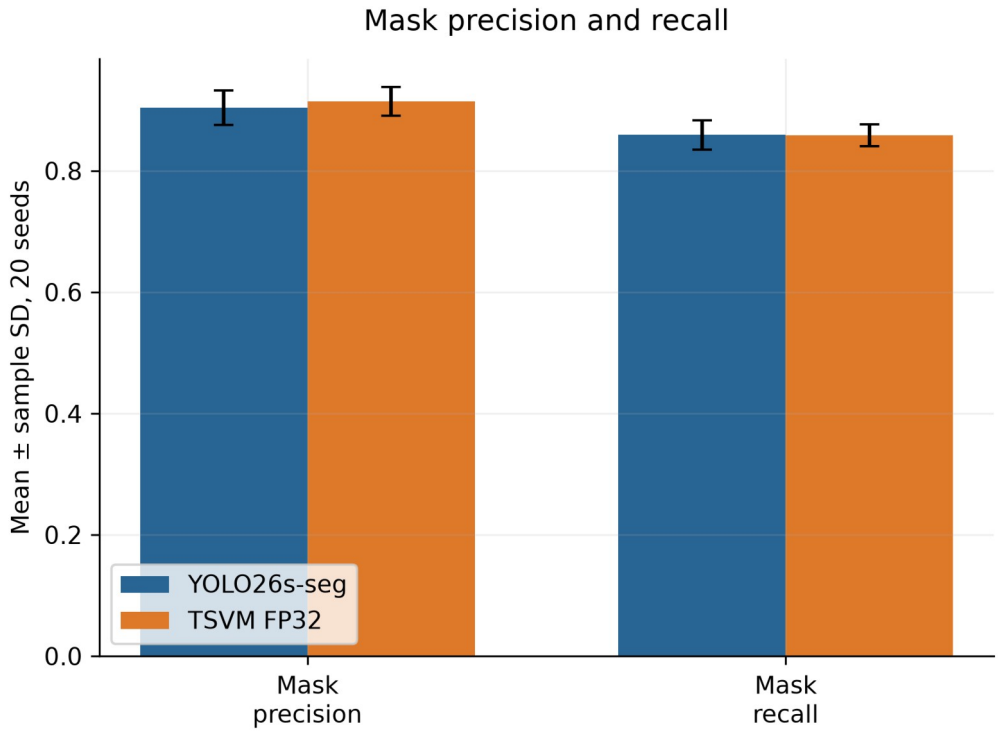
Hình 5.2 So sánh mask mAP50 trên 20 seed

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Mask mAP50 tăng từ 0,906889 lên 0,908274, tương ứng khoảng 0,1385 điểm phần trăm, nhỏ hơn mức tăng của mAP50–95. Trong phạm vi các chỉ số quan sát, TSVM có xu hướng cải thiện rõ hơn khi phép ghép yêu cầu mức IoU chặt hơn. Tuy nhiên, nhận xét này là mô tả hai độ đo tổng hợp; để khẳng định cơ chế cải thiện định vị cần xem AP tại từng ngưỡng IoU hoặc phân tích sai số mặt nạ của cùng checkpoint.

Giá trị mAP50 cao hơn mAP50–95 của cả hai nhóm là phù hợp với yêu cầu ghép dễ hơn ở IoU 0,50. Khoảng cách đó gợi ý vẫn còn dư địa nâng chất lượng định vị mặt nạ khi tăng độ chặt của phép đánh giá. Không thể từ khoảng cách này suy ra chính xác lỗi ở biên, ở vùng phản quang hay ở các polyp nhỏ nếu chưa phân nhóm ca và đo trực tiếp.

5.3.3 Mối quan hệ Precision và Recall



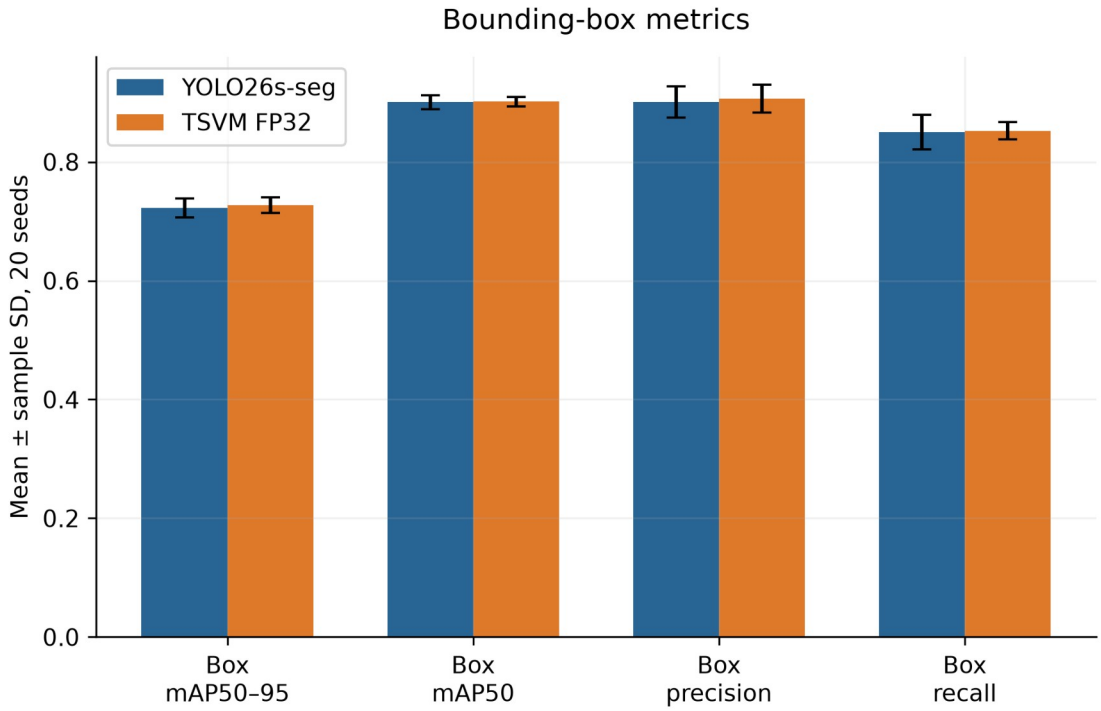
Hình 5.3 Mask Precision và Recall của hai nhóm thực nghiệm

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Mask Precision trung bình tăng từ 0,904095 lên 0,914452, trong khi mask Recall giảm nhẹ từ 0,859367 xuống 0,858979. Mức giảm Recall khoảng 0,0388 điểm phần trăm, rất nhỏ trong thiết lập hiện tại. Kết quả không hỗ trợ cách diễn đạt mọi chỉ số đều tăng; thay vào đó, nó mô tả một xu hướng Precision tăng cùng Recall gần như giữ nguyên về mức trung bình.

Precision và Recall ghi trong results.csv được lựa chọn theo cơ chế đánh giá của thư viện trên đường confidence. Chúng không nhất thiết là điểm làm việc confidence 0,25 đang dùng để minh họa trên ứng dụng. Vì vậy, không được dùng trực tiếp bảng này để tuyên bố tỷ lệ bỏ sót hoặc báo động giả của Web ở một ngưỡng cố định. Khi lựa chọn ngưỡng triển khai cần khảo sát lại đường precision –recall và mục tiêu sử dụng cụ thể.

5.3.4 Chất lượng hộp bao và quan hệ với mặt nạ



Hình 5.4 Các chỉ số hộp bao trên hai nhóm 20 seed

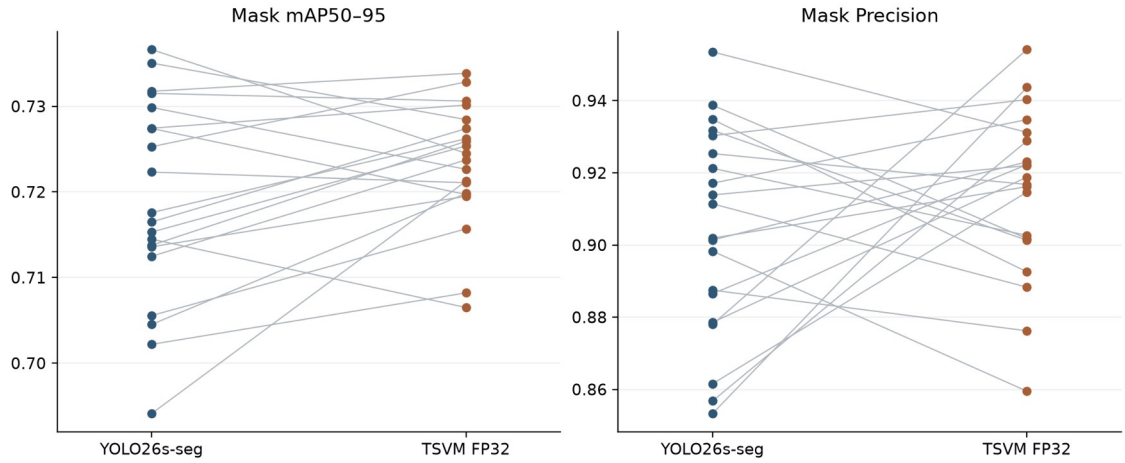
Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Box mAP50–95 tăng từ khoảng 0,7228 lên 0,7273; box mAP50 tăng từ khoảng 0,9006 lên 0,9014. Box Precision và Recall cũng tăng về trung bình. Các kết quả này cho thấy thay đổi backbone tác động đến phần biểu diễn chung của đầu phát hiện và phân đoạn, thay vì chỉ xuất hiện ở một lớp xử lý mặt nạ độc lập.

Tuy vậy, hộp bao tốt không bảo đảm mặt nạ bám sát toàn bộ tổn thương. Hai dự đoán có thể có hộp bao gần giống nhưng khác nhau về vùng lõm, vùng phản quang hoặc phần rìa. Cần đọc chỉ số hộp bao như một khía cạnh bổ sung cho kết quả mặt nạ, không dùng nó để thay cho bằng chứng về hình dạng hay topology.

5.4 CHÊNH LỆCH GHÉP CẶP VÀ ĐỘ BẤT ĐỊNH

5.4.1 Quan sát từng cặp seed



Hình 5.5 Kết quả ghép cặp mask mAP50 95 và Precision của 20 seed

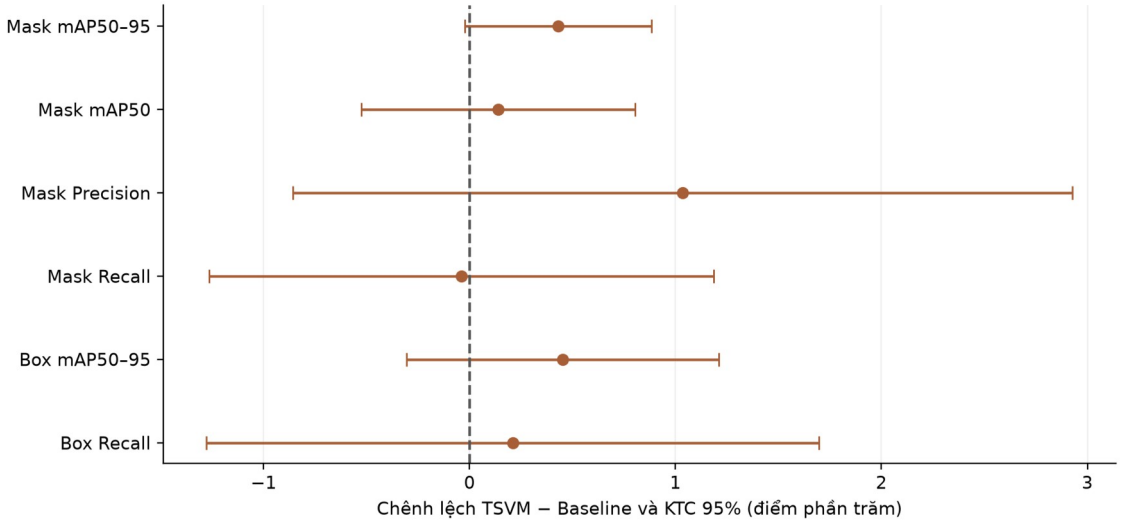
Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Mỗi đường nối trong Hình 5.5 biểu diễn cùng một chỉ số seed giữa hai kiến trúc. Có những cặp đi lên và những cặp đi xuống; vì vậy, một lượt chạy riêng lẻ có thể dẫn đến nhận xét khác với kết quả trung bình. Cách trình bày ghép cặp giữ lại các trường hợp TSVM thấp hơn baseline, giúp tránh chỉ chọn các seed có lợi cho mô hình đề xuất.

5.4.2 Khoảng tin cậy và ý nghĩa thống kê

Bảng 5.2 Khoảng tin cậy và kiểm định ghép cặp hai phía

Chỉ số	KTC 95% của Δ	p t-test	p Wilcoxon
Mask mAP50–95	[-0,0002; 0,0088]	0,0614	0,0696
Mask mAP50	[-0,0053; 0,0080]	0,6675	0,8124
Mask Precision	[-0,0086; 0,0293]	0,2662	0,4980
Mask Recall	[-0,0126; 0,0119]	0,9479	0,9256
Box mAP50–95	[-0,0031; 0,0121]	0,2275	0,1054
Box Recall	[-0,0128; 0,0170]	0,7709	0,9563
Val segmentation loss	[-0,0807; 0,0138]	0,1552	0,1769



Hình 5.6 Chênh lệch trung bình và khoảng tin cậy 95 phần trăm

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

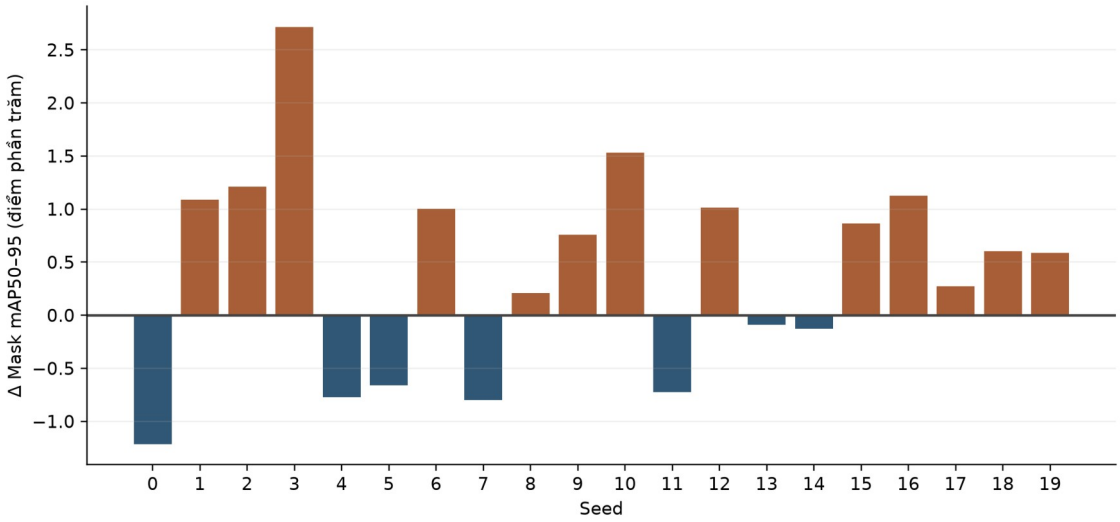
Đối với mask mAP50–95, kiểm định t ghép cặp cho p = 0,0614 và Wilcoxon cho p = 0,0696. Khoảng tin cậy 95% của chênh lệch là [−0,000228; 0,008834], chứa 0. Kết quả hiện phù hợp với xu hướng tăng trung bình nhưng chưa đủ bằng chứng ở ngưỡng 0,05 để kết luận TSVM có ưu thế trung bình trong thiết lập này.

Diễn giải theo điểm phần trăm, khoảng tin cậy trải từ khoảng −0,0228 đến +0,8834. Như vậy, dữ liệu vẫn phù hợp cả với một mức giảm rất nhỏ lẫn một mức tăng gần 0,9 điểm phần trăm. Cách đọc này cho thấy độ bất định về độ lớn tác động; nó có ích hơn việc chỉ ghi một dấu sao cạnh trung bình. Các khoảng trong Hình 5.6 cũng đều giao đường 0 đối với các độ đo thể hiện.

Việc tăng số seed làm rõ hơn biến thiên tối ưu hóa trên cùng tập chia nhưng không thay thế việc thu thập tập kiểm thử ngoài. Nếu tiếp tục thực nghiệm, nên xác định trước số seed, tiêu chí chính và quy tắc chọn checkpoint; không dừng ngay khi một p-value vừa đi qua ngưỡng 0,05.

5.5 PHÂN BỐ KẾT QUẢ VÀ ĐỘ ỔN ĐỊNH GIỮA SEED

5.5.1 Giá trị theo từng seed



Hình 5.7 Chênh lệch mask mAP50 95 của từng cặp seed

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

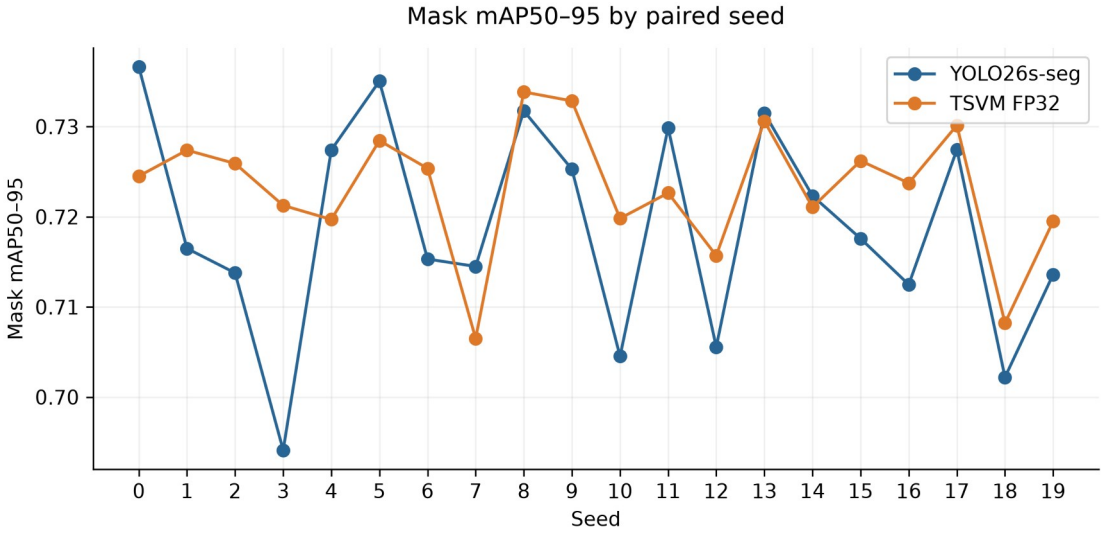
Bảng 5.3 Kết quả mask mAP50 95 và epoch theo từng seed

Seed	Epoch B	Epoch T	Baseline	TSVM FP32	Δ
0	89	88	0,73663	0,72451	-0,01212
1	98	98	0,71649	0,72739	0,01090
2	100	83	0,71377	0,72591	0,01214
3	61	89	0,69411	0,72126	0,02715
4	99	97	0,72741	0,71970	-0,00771
5	94	90	0,73505	0,72845	-0,00660
6	81	95	0,71531	0,72537	0,01006
7	71	88	0,71449	0,70650	-0,00799
8	96	93	0,73175	0,73385	0,00210
9	84	96	0,72528	0,73285	0,00757
10	82	100	0,70452	0,71982	0,01530
11	99	87	0,72987	0,72264	-0,00723
12	87	89	0,70556	0,71569	0,01013

Seed	Epoch B	Epoch T	Baseline	TSVM FP32	Δ
13	97	99	0,73151	0,73061	-0,00090
14	74	96	0,72232	0,72107	-0,00125
15	80	84	0,71758	0,72621	0,00863
16	96	93	0,71248	0,72374	0,01126
17	73	100	0,72744	0,73015	0,00271
18	94	47	0,70220	0,70821	0,00601
19	71	91	0,71359	0,71949	0,00590

TSVM có mask mAP50–95 cao hơn ở 13 trong 20 seed, tương ứng 65% số cặp quan sát; baseline cao hơn ở 7 seed. Kiểm định dấu hai phía cho p khoảng 0,2632. Tỷ lệ thắng này là thống kê mô tả của 20 cặp đang có, không phải xác suất đã biết rằng TSVM sẽ thắng trong một lần huấn luyện mới hoặc trên một tập dữ liệu mới.

Seed 3 có mức tăng lớn nhất, khoảng +0,02715, trong khi seed 0 có mức giảm lớn nhất, khoảng −0,01212. Ví dụ này cho thấy vì sao không nên chọn một seed thuận lợi để đại diện phương pháp. Seed 0 được tích hợp vào ứng dụng và benchmark theo quy tắc chọn một seed cố định cho cả hai nhóm; việc chọn nó không hàm ý TSVM tốt hơn baseline ở seed đó.

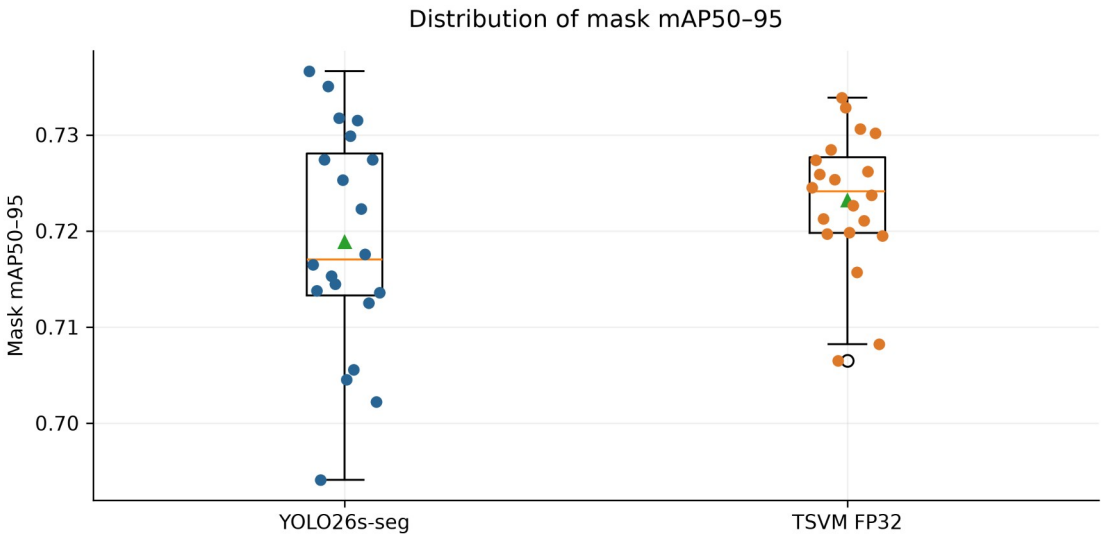


Hình 5.8 Diễn biến mask mAP50 95 theo chỉ số seed

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Trục seed trong Hình 5.8 chỉ là nhãn của các lượt độc lập, không phải thời gian. Đường nối giúp đối chiếu vị trí các giá trị nhưng không thể được diễn giải như một quá trình học liên tục qua seed. Hai đường đan xen phản ánh sự khác biệt tối ưu hóa giữa các kiến trúc; không có quy luật seed lớn hơn dẫn đến chất lượng tốt hơn.

5.5.2 Phân bố và khoảng biến thiên



Hình 5.9 Phân bố mask mAP50 95 của hai mô hình

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

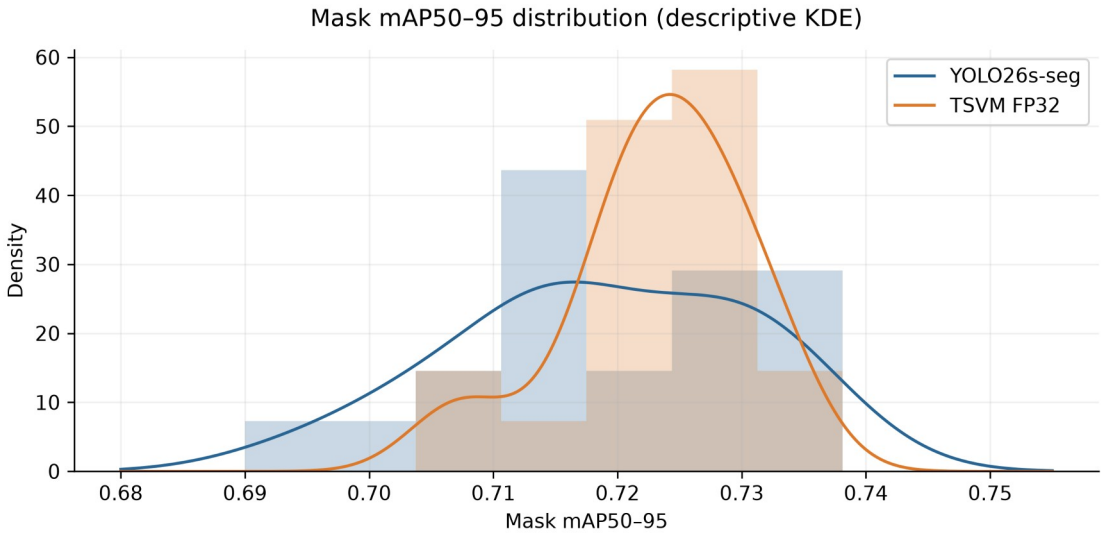
Bảng 5.4 Phân bố mask mAP50 95 theo nhóm

Mô hình	Min	Median	Max	Range
Baseline	0,69411	0,71703	0,73663	0,04252
TSVM FP32	0,70650	0,72413	0,73385	0,02735

Biểu đồ hộp cho thấy vùng tập trung và các giá trị nằm xa phần lớn quan sát. Baseline đạt cực đại cao hơn, nhưng cũng có cực tiểu thấp hơn. TSVM có khoảng phân tán hẹp hơn trong tập seed khảo sát. Nhận xét này phù hợp với SD nhỏ hơn, song cần đọc cùng các điểm dữ liệu: sự thu hẹp có thể chịu ảnh hưởng của một số seed thấp ở baseline, chứ không chứng minh mọi lượt TSVM đều tốt hơn.

Kiểm định Levene theo trung vị đối với mask mAP50–95 cho p khoảng 0,0257. Đây là phân tích thăm dò về phân tán; hai nhóm có tổ chức ghép seed và báo cáo xem nhiều chỉ số bổ sung. Do đó, kết luận chính vẫn dựa vào SD và phân

bố quan sát được, không xem một phép kiểm định phương sai riêng lẻ là chứng minh ổn định cho mọi cấu hình.



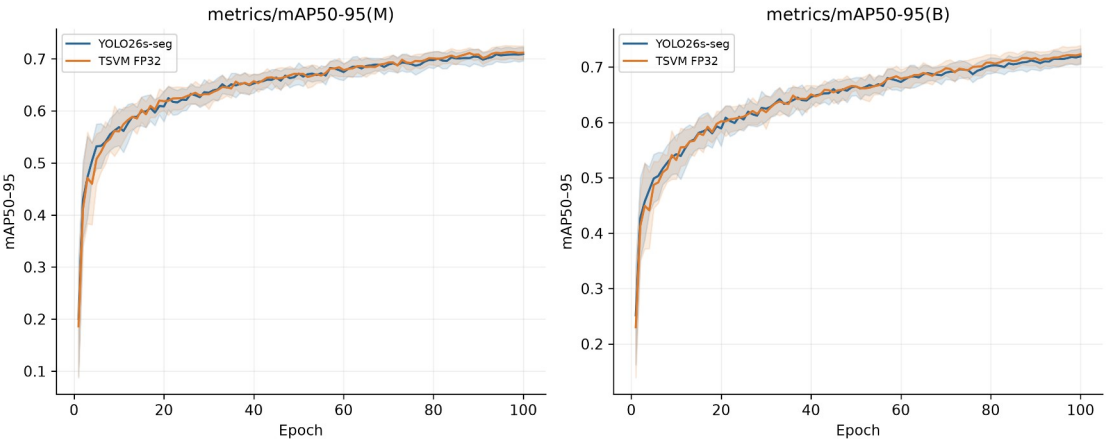
Hình 5.10 Phân bố tần suất mask mAP50 95 giữa các seed

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Hình 5.10 bổ sung góc nhìn về vùng giá trị tập trung. Với chỉ 20 quan sát mỗi nhóm, hình dạng histogram hoặc đường làm trơn phụ thuộc cách chia khoảng và tham số vẽ. Vì vậy, hình được dùng để quan sát phân bố, không để khẳng định dữ liệu tuân theo một phân phối xác suất cụ thể hoặc tồn tại nhiều nhóm con có ý nghĩa.

5.6 DIỄN BIẾN HUẤN LUYỆN VÀ MẤT MÁT KIỂM ĐỊNH

5.6.1 Đường mAP theo epoch



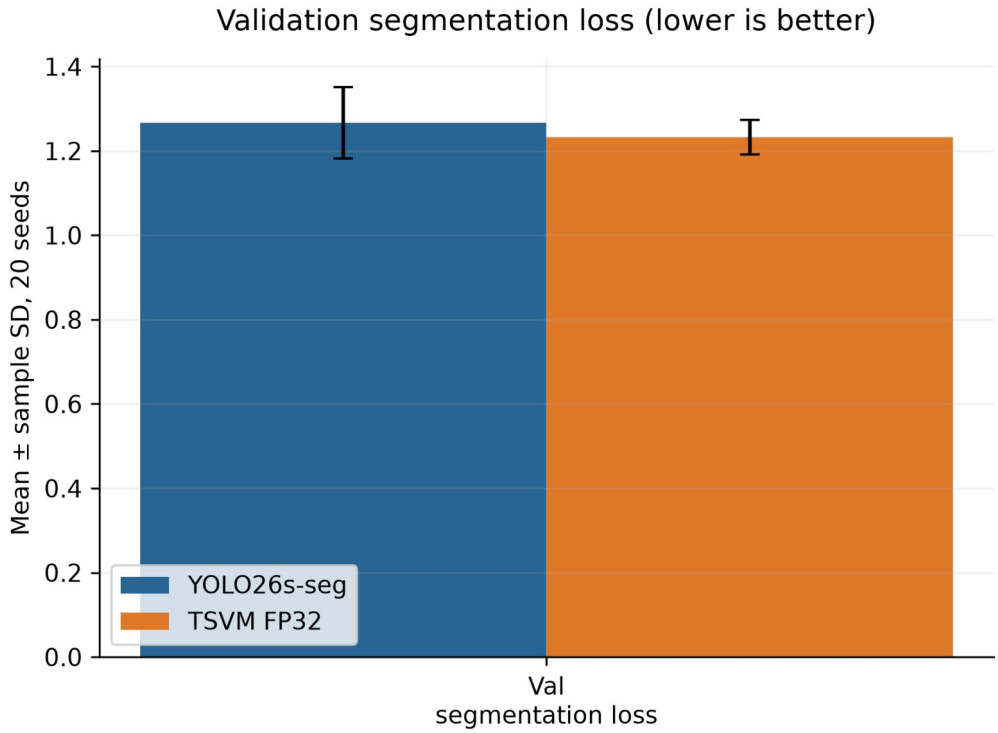
Hình 5.11 Trung bình và độ lệch chuẩn các đường mAP theo epoch

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Các đường trung bình được tính tại cùng epoch trên 20 lượt của mỗi mô hình. Chúng thể hiện giai đoạn chất lượng tăng nhanh ban đầu rồi tiếp tục thay đổi chậm hơn về cuối. Vùng tô là trung bình cộng trừ một SD giữa các seed ở từng epoch, không phải khoảng tin cậy của mô hình tốt nhất và cũng không phải kết quả của một lần chạy cụ thể.

Các đường mAP của hai nhóm gần nhau trên phần lớn quá trình. Vì bảng cuối chọn riêng epoch tốt nhất cho từng lượt, điểm trung bình trong bảng không nhất thiết trùng với giá trị cuối của đường trung bình. Một phép lấy trung bình cực đại và một phép lấy cực đại của đường trung bình là hai phép toán khác nhau. Không được dùng sự khác biệt đó như bằng chứng số liệu không nhất quán.

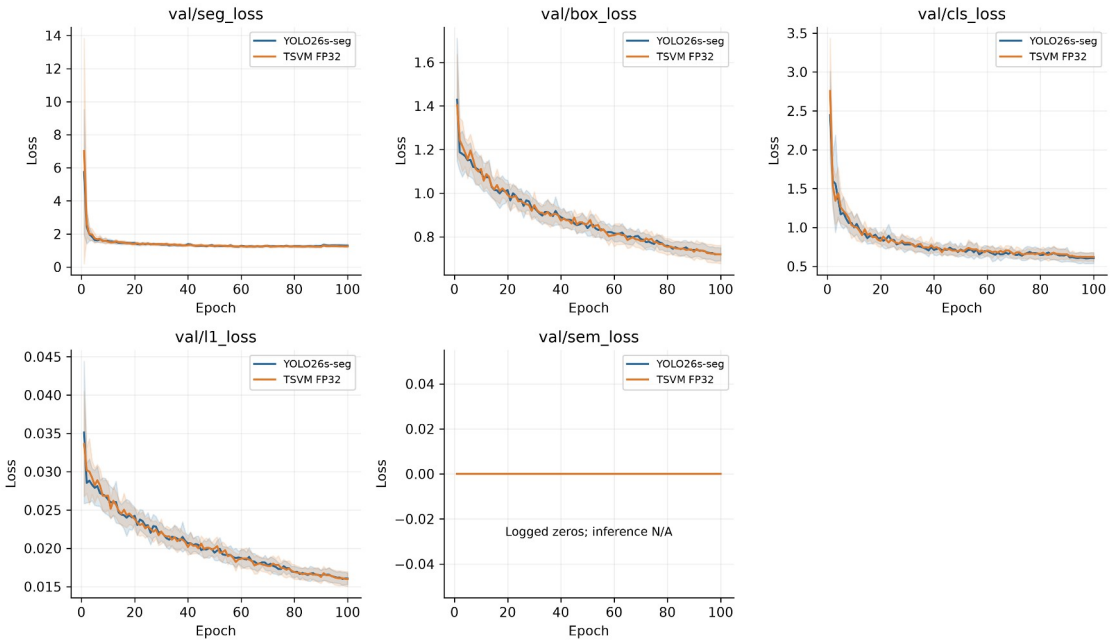
5.6.2 Validation loss



Hình 5.12 Validation segmentation loss tại epoch chọn theo mask

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Validation segmentation loss trung bình giảm từ 1,2663 xuống 1,2329, chênh khoảng −0,0334. Mức loss thấp hơn là dấu hiệu bổ sung về giá trị hàm mục tiêu tại các mốc đã chọn, nhưng kiểm định t ghép cặp cho p khoảng 0,1552 nên chưa có bằng chứng về khác biệt trung bình ở mức 0,05. Loss này không phải cực tiểu loss của mỗi lượt vì quy tắc chọn vẫn là mask mAP50–95.



Hình 5.13 Diễn biến các thành phần validation loss qua 100 epoch

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Các thành phần loss được quan sát riêng vì chúng có thang đo và ý nghĩa khác nhau. Giảm loss phân đoạn không tự động bảo đảm AP tăng ở mọi ngưỡng IoU; loss hộp bao hoặc phân lớp cũng không thể được cộng tùy ý để tạo một độ đo chất lượng mới nếu không theo đúng hàm huấn luyện. Báo cáo dùng các đường này để mô tả quá trình tối ưu và mức dao động, không quy đổi thành tỷ lệ cải thiện lâm sàng.

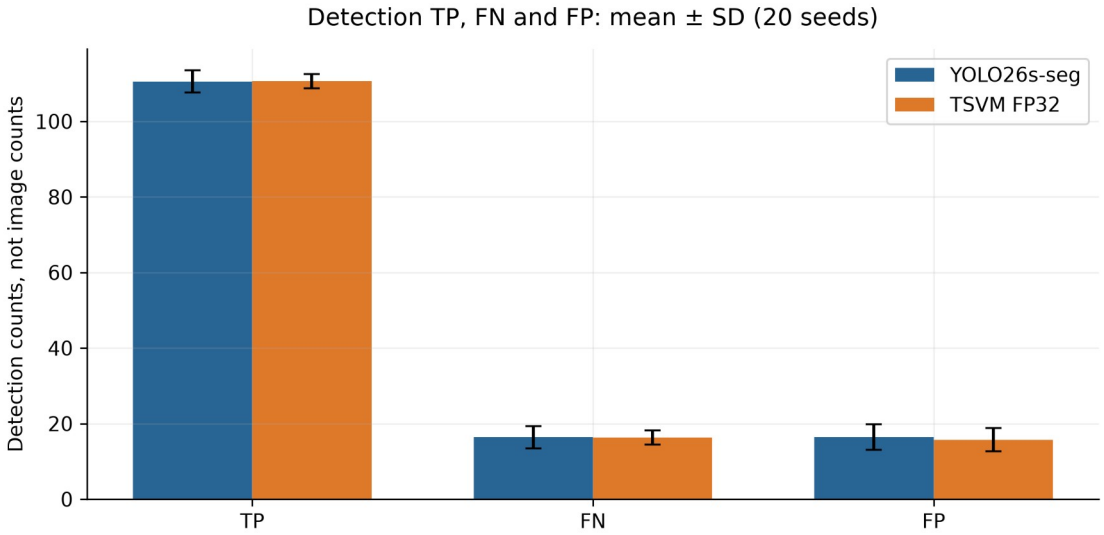
Việc tắt Mosaic trong 10 epoch cuối làm thay đổi phân bố ảnh đầu vào huấn luyện ở giai đoạn này. Tuy nhiên, chỉ từ một biến động trên đường loss không thể xác định nguyên nhân duy nhất là tắt Mosaic; ảnh hưởng đó cần một thực nghiệm kiểm soát riêng. Tương tự, epoch tối ưu muộn hơn không tự chứng minh mô hình hội tụ nhanh hoặc chậm hơn theo thời gian thực.

5.7 MA TRẬN NHẦM LẪN VÀ CÁC SAI SỐ PHÁT HIỆN

5.7.1 Số đếm đối tượng

Bảng 5.5 Thống kê bổ sung từ ma trận phát hiện theo đối tượng

Mô hình	TP	FP	FN
Baseline	110,55 ± 2,95	16,45 ± 3,43	16,45 ± 2,95
TSVM FP32	110,65 ± 1,87	15,75 ± 3,02	16,35 ± 1,87



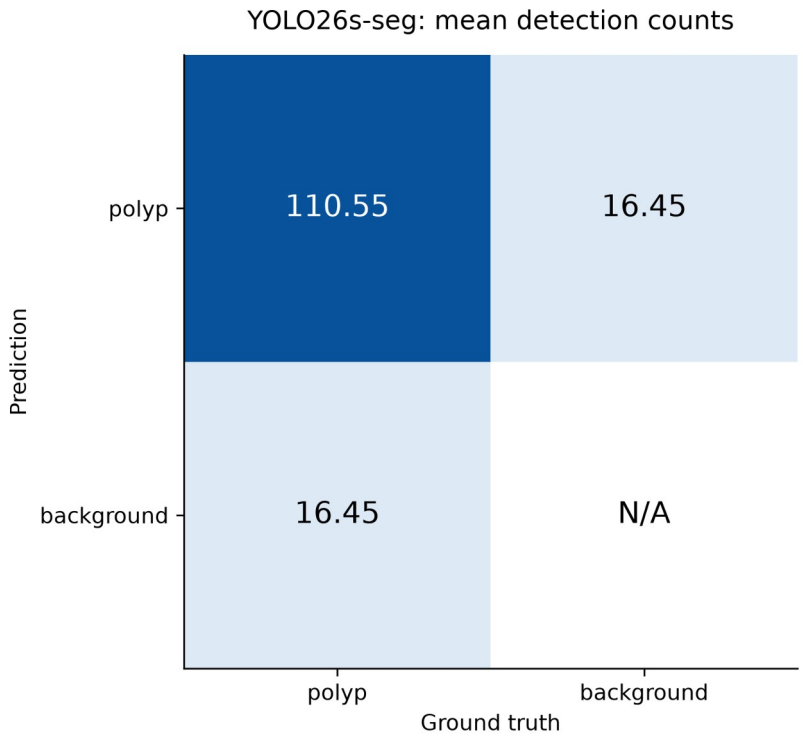
Hình 5.14 So sánh số đối tượng đúng sai và bỏ sót trung bình

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Số TP trung bình là 110,55 ở baseline và 110,65 ở TSVM FP32; FP giảm từ 16,45 xuống 15,75; FN giảm từ 16,45 xuống 16,35. Chênh lệch trung bình của các số đếm nhỏ so với mức phân tán giữa lượt. Chúng bổ sung góc nhìn về lỗi phát hiện ở cấu hình tạo ma trận, không phải một phép chứng minh riêng rằng mô hình đã giảm báo động giả theo ảnh.

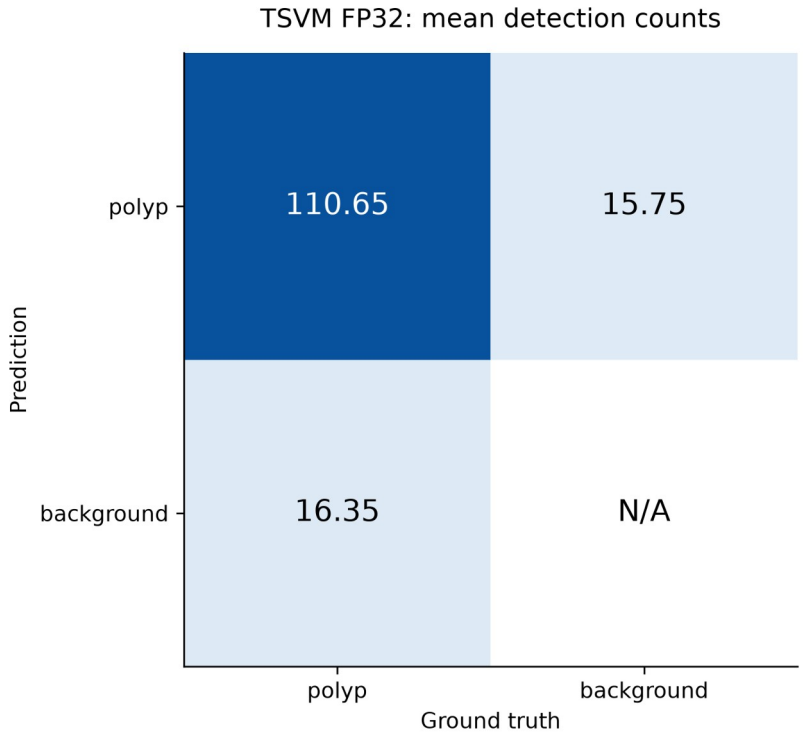
Các số liệu được trích từ hình ma trận nhầm lẫn lưu sau đánh giá checkpoint trong kết quả thực nghiệm; mốc checkpoint này có thể khác epoch tối đa mask được chọn trong bảng chính. Ngưỡng confidence và quy tắc ghép dùng để tạo ma trận có thể khác điểm làm việc được lựa chọn khi thư viện tính Precision/Recall trong results.csv. Do đó, không thể yêu cầu tỷ số của trung bình TP và FP khớp hoàn toàn với trung bình Precision ở bảng tổng hợp; hai số còn khác nhau về cách lấy trung bình.

5.7.2 Cách đọc các ô background



Hình 5.15 Ma trận số đếm trung bình của baseline

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.



Hình 5.16 Ma trận số đếm trung bình của TSVM FP32

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Trong ma trận phát hiện của Ultralytics, background không đóng vai trò một lớp đối tượng được gán hộp bao như lớp polyp. Ô tương ứng đối tượng thật nhưng không có dự đoán phù hợp biểu thị FN; ô dự đoán polyp không ghép với đối tượng thật biểu thị FP. Ô background–background không được tích lũy thành số ảnh âm dự đoán đúng và vì vậy được ghi không áp dụng trong các hình đã hiệu chỉnh.

Cột TN_reconstructed của bộ tổng hợp cũ được tạo bằng 40 trừ FP không hợp lệ cho bài toán này: FP là số đối tượng dự đoán sai, có thể xuất hiện trên ảnh có polyp, và một ảnh có thể chứa nhiều FP. Báo cáo không sử dụng cột đó để tính specificity, accuracy hai lớp hoặc tỷ lệ ảnh nền báo động giả. Các tệp gốc được giữ để truy vết; phần tổng hợp mới tách rõ chỉ số hợp lệ khỏi giá trị không sử dụng.

Nếu muốn đánh giá riêng 40 ảnh nền, cần suy luận từng ảnh với một ngưỡng cố định, ghi ảnh nào có ít nhất một dự đoán sai, rồi tính tỷ lệ ảnh báo động giả trên số ảnh nền. Để đánh giá bỏ sót theo ảnh cũng phải định nghĩa một ảnh có nhiều polyp được coi là thành công khi tìm ít nhất một hay tìm đủ tất cả. Những quy tắc này khác đánh giá theo đối tượng đang trình bày.

5.7.3 Đánh giá định tính và phạm vi hình minh họa

Hình suy luận tại Chương 4 sử dụng cùng ảnh, cùng quy trình RGB và các checkpoint thực sự được nạp vào ứng dụng. Nó xác nhận khả năng trả mặt nạ và metadata, nhưng hai ảnh minh họa không tạo thành một đánh giá chất lượng toàn tập. Confidence khác nhau trên cùng ảnh cũng không có nghĩa xác suất bệnh lý khác nhau; đây là điểm số của mô hình theo cấu hình dự đoán.

Để khảo sát nhận thức hình dạng, cần chọn trước các nhóm ca như polyp nhỏ, rìa mờ, phản quang hoặc nếp gấp, áp dụng cùng checkpoint và ngưỡng, rồi đối chiếu nhãn với dự đoán theo tiêu chí thống nhất. Mọi khẳng định TSVM bám biên hơn, ít đứt vùng hơn hoặc giữ topology tốt hơn cần phép đo và bộ ca tương ứng. Các chỉ số AP hiện có chỉ hỗ trợ giả thuyết để kiểm tra tiếp, không thay cho bằng chứng trực tiếp đó.

5.8 CHI PHÍ MÔ HÌNH VÀ BENCHMARK SUY LUẬN FP32

5.8.1 Quy mô tham số và checkpoint

Hai checkpoint có lần lượt 11.434.463 và 12.254.688 tham số. TSVM tăng 820.225 tham số, khoảng 7,17%, do thay đổi thành phần xử lý đặc trưng. Số

tham số không thay đổi chỉ vì chuyển kiểu số FP16 sang FP32; điều thay đổi là số byte biểu diễn và có thể là chi phí tính toán trên từng thiết bị.

Bảng 5.6 Quy mô hai checkpoint được tích hợp vào ứng dụng

Đại lượng	**YOLO26s seg**	**TSVM FP32**
Số tham số	11.434.463	12.254.688
Dung lượng tệp best.pt MiB	22,27	47,37
Seed minh họa	0	0
Kiểu suy luận	Float32	Float32
Benchmark CPU FP32	Đã đo tại Bảng 5.7	Đã đo tại Bảng 5.7
Bộ nhớ tham số FP32 MiB	43,62	46,75
Benchmark GPU	Chưa đo	Chưa đo

Dung lượng tệp baseline khoảng 22,27 MiB và TSVM FP32 khoảng 47,37 MiB. Không nên xem chênh lệch dung lượng này là mức tăng kiến trúc thuần túy, vì tệp checkpoint có thể chứa kiểu số, buffer và metadata khác nhau. Benchmark chuyển cả hai mô hình sang float32 trước khi đo, giúp so sánh suy luận trong cùng điều kiện số; dung lượng tệp trên đĩa vẫn được báo cáo theo đúng tệp gốc.

5.8.2 Giao thức đo và phạm vi thời gian

Benchmark được thực hiện trên CPU AMD Ryzen 7 250 với Radeon 780M Graphics, 8 lõi vật lý và 16 luồng logic, RAM hệ thống khoảng 15,31 GiB. Môi trường đo là Windows, Python 3.12.14, PyTorch 2.6.0+cpu và Ultralytics 8.4.103 được nạp thực tế từ môi trường dịch vụ AI. Phiên đo không có CUDA khả dụng. Đây là môi trường benchmark cục bộ, được ghi tách khỏi cấu hình notebook huấn luyện và mã fork lưu trong dự án.

Hai checkpoint seed 0 được kiểm tra SHA-256, lớp tùy biến và tensor hữu hạn trước khi đo. Cả hai chạy float32, tắt autocast, batch 1, kích thước tensor chính xác 1×3×640×640, rect=False, confidence 0,25, IoU 0,70 và max_det 300. Số luồng PyTorch được cố định ở 4 intra-op và 1 inter-op sau khởi tạo predictor, vì quá trình khởi tạo backend có thể tự thay đổi thiết lập số luồng. Mỗi mô hình chạy trong một tiến trình riêng, baseline trước rồi TSVM.

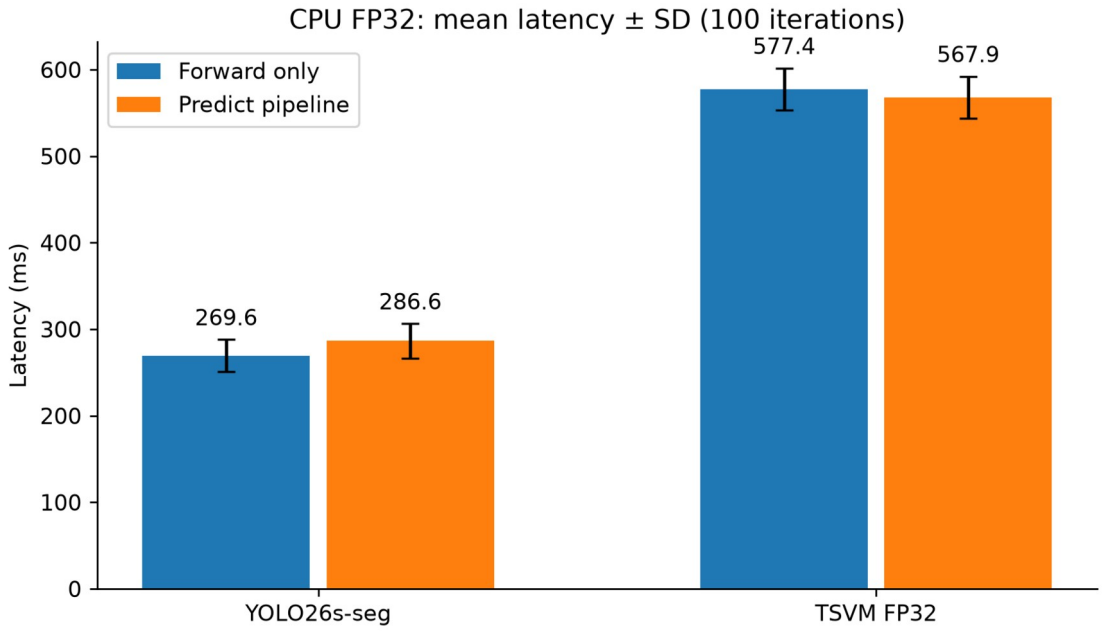
Mỗi chế độ có 20 lượt warmup và 100 lượt bấm giờ bằng perf_counter_ns trong torch.inference_mode. Chế độ forward only tái sử dụng tensor đã chuẩn hóa từ cùng một ảnh, chỉ bấm giờ thực thi mạng. Chế độ predict pipeline dùng 20 ảnh validation cố định gồm 10 ảnh polyp và 10 ảnh nền, giải mã PIL RGB trước vùng đo, rồi lặp 5 chu kỳ. Nó bao gồm tiền xử lý, forward và hậu xử lý mặt nạ thật của Ultralytics; không bao gồm đọc tệp, giải mã JPEG, HTTP, hàng đợi, vẽ overlay hoặc mã hóa Base64.

Mạng được đo sau bước chuẩn bị suy luận tiêu chuẩn của Ultralytics, gồm fusion Conv–BatchNorm và loại nhánh one2many khỏi đầu end-to-end. Số tham số kiến trúc được báo cáo trước fusion. Các mốc bấm giờ gốc, thông tin môi trường và danh sách ảnh có mã băm được lưu trong efficiency_benchmark/raw; bảng và biểu đồ được tái tạo bằng scripts đi kèm. Tổng cộng có 400 lượt đo thật, chia thành bốn nhóm 100 lượt.

5.8.3 Kết quả đo thực tế

Bảng 5.7 Kết quả benchmark CPU FP32 với 100 lượt đo mỗi chế độ

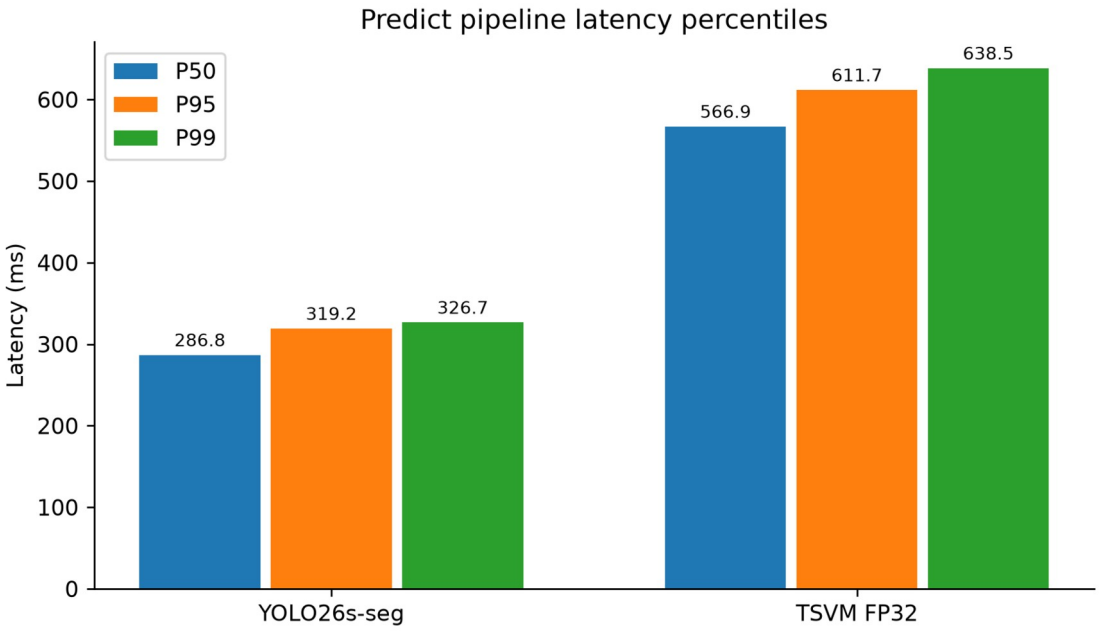
Đại lượng	**YOLO26s seg**	**TSVM FP32**
Forward mean ± SD ms	269,56 ± 18,71	577,36 ± 24,40
Forward p50 ms	270,97	576,84
Forward p95 ms	297,76	623,50
Forward p99 ms	308,62	636,82
Forward ảnh mỗi giây	3,71	1,73
Pipeline mean ± SD ms	286,56 ± 20,09	567,86 ± 24,45
Pipeline p50 ms	286,82	566,94
Pipeline p95 ms	319,22	611,67
Pipeline p99 ms	326,73	638,48
Pipeline ảnh mỗi giây	3,49	1,76



Hình 5.17 Độ trễ trung bình của forward và pipeline suy luận FP32

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

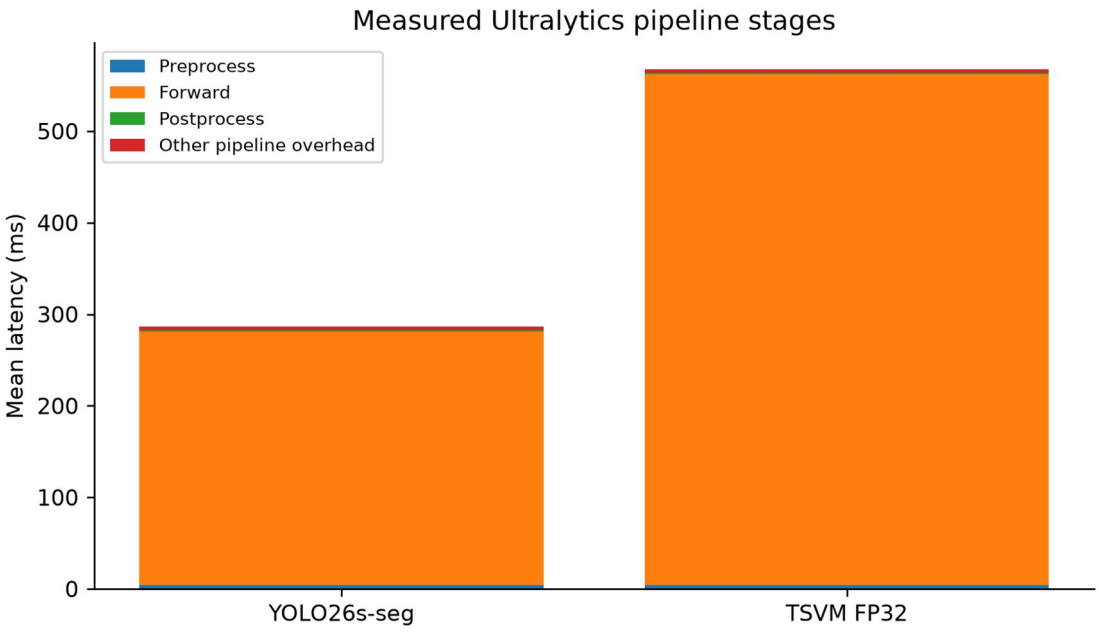
Forward only của baseline đạt trung bình 269,56 ms, còn TSVM FP32 đạt 577,36 ms, tương ứng chậm hơn khoảng 2,14 lần. Pipeline predict có trung bình 286,56 ms ở baseline và 567,86 ms ở TSVM FP32, chậm hơn khoảng 1,98 lần. Nghịch đảo của thời gian trung bình tương ứng khoảng 3,49 và 1,76 ảnh mỗi giây khi xử lý nối tiếp batch 1; đây không phải FPS đo trên video hoặc thông lượng khi chạy đồng thời nhiều yêu cầu.



Hình 5.18 Các phân vị độ trễ của pipeline predict

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Median pipeline lần lượt là 286,82 và 566,94 ms; p95 là 319,22 và 611,67 ms; p99 là 326,73 và 638,48 ms. Các phân vị mô tả những lượt chậm hơn trong cùng phiên đo, giúp tránh chỉ nhìn trung bình. Với 100 lượt mỗi nhóm, p99 chịu ảnh hưởng mạnh của một vài lượt lớn; muốn đặc tả độ trễ đuôi cho vận hành dài hạn cần số lượt và tải ứng dụng lớn hơn.

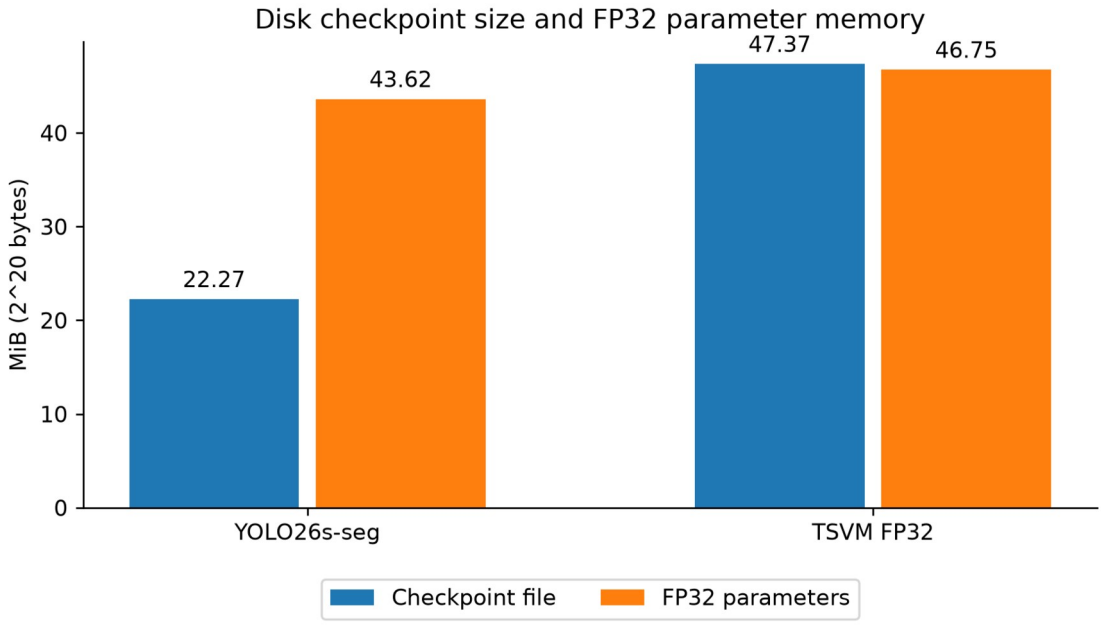


Hình 5.19 Các thành phần thời gian được đo thật trong pipeline

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Thời gian inference nội bộ trung bình khoảng 276,96 ms ở baseline và 558,06 ms ở TSVM; tiền xử lý khoảng 4,35 và 4,16 ms; hậu xử lý khoảng 1,10 và 1,46 ms. Phần còn lại giữa tổng các mốc nội bộ và đồng hồ bao ngoài là chi phí điều phối của pipeline. Các thành phần này lấy từ kết quả đo thực tế, không chia tỷ lệ giả định giữa backbone và head.

Forward only và pipeline là hai loạt đo với đầu vào và giai đoạn thực hiện khác nhau. Vì vậy, không lấy trung bình pipeline trừ trung bình forward only để ước tính hậu xử lý. Trường hợp TSVM có pipeline trung bình thấp hơn forward only một ít không biểu thị chi phí âm; nó phản ánh hai phân bố phép đo khác nhau. Chỉ các mốc nội bộ của cùng một lệnh predict mới được dùng để phân rã pipeline.



Hình 5.20 Dung lượng tệp và bộ nhớ tham số khi dùng float32

Nguồn: nhóm thực hiện từ mã nguồn hoặc số liệu của đề tài.

Bộ nhớ tham số float32 trước fusion khoảng 43,62 MiB ở baseline và 46,75 MiB ở TSVM, phù hợp với mức tăng số tham số 7,17%. RSS lớn nhất trong các mẫu lấy ngoài vùng bấm giờ lần lượt khoảng 511,97 và 545,60 MiB; đây là bộ nhớ toàn tiến trình được lấy mẫu, không phải peak allocator hoặc dung lượng riêng của mạng. Tệp baseline lưu một số tensor dạng FP16 nhưng được chuyển sang FP32 khi suy luận, nên kích thước tệp 22,27 MiB không phản ánh trực tiếp RAM tham số float32.

Benchmark không đưa ra GFLOPs tổng cho TSVM vì chưa có bộ đếm được kiểm chứng cho các phép quét tùy biến. Chỉ đếm convolution và linear sẽ bỏ sót

phần xử lý trạng thái. VRAM không áp dụng trong phiên CPU này, thay vì được ghi thành 0 MiB GPU. Thứ tự chạy cố định, nhiệt độ, quản lý điện năng và tác vụ nền có thể tác động số đo; 100 lượt timing không phải 100 seed độc lập để kiểm định chất lượng mô hình.

5.8.4 Ý nghĩa đối với ứng dụng

Hiệu năng được đọc cùng chất lượng: mức tăng AP trung bình nhỏ cần được cân nhắc với thời gian xử lý và bộ nhớ bổ sung. Một mô hình có số tham số tăng ít vẫn có thể chậm hơn nhiều nếu module sử dụng nhiều thao tác nhỏ, phép sắp xếp tensor hoặc vòng quét chưa được tối ưu trên thiết bị đang đo. Ngược lại, độ trễ CPU không đủ để suy ra tỷ lệ chậm hơn trên GPU.

Benchmark không bao gồm toàn bộ hành trình người dùng từ tải ảnh, truyền mạng, chờ hàng đợi, truy cập MongoDB đến hiển thị giao diện. Các số đo tầng mô hình và pipeline suy luận là căn cứ để tìm nút thắt, còn thời gian đáp ứng Web cần kiểm tra toàn trình riêng. Không dùng phép nghịch đảo một thời gian đơn lẻ để tuyên bố ứng dụng nội soi video thời gian thực.

5.9 ĐỐI CHIẾU VỚI KẾT QUẢ CÔNG BỐ

Bảng 5.8 Kết quả tham khảo do tác giả công bố trên Kvasir SEG

Nguồn và mô hình	Dice	IoU	Giao thức tóm tắt
Polyp-Mamba [17], Bảng 1	0,940	0,881	900 Kvasir + 550 ClinicDB train; ảnh 224
Topo-VM-UNetV2 [18], Bảng I	0,9195	0,8654	900 Kvasir + 550 ClinicDB train; ảnh 256
CSG-Mamba [19], Bảng 1	0,9220	0,8677	800/100/100 Kvasir; 3 seed; ảnh 256

Ghi chú: Polyp-Mamba lấy số ở bản PDF hội nghị, không trộn với biến thể độ phân giải khác trong phần phản hồi tác giả. Các kết quả này không cùng giao thức với BG20.

Các giá trị Dice/IoU trong Bảng 5.8 do tác giả công trình công bố, không phải mô hình được nhóm huấn luyện lại. Polyp-Mamba được lấy từ PDF hội nghị; Topo-VM-UNetV2 và CSG-Mamba được ghi đúng trạng thái tài liệu tương

ứng. Mỗi công trình sử dụng nguồn train, cách chia, kích thước ảnh và nhiệm vụ đầu ra riêng.

Kvasir-SEG nguyên bản và BG20 không có cùng thành phần mẫu, vì BG20 bổ sung ảnh normal-cecum không có đối tượng. Dice/IoU theo điểm ảnh cũng không cùng định nghĩa với mask mAP50–95 theo đối tượng. Do đó, bảng cung cấp bối cảnh cho mức độ hoàn thiện của nghiên cứu, không lập thứ hạng trực tiếp giữa TSVM và những công trình đó.

Muốn so sánh trực tiếp, cần áp dụng các phương pháp trên cùng tập chia, cùng nhãn gốc, cùng chính sách xử lý ảnh âm và cùng độ đo. Nếu hợp nhất instance mask thành một mặt nạ semantic để tính Dice, phải nêu quy tắc hợp nhất, confidence, ngưỡng mask và xử lý ảnh mà cả dự đoán lẫn nhãn đều rỗng. Không được quy đổi AP sang Dice bằng một hệ số hoặc công thức không tương ứng.

5.10 TỔNG HỢP BẰNG CHỨNG VÀ HẠN CHẾ

Kết quả 20 seed cho thấy TSVM FP32 có xu hướng tăng mask mAP50–95 trung bình và giảm độ phân tán so với baseline trên cùng tập kiểm định. Precision tăng về trung bình, Recall gần như giữ nguyên, còn validation segmentation loss thấp hơn. Tuy nhiên, khoảng tin cậy của chênh lệch độ đo chính vẫn chứa 0 và các kiểm định trung bình chưa đạt ngưỡng 0,05. Đây là mức bằng chứng phù hợp để mô tả một hướng cải tiến cần kiểm chứng thêm.

Tập kiểm định đã tham gia chọn epoch, chưa có tập test ngoài và chưa có định danh bệnh nhân để xác nhận chia độc lập theo bệnh nhân. Nguồn ảnh âm chỉ từ normal-cecum nên chưa bao phủ mọi mô lành. Việc không có ảnh trùng hoàn toàn theo SHA-256 không loại trừ các khung hình gần giống hoặc ảnh cùng bệnh nhân. Các giới hạn này ảnh hưởng trực tiếp đến phạm vi khái quát của kết quả.

Thực nghiệm chưa có ablation đủ seed để tách tác dụng SS2D, nhánh hình dạng và guidance. Chưa có phép đo topology, đường biên hoặc khoảng cách mặt nạ cho cùng checkpoint. Bởi vậy, không thể quy toàn bộ chênh lệch AP cho một thành phần hoặc dùng tên topology-aware làm bằng chứng bảo toàn cấu trúc. FP32 là yêu cầu triển khai số của checkpoint; ảnh hưởng riêng của kiểu số cũng cần một đối chứng được thiết kế riêng nếu muốn định lượng.

Phép benchmark mới bổ sung thông tin chi phí suy luận trên thiết bị và giao thức đã ghi nhận. Nó không chứng nhận hiệu năng GPU, khả năng tải đồng thời hay hiệu quả lâm sàng. Kết quả kiểm thử phần mềm xác nhận hành vi được kiểm

tra của dịch vụ và client, trong khi đánh giá khả năng ứng dụng trong nội soi thực tế cần dữ liệu, quy trình và thẩm định riêng.

5.11 TÓM TẮT CHƯƠNG

Chương đã đối chiếu 40 lượt huấn luyện, trình bày kết quả theo nhóm độ đo, phân tích từng seed, khảo sát diễn biến học và diễn giải ma trận đối tượng đúng phạm vi. Mask mAP50–95 tăng trung bình 0,4303 điểm phần trăm, nhưng chưa đủ bằng chứng về ưu thế trung bình ở mức 0,05. Bộ hình và bảng được liên kết với các thư mục kết quả có thể tái tạo, đồng thời benchmark FP32 được thực hiện và báo cáo tách khỏi số liệu chất lượng 20 seed.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

KẾT QUẢ ĐẠT ĐƯỢC

Đề tài đã xây dựng khối C2TSVMamba thay C2PSA ở tầng 10 của YOLO26s-seg và trình bày luồng tính toán giữa SS2D bốn hướng với nhánh hình dạng đa tỉ lệ, có hướng. Bộ BG20 được tổ chức đúng định dạng instance segmentation, sử dụng nhãn rỗng cho mẫu âm. Phần mô tả tiền xử lý được đối chiếu với tài liệu chuyên biệt, mã chuyển nhãn và cấu hình tăng cường dữ liệu.

Thực nghiệm chính gồm 40 lượt, mỗi lượt 100 epoch; số liệu và thống kê được trích lại nhất quán cho 20 cặp seed. Mask mAP50–95 tăng từ 0,718868 ± 0,011768 lên 0,723171 ± 0,007197. Tuy nhiên, paired t-test $p = 0,0614$ và khoảng tin cậy chứa 0 nên chưa kết luận ưu thế trung bình ở mức 0,05. Đây là kết quả cần được báo cáo cùng mức độ bất định, thay vì chỉ nêu một tỷ lệ tăng.

PolypVision đã được chuyển từ hai mô hình Nano của giai đoạn trước sang cặp checkpoint YOLO26s-seg và TSVM FP32. Dịch vụ đăng ký đúng module tùy biến, giữ suy luận float32 và truyền ảnh đúng quy ước RGB. Kết quả nạp và dự đoán trên ảnh minh họa xác nhận đường tích hợp kỹ thuật, trong khi các kiểm thử API và client xác nhận các hành vi phần mềm tương ứng.

Benchmark CPU FP32 với 20 lượt warmup và 100 lượt đo mỗi chế độ cho thấy pipeline predict của baseline trung bình 286,56 ms và TSVM 567,86 ms trong điều kiện cố định. Như vậy, chi phí suy luận tăng đáng kể so với mức tăng tham số 7,17%; đánh giá phương pháp cần xem đồng thời mức tăng AP nhỏ và độ trễ bổ sung. Những số đo này giới hạn ở thiết bị CPU và các checkpoint seed 0 đã công bố.

GIỚI HẠN CỦA KẾT LUẬN

Nghiên cứu chưa có tập test ngoài, ablation đầy đủ hoặc bằng chứng topology tường minh. Bộ chỉ số hiện có thiên về phát hiện từng đối tượng, nên chưa trả lời hoàn chỉnh câu hỏi về độ bám biên theo điểm ảnh. Ứng dụng phục vụ minh họa, chưa được đánh giá trong quy trình lâm sàng. Việc duy trì FP32 là điều kiện bảo toàn checkpoint và không tự chứng minh một cải tiến kiến trúc về độ chính xác.

HƯỚNG PHÁT TRIỂN

Ưu tiên đầu tiên là đánh giá lại cùng một tập kiểm định từ các checkpoint đã lưu, dùng chính xác tiêu chí chọn best.pt và lưu kết quả dạng JSON/CSV. Tiếp theo là đánh giá trên tập ngoài như ClinicDB, ColonDB và ETIS theo một giao thức thống nhất, đồng thời bổ sung các nhóm ảnh âm đa dạng hơn. Những phép đo này cần hoàn tất trước khi đưa ra kết luận về khái quát hóa.

Về phương pháp, cần chạy bốn chế độ ablation với ngân sách seed phù hợp, bổ sung độ đo biên và lỗi liên thông. Nếu tiếp tục dùng thuật ngữ topology-aware như một đóng góp chính, cần kiểm chứng bằng Betti errors hoặc phương pháp topology phù hợp. Về triển khai, có thể nghiên cứu chuẩn hóa đầu ra SS2D và kernel quét hiệu quả hơn; mọi tối ưu precision phải được kiểm tra lại trên checkpoint và chất lượng phân đoạn.

Về ứng dụng, hướng mở rộng gồm ghi metadata checkpoint trong từng bản ghi, tạo báo cáo so sánh có thể truy vết, và kiểm thử toàn trình với môi trường triển khai được cố định. Video, tải đồng thời và thời gian đáp ứng cần benchmark riêng. Các chức năng này là kế hoạch phát triển, không được cộng vào những kết quả đã hoàn thành của thực nghiệm ảnh tĩnh.

TÀI LIỆU THAM KHẢO

[1] D. Jha và cộng sự, "Kvasir-SEG: A Segmented Polyp Dataset," MMM, 2020, tr. 451 –462. DOI: 10.1007/978-3-030-37734-2_37. https://arxiv.org/abs/1911.07069

[2] K. Pogorelov và cộng sự, "Kvasir: A Multi-Class Image Dataset for Computer Aided Gastrointestinal Disease Detection," MMSys, 2017, tr. 164–169. DOI: 10.1145/3083187.3083212. https://datasets.simula.no/kvasir/

[3] D. Jha, "Kvasir-SEG Data: Polyp segmentation & detection," Kaggle. https://www.kaggle.com/datasets/debeshjha1/kvasirseg. Truy cập 15/10/2026.

[4] Ultralytics, "Instance Segmentation Datasets Overview." https://docs.ultralytics.com/datasets/segment/. Truy cập 15/10/2026.

[5] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[6] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 1979, tr. 62–66. DOI: 10.1109/TSMC.1979.4310076.

[7] R. C. Gonzalez và R. E. Woods, Digital Image Processing, ấn bản 4. Pearson, 2018.

[8] S. Suzuki và K. Abe, "Topological Structural Analysis of Digitized Binary Images by Border Following," Computer Vision, Graphics, and Image Processing, 30(1), 1985, tr. 32–46. DOI: 10.1016/0734-189X(85)90016-7.

[9] D. H. Douglas và T. K. Peucker, "Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or its Caricature," Cartographica, 10(2), 1973, tr. 112–122. DOI: 10.3138/FM57-6770-U75U-7727.

[10] A. Bochkovskiy, C.-Y. Wang và H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," 2020. https://arxiv.org/abs/2004.10934

[11] Ultralytics, "Data Augmentation using Ultralytics YOLO." https://docs.ultralytics.com/guides/yolo-data-augmentation/. Truy cập 15/10/2026.

[12] O. Ronneberger, P. Fischer và T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015. https://arxiv.org/abs/1505.04597

[13] D.-P. Fan và cộng sự, "PraNet: Parallel Reverse Attention Network for Polyp Segmentation," MICCAI, 2020. https://arxiv.org/abs/2006.11392

[14] A. Gu và T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023. https://arxiv.org/abs/2312.00752

[15] Y. Liu và cộng sự, "VMamba: Visual State Space Model," NeurIPS, 2024. https://arxiv.org/abs/2401.10166

[16] Ultralytics, “Ultralytics YOLO26” và mã cấu hình Segment26 của fork 8.4.127 sử dụng trong đề tài. https://docs.ultralytics.com/models/yolo26/. Truy cập 15/10/2026.

[17] Z. Xu và cộng sự, “Polyp-Mamba: Polyp Segmentation with Visual Mamba,” MICCAI, 2024. DOI: 10.1007/978-3-031-72111-3_48. https://papers.miccai.org/miccai-2024/paper/0697_paper.pdf

[18] D. Adame và cộng sự, “Topo-VM-UNetV2: Encoding Topology into Vision Mamba UNet for Polyp Segmentation,” arXiv preprint, 2025. https://arxiv.org/abs/2505.06210

[19] Y. Wang, J. Wu, J. Song và S. Ren, “CSG-Mamba: A Convolutional Scoring Gating Vision State Space Network for Endoscopic Polyp Segmentation,” arXiv preprint, 2026. https://arxiv.org/abs/2608.14146

[20] S.-W. Chan và cộng sự, “MambaPSA: A Mamba-based Replacement for C2PSA in YOLO26,” arXiv preprint, 2026. https://arxiv.org/abs/2607.12681

[21] T. K. Dutta, S. Majhi, D. R. Nayak và D. Jha, “Mamba Guided Boundary Prior Matters: A New Perspective for Generalized Polyp Segmentation,” 2025. https://arxiv.org/abs/2507.01509

[22] I. Loshchilov và F. Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019. https://arxiv.org/abs/1711.05101

[23] SciPy, “scipy.stats.ttest_rel.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_rel.html. Truy cập 15/10/2026.

[24] SciPy, “scipy.stats.wilcoxon.” https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wilcoxon.html. Truy cập 15/10/2026.

PHỤ LỤC A NGUỒN SỐ LIỆU VÀ QUY TẮC TÁI LẬP

Các đường dẫn dưới đây được tính tương đối từ thư mục archive để tránh phụ thuộc máy cá nhân. Hai nhóm chính chỉ lấy seed 0–19. Không thay thế thư mục TSVM_FP32 bằng nhóm TSVM không có hậu tố FP32.

Mã cơ bản: ultralytics/.

Mã cải tiến: ultralytics_Topology-Shape-aware_VMamba_FP32/.

YAML đề xuất: cfg/models/26/yolo26-seg-TopologyShapeVMamba.yaml trong mã cải tiến.

Kết quả cơ bản: Ket_Qua_V2/KetQua_Nen/YOLOv26s-seg/.

Kết quả đề xuất: Ket_Qua_V2/KetQua_Nen/Kvasir_BG20_YOLO26s_seg_TSVM_FP32/.

Tổng hợp chính: Ket_Qua_V2/KQ_Nen_DX_20seed_FP32/.

Notebook: Cell Kaggle/kvasir-yolo26s-seg.ipynb và kvasir-yolo26s-seg-topology-shape-awar.ipynb.

Tiền xử lý: Stracth/Du_Lieu/convert_kvasir_to_yolo_seg.py và Bao_cao/Tien_Xu_Ly_Du_Lieu.docx.

Thuật toán trích xuất: đọc results.csv, chuẩn hóa tên cột, chọn dòng đầu tiên đạt max metrics/mAP50-95(M), lấy các chỉ số khác trên cùng dòng, ghép hai kiến trúc theo seed và tính độ lệch chuẩn mẫu. best.pt không được tái định nghĩa bằng quy tắc max-mask này.

PHỤ LỤC B ĐỐI CHIẾU EPOCH VÀ CHECKPOINT

Bảng B.1 Epoch chọn theo mask và theo fitness của 40 lượt

Mô hình	Seed	Max mask	Max fitness
Baseline	0	89	89
Baseline	1	98	98
Baseline	2	100	100
Baseline	3	61	100
Baseline	4	99	100
Baseline	5	94	94
Baseline	6	81	98
Baseline	7	71	62
Baseline	8	96	96
Baseline	9	84	84
Baseline	10	82	81
Baseline	11	99	99
Baseline	12	87	87
Baseline	13	97	94
Baseline	14	74	88
Baseline	15	80	80
Baseline	16	96	96
Baseline	17	73	73
Baseline	18	94	84
Baseline	19	71	71
TSVM FP32	0	88	88
TSVM FP32	1	98	98
TSVM FP32	2	83	83

Mô hình	Seed	Max mask	Max fitness
TSVM FP32	3	89	89
TSVM FP32	4	97	97
TSVM FP32	5	90	90
TSVM FP32	6	95	98
TSVM FP32	7	88	70
TSVM FP32	8	93	99
TSVM FP32	9	96	100
TSVM FP32	10	100	100
TSVM FP32	11	87	87
TSVM FP32	12	89	88
TSVM FP32	13	99	100
TSVM FP32	14	96	96
TSVM FP32	15	84	81
TSVM FP32	16	93	93
TSVM FP32	17	100	100
TSVM FP32	18	47	47
TSVM FP32	19	91	90

PHỤ LỤC C CẤU HÌNH VÀ KIỂM TRA ỨNG DỤNG

Ứng dụng nằm tại DeepLearning/Ung_Dung/polypweb. Thư mục polypweb/ai-service là FastAPI, polypweb là Spring Boot và app_polyp là Flutter. Tệp models/experiment_manifest.json trong dịch vụ AI xác định checkpoint mặc định của từng model key.

YOLO26s-seg; seed 0; yolo26s-seg-best.pt

SHA-256: 61ebd02851f5a12921ab10f64ade433c745f87aac91fd41a310cd00c7d2a3906

YOLO26s-TSVM-FP32-seg; seed 0; yolo26s-tsvm-fp32-best.pt

SHA-256: b0578c7612e1e48cc490ef71eca5f025fb1eeb5a02c1fe6cb7e869d639e2c7a8

Để chạy dịch vụ AI, kích hoạt môi trường đã cài đúng dependencies, vào polypweb/ai-service và dùng python -m uvicorn main:app --host 127.0.0.1 --port 8000. Kiểm tra /health và /api/v1/models trước khi gửi ảnh. Với điện thoại thật, cấu hình host và AI_BASE_URL theo địa chỉ mạng truy cập được; không mặc định localhost của điện thoại là máy chủ.

Web được khởi chạy bằng Maven Wrapper trong thư mục polypweb sau khi cấu hình MongoDB và kho lưu ảnh. Mobile chạy trong app_polyp với flutter run --dart-define=AI_BASE_URL=http://địa_chỉ_máy_chủ:8000. Không đưa tài khoản, mật khẩu, khóa hoặc ảnh cá nhân vào tài liệu chia sẻ.

Để đổi checkpoint, dùng AI_BASELINE_MODEL_PATH và AI_VMAMBA_MODEL_PATH, đồng thời kiểm tra model key, kiến trúc C2TSVMamba, nhãn polyp, kiểu float32 và mã băm. Không nạp checkpoint Nano cũ dưới nhãn TSVM FP32 mới.

PHỤ LỤC D KẾ HOẠCH THỰC NGHIỆM BỔ SUNG

Bảng D.1 Các phép kiểm cần bổ sung trước khi mở rộng kết luận

Hạng mục	Thiết kế cần thực hiện	Đầu ra
Đánh giá checkpoint	Validate best.pt trên cùng dữ liệu và cấu hình	JSON/CSV gắn SHA-256
Ablation	Bốn mode, cùng seed và pretrained policy	Mean SD và Δ ghép cặp
Độ đo điểm ảnh	Quy định hợp nhất instance và ảnh rỗng	Dice IoU Boundary F1 HD95
Topology	Định nghĩa lỗi liên thông và lỗ	Betti error hoặc phép đo phù hợp
Tập ngoài	Không fine-tune trên tập test	ClinicDB ColonDB ETIS
Ảnh nền	Âm tính đa nguồn và cấp ảnh	False-alarm rate theo ảnh
Tốc độ mở rộng	Đo GPU và tải ứng dụng sau benchmark CPU	Median p95 và thời gian toàn trình

Các hạng mục trên là kế hoạch kiểm chứng, không phải các kết quả đã được cộng vào báo cáo hiện tại. Thứ tự ưu tiên là xác nhận checkpoint, đánh giá ngoài, phân tích ablation rồi mới tối ưu triển khai.